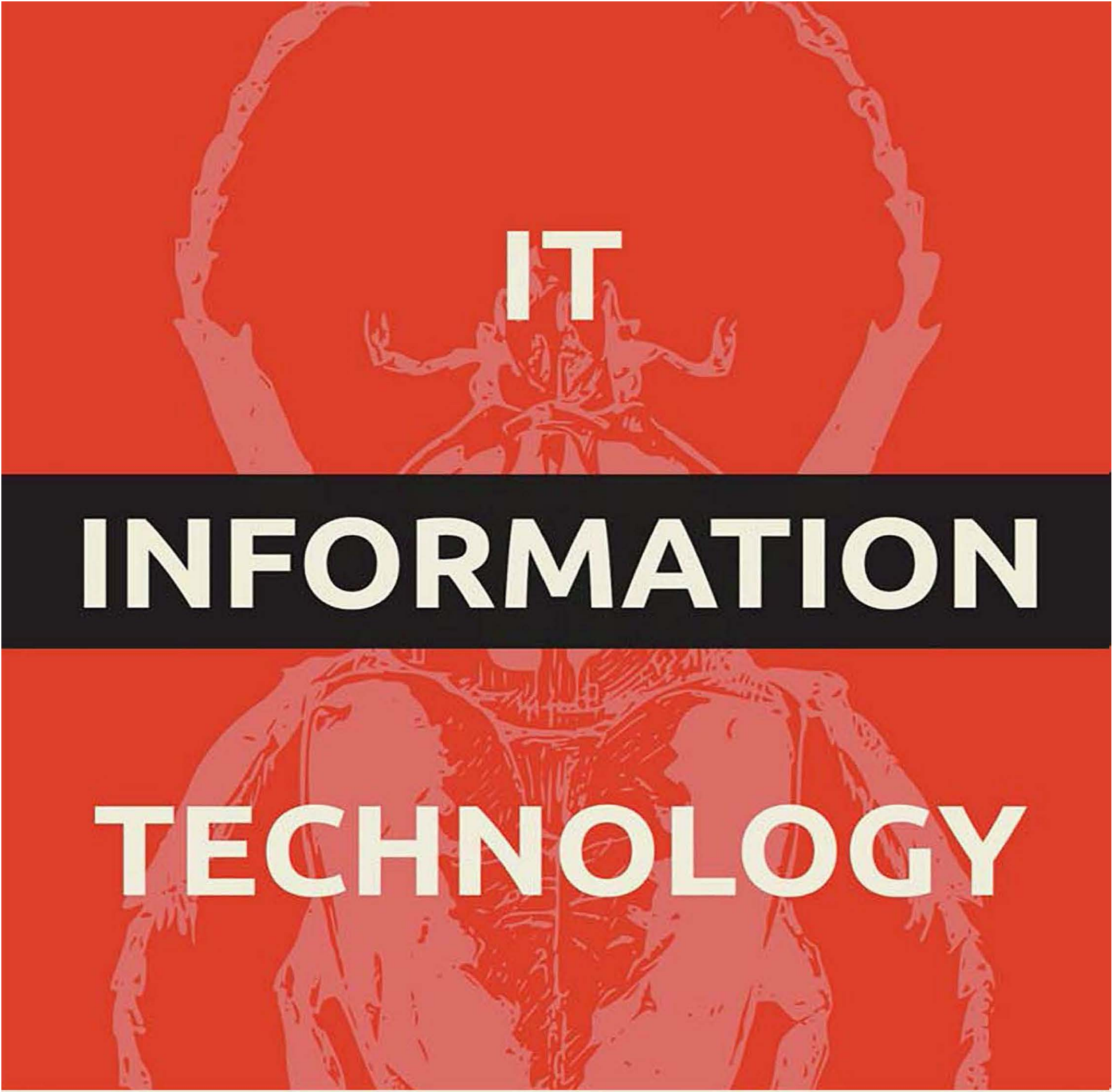




IT INFORMATION TECHNOLOGY

The Definitive Guide
LINUX

CONTENTS

[LINUX BASIC COMMANDS]	pg. 4 - 18
[BOOTABLE MEDIA]	pg. 19 - 42
- efibootmgr	pg. 21
- Installing rEFInd Boot Manager	pg. 21
- UEFI - Booting - Documentation - Extra	pg. 22 - 37
- Reinstall grub2 on UEFI systems ext4 & btrfs [HARDWARE]	pg. 38 - 42
- [Hardware] showing hardware details	pg. 43
[VIRTUALIZATION]	pg. 44 - 62
- virtualbox raw disk acces for version 6.1	pg. 46 - 48
- virtualbox raw disk acces for version 7	pg. 49 - 52
- KVM Virtual guest virtual cores (Enable Hyperthreading Virtual cores to guest kvm)	pg. 53
- PCI - passthrough with KVM/libvirt qemu/virtmanager	pg. 54 - 57
- Passthrough - Storage Devices	pg. 58 - 59
- How to mount qcow2/vdi disk image on Linux	pg. 60 - 61
- convert virtual file image formats	pg. 62
[SPARSE FILES]	pg. 63 - 65
[How to mount and read/write raw images [LOSETUP] (.img .iso ..etc) and other image file types on Linux]	pg. 66 - 73
[KERNEL BOOT PARAMETERS/KERNEL PARAMETERS]	pg. 74 - 79
[KERNEL MODULE] (autoload modules at bootup)	pg. 80 - 81
[DD COMMAND LINUX & SECURELY WIPE DISKS]	pg. 82 - 94
-dd command	pg. 82 - 86
-Securely Wipe Disks	pg. 87 - 94
[PERSISTENT BLOCK NAMING]	pg. 95 - 102
[TIPS & TRICKS]	pg. 103
- automatically add users to groups	pg. 103
- debconf (package installation automation)	pg. 104
- [TAR archiver & compressor]	pg. 105 - 107
- [7z Archiver & compressor]	pg 108 - 115
- reorder drive partition numbers	pg. 116 - 117

[NETWORKING] pg. 118 -

- **Linux internet sharing From one interface into another & LAN over Ethernet between two+ machines pg. 118**
- **Linux Connect two computers with ethernet cable & share files (via samba or other software) pg. 119 - 122**
- **Linux Change Ethernet NIC Speed & settings (ETHTOOL) pg. 123 - 127**
- **Linux Change Network Interface Name pg. 128**
- **Linux Add/Del Virtual Network Interface(s) to a Physical NETIFACE (VLAN) pg. 129**
- **Linux Commands To View & Edit Network Information pg. 130**
- **Linux Virtual Wi-Fi interfaces for simultaneous use of a Wi-Fi adapter in different modes [milosderov] pg. 131 - 143**

LINUX BASIC COMMANDS



linuxjourney.com



Command line / Terminal

pwd: (present working directory)

syntax: pwd

Shows which directory you are currently in.

cd: (change directory)

syntax: cd </directory/location>

Change the current working directory to another specified directory.
Each time you interact with your command prompt, you are working within a directory.
There are two different ways to specify a path, with absolute and relative paths:

- Absolute path: This is the path from the root directory. The root directory is commonly shown as a slash. Every time your path starts with / it means you are starting from the root directory. For example, /home/pete/Desktop. (if the path starts with a slash (/) it is the absolute path to the directory)
- Relative path: This is the path from where you are currently in filesystem. If I was in location /home/pete/Documents and wanted to get to a directory inside Documents called taxes, I don't have to specify the whole path from root like /home/pete/Documents/taxes, I can just go to taxes/ instead.

When used without any argument, cd will take you to your home directory.

cd shortcuts:

- .. (parent directory). Takes you to the directory above your current.
- ../../ to move two levels up (move two parent directories up...or more..etc)
- ../<directory> move one level up and enter a directory in that level
- ~ (home directory). This directory defaults to your home directory
- ~/<directory> navigate to a directory inside your home directory
- - (previous directory). This will take you to the previous directory you were just at.

```
$ cd ..  
$ cd ../../  
$ cd ../Downloads  
$ cd ~  
$ cd ~/Documents  
$ cd -
```

Directories with space in their names:

If the directory you want to change to has spaces in its name, you should either surround the path with quotes or use the backslash (\) character to escape the space:

```
cd 'dir with spaces'  
cd dir\ with\ spaces
```



ls: (list directories contents)

syntax: `ls <options>`

The ls command will list directories and files in the current directory by default, however you can specify which path you want to list the directories of.

- `ls`
- `ls /deirectory`

Also note that not all files in a directory will be visible, Filenames that start with `.` are hidden, you can view them however with the ls command and pass the `-a` flag to it (a for all)

- `ls -a`

show a detailed list of files in a long format. This will show you detailed information, starting from the left: file permissions, number of links, owner name, owner group, file size, timestamp of last modification, and file/directory name.

- `ls -l`
- `ls -la` (to show all hidden and in long format)

touch: (create files)

syntax: `touch <filename.file>`

Touch allows you to the create new empty files.

Touch is also used to change timestamps on existing files and directories

- `touch filename.file`
- `touch /directory/location/filename.file`

You can use touch to create more than one empty file at a time:

- `touch file1 file2 Directory/file3`

You can auto-generate filenames:

- `touch filename{1..10}.txt`
- `touch filename{1..10}.txt Downloads/filename{1..5}.txt`

file: (determine file type)

syntax: `file <file>` or `file <options> <file>`

file is used to recognize the type of a file. It is very useful when you need to find out the filetype of a file which does not have a file extension (filename extension) or with a file extension thats different from the actual filetype. In Linux, filenames aren't required to represent the contents of the file

You can also pass the multiple files at once to get type of given files:

- `file doc text2`

find the type of all files in a particular directory: (use `*` wildcard)

- `file Downloads/*`
- `file *.txt`
- `file *`

Command line / Terminal



cat: (concatenate file(s) and print to the standard output)

syntax: `cat <file(s)>` or `cat <options> <file(s)>`

cat command allows us to view contents of a file, create single or multiple files, concatenate files and redirect output in terminal or files.

cat the contents of a file into another file: (will create a new file if it doesn't already exist)

- `cat file1 > file2`
- `cat file1 >> file2`
- `cat file1 | tee file2`
- `cat file1 | tee -a file2`
- `cat file1 | sudo tee file2`
- `cat file1 | sudo tee -a file2`

The `>` is a redirection operator that allows us to change where standard output goes. It allows us to send the output of `echo Hello World` to a file instead of the screen. If the file does not already exist it will create it for us. However, if it does exist it will overwrite it.

The `>>` redirection operator will append to the end of the file. If the file doesn't already exist it will create it for us. However, if it does exist it will append the output to the end of the file.

The `|` (pipe) allows us to get the stdout of a command and make that the stdin to another process. It takes the output of a command and makes it the input of another command.

The `tee` command reads standard input (stdin) and writes it to both standard output (stdout) and one or more files. `tee` is usually part of a pipeline, and any number of commands can precede or follow it.

View contents of a file(s):

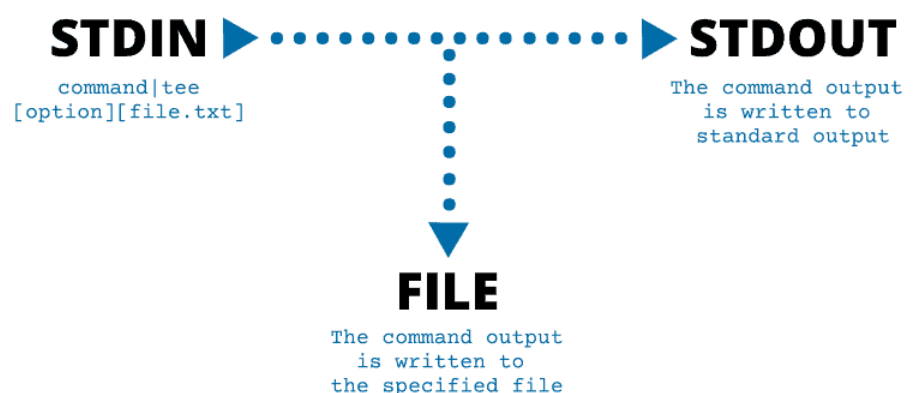
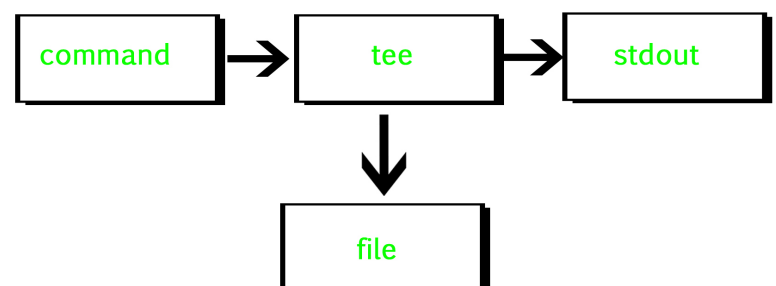
- `cat file1`
- `cat file1 file2`

Redirecting Multiple Files to a Single File:
(combining files)

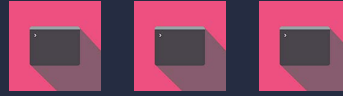
- `cat file1 file2 > file3`
- `cat file1 file2 >> file3`
- `cat file1 file2 | tee file3`
- `cat file1 file2 | tee -a file3`

****Note:** you can also use the command `echo` to write to a file:

```
echo "text" > file
echo "text" >> file
```



Command line / Terminal



less: (view files/very large files)

syntax: `less <file>` or `less <options> <file>`

The text is displayed in a paged manner, so you can navigate through a text file page by page.

Use the following command to navigate through less:

- `q` - Used to quit out of less and go back to your shell.
- Page up, Page down, Up and Down - Navigate using the arrow keys and page keys.
- `g` - Moves to beginning of the text file.
- `G` - Moves to the end of the text file.
- `/search` - You can search for specific text inside the text document. Prefacing the words you want to search with `/`
- `h` - If you need help about how to use less while you're in less, use help.
- `+F` - monitor files in real time
- `<#>g` - move to the `#` line of the output... e.g. `6g` moves to the 6th line
- you can view multiple files with `less <file1> <file2>` etc...

history: (display the history of the commands that you have previously entered)

syntax: `history` or `history <options>`

This program keeps track of everything that we type into the command line.

Allows you to review and repeat your previous commands.

Show a numbered list of all entered commands:

- `history`

Run the previous command:

- `!!`
- `sudo !!`

Running a command from the history list:

- `!<#>` ...e.g. `!37` .. will run the 37th command

Delete a command from history:

- `history -d <#>`

Delete entire command history

- `history -c`

Search for command in history with `grep`:

- `history | grep <search-command>`

Search for a command in history interactively:

- in terminal ,press `CTRL+R`
you can modify the desired command found in interactive mode by pressing left/right arrow keys

Command line / Terminal



cp: (copy)

syntax: `cp <source> <destination>` or `cp <options> <source> <destination>`

copy files and directories.

* if the destination doesn't exist, then first it creates it and content is copied to it, but if you copy a file over to a directory that has the same filenames, the file will be overwritten with whatever you are copying over. if you have a file that you don't want to get accidentally overwritten. You can use the `-i` flag (interactive) to prompt you before overwriting a file

Copy a file to another directory:

- `cp file /destination/location/`

Copy multiple files to a destination:

- `cp file1 file2 file3 directory`

You can copy multiple files and directories as well as use wildcards. A wildcard is a character that can be substituted for a pattern based selection, giving you more flexibility with searches. You can use wildcards in every command for more flexibility.

- `*` the wildcard of wildcards, it's used to represent all single characters or any string.
- `?` used to represent one character
- `[]` used to represent any character within the brackets

e.g. `$ cp *.jpg /home/pete/Pictures` `$ cp * /home/place/`

Copy a directory to another location (`-r` flag):

- `cp -r directory/ /other/directory/location`

If you want to be prompted before overwriting a file, use the `-i` (interactive) option:

- `cp -i mycoolfie /home/pete/Pictures`

You can combine flags:

- `cp -ir directory /other/directory`
- `cp -ir directory/* /other/directory`

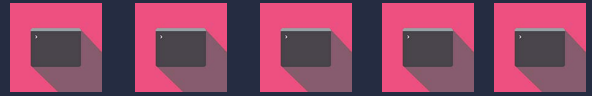
Update (`-u` flag), copy only when the SOURCE file is newer than the destination file or when the destination file is missing:

- `cp -u source_file destination`
- `cp -iru source/directory/* destination/location`

Archive (`-a` flag), copies files/directories and preserves ownership, time-stamps attributes.

- `cp -a source destination`
- `cp -a source/* destination`

Command line / Terminal



mv: (move)

syntax: `mv <source> <destination>`

Used for moving files, directories or/and also renaming them. if you mv a file or directory it will overwrite anything in the same directory with the same name, So you can use the -i flag to prompt you before overwriting anything.

move a file or/and directory to a different location:

- `mv file_1 /dir/location/`

Rename a file/directory:

- `mv name_old name_new`
- `mv /place/name_old /dir/location/name_new`

Prompt you for confirmation before overwiting anything named the same (-i flag):

- `mv -i /place/* /other/place/`
- `mv -i file1 /location/`

Do not overwrite anything (-n flag):

- `mv -n TEST dir1/TEST`

mkdir: (make directory)

syntax: `mkdir <dir_name>` or `mkdir <options> <dir_name>`

Used for creating directories. By defuault creates a directory in the current working directory, unless specified to create it in another directory location.

Create a directory:

- `mkdir dir_name`
- `mkdir /location//dir_name`

Create multiple directories at once:

- `mkdir dir1 dir2 dir3`

Create subdirectories/parent directories, within a newly created directory (-p flag):

- `mkdir -p books/hemmingway/favorites`

Create a directory and set the directories permissions simultaneously (-m flag):

- `mkdir -m 777 dir_name`
- `mkdir -m a= rwx dir_name`

Command line / Terminal



rm: (remove)

syntax: `rm <file>` or `rm <options> <file>`

rm command is used to remove or delete the files or directories. By default, the rm command will not remove the directories; you need to use the `-r` flag to delete directories.

Delete a file:

- `rm file1`

Delete a directory and all its contents (`-r` flag):

- `rm -r dir1`

Remove multiple files:

- `rm file1 file2 file3`

Prompt before every removal (`-i` flag):

- `rm -i file`
- `rm -ir dir`
- `rm -ir /location/*`

Force deletion of files or directories, whether they are write protected or not (`-f` flag):

- `rm -f file`
- `rm -rf /location/*`
- `rm -rfi dir/*`

find: (find files)

syntax: `find [where to start searching from] [-options] [what to find]`

Find files and directories in the system, and can perform subsequent operations on them. It supports searching by file, folder, name, creation date, modification date, owner and permissions. By using the `'-exec'` other UNIX commands can be executed on files or folders found.

find a file

- `find /home -name file22`



locate: (find/search files)

syntax: `locate <file_name>`

locate command in Linux is used to find the files by name. locate works better and faster than find command counterpart because instead of searching the file system when a file search is initiated, it would look through a database.

By default, locate command does not check whether the files found in the database still exist and it never reports files created after the most recent update of the relevant database. If it doesn't find a file, it doesn't necessarily mean that the file does not exist on the system. The locate command works on an index i.e. a database of file locations. When you use the command, it refers to this index instead of searching the entire filesystem. This is the reason why locate is super quick at finding files in Linux.

Using locate command is dead simple. You just have to specify the file name:

- `locate <filename>`

perform a case-insensitive search (-i flag):

- `locate -i <filename>`

using **updatedb** to create your own index for locate command:

updatedb; updates file name databases used by GNU locate. The file name databases contain lists of files that were in particular directory trees when the databases were last updated. The file name of the default database is determined when locate and updatedb are configured and installed. The frequency with which the databases are updated and the directories for which they contain entries depend on how often updatedb is run, and with which arguments.

create an index of a specified directory (and its subdirectories and files), you can use the updatedb command like this:

- `updatedb --localpaths='<desired_directory>' --output=<location_of_indexfile_name>`

e.g. `updatedb --localpaths='/mnt/XTGO-EXT4' --output=/mnt/XTGO-EXT4/dbfile`

where:

`--localpaths='path1 path2...'`

Non-network directories to put in the database. Default is /.

`--output=dbfile`

The database file to build. Default is system-dependent. In Debian GNU/Linux, the default is /var/cache/locate/locatedb.

Once you have created this index file, you can ask locate command to explicitly use this index:

- `locate -d index_file file_name`



help: (help)

syntax: <command> --help

the help command displays information about commands.

e.g: Display a brief help description of the command cp.

- cp --help

man: (manual pages)

syntax: man <command>

man command in Linux is used to display the user manual of commands (commands that have a manual page available for it, most do). It provides a detailed view of the command and its usage.

e.g.: Display manual pages for mv command:

- man mv

whatis: (one line manual page description of a command)

syntax: whatis <command>

the "whatis" command is used to offer a one-line overview of command, option, or a flag

e.g. get a whatis description of mkdir command

- whatis mkdir

alias: (create a custom reference to another command)

syntax: alias <custom_name>='<custom-command>'

The shell alias is simply a way to reference another command. It can be used to avoid repetitive long typing of commands among other things.

There are many advantages of writing alias and keeping them in user property files like .bashrc and .profile file in users home directories. Some of the advantages of alias as listed below:

- 1) Time-saving: We can save a good amount of time by shortening the length of a command so that we dont need to type as many characters.
- 2) We dont need to remember bigger command, we just create a short name for it (alias) and use it.
- 3) We can run multiple commands (using command chaining ;) with just single alias.
- 4) We can use functions to make our alias more powerful, though we do not use alias command directly.

How to define an alias:

- alias a='cmd'
- alias a='cmd1;cmd2'

How to define an alias permanently:

The above-mentioned method will not keep your alias across reboots and logouts we have to make these alias available all the time. To do that we have to edit our shell profiles files(~/.bashrc for BASH shell) and give alias entries in it as shown below.

- \$vi ~/.bashrc
Give entries as below to bottom of ~/.bashrc file
- alias ll='ls -l'

Unset an alias (unalias):

- unalias <alias_name>

list currently defined aliases:

- alias



apropos: (searches the manual pages names and descriptions for a keyword or regular expression.)

syntax: `apropos <keyword>` or `apropos <options> <keyword>`

Each manual page has a short description available within it. `apropos` searches the descriptions and names for instances of keyword.

examples:

- `apropos compress`
- `apropos grub`

env: (environment variables)

In Linux based systems environment variables are a set of dynamic named values, stored within the system that are used by applications launched in shells or subshells. In simple words, an environment variable is a variable with a name and an associated value.

Environment variables allow you to customize how the system works and the behavior of the applications on the system. For example, the environment variable can store information about the default text editor or browser, the path to executable files, or the system locale and keyboard layout settings.etc...

There are several commands available that allow you to list and set environment variables in Linux:

- `env` – The command allows you to run another program in a custom environment without modifying the current one. When used without an argument it will print a list of the current environment variables.
- `printenv` – The command prints all or the specified environment variables.
- `set` – The command sets or unsets shell variables. When used without an argument it will print a list of all variables including environment and shell variables, and shell functions.
- `unset` – The command deletes shell and environment variables.
- `export` – The command sets environment variables.

List environment variables:

The most used command to displays the environment variables is `printenv`, but you can also use `echo ${env_name}`.

For example, to display the value of the `HOME` environment variable with `printenv` you would run:

- `printenv HOME`

To display the current user in the terminal session (`$USER`) environment variable using `echo`:

- `echo $USER`

To display the directories of `$PATH`, `PATH` environment variable:

- `echo $PATH`
- `printenv PATH`

To see environment variables for current terminal session:

- `env`
- `printenv`



File Permissions



Linux File/Directory Permissions:

In Linux, file permissions, attributes, and ownership; control the access level that the system processes and users have to files. This ensures that only authorized users and processes can access specific files and directories.

For effective security, Linux divides authorization into 2 levels.

1. Ownership
2. Permission

The basic Linux permissions model works by associating each system file with an owner and a group and assigning permission access rights for three different classes of users.

Ownership of Linux Files:

Every file and directory on your Unix/Linux system is assigned 3 types of owner, given below:

- **user** (the file owner): user is the owner of the file. By default, the person who created a file becomes its owner. Hence, a user is also sometimes called owner. The ownership can be changed as well (chown command).
- **group** (the group members): A user- group can contain multiple users. All users belonging to a group will have the same Linux group permissions access to the file. Every user is part of a certain group(s). A group consists of several users and this is one way to manage users in a multi-user environment. The group ownership can be changes as well (chgrp command).
- **others** (everybody else): Any other user who has access to a file. This person has neither created the file, nor he belongs to a usergroup who could own the file. Practically, it means everybody else. Hence, when you set the permission for others, it is also referred as set permissions for the world.

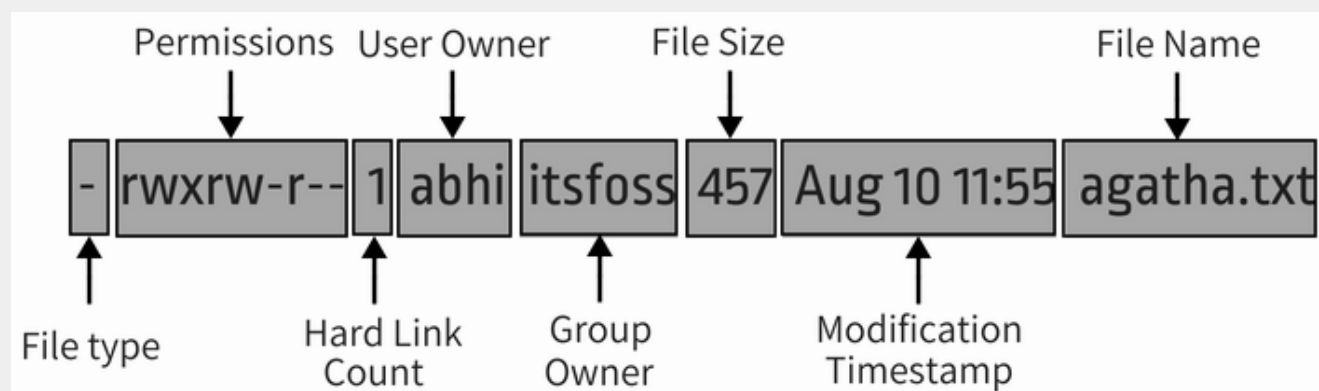
Permissions:

Every file and directory in your UNIX/Linux system has following 3 permissions defined for all the 3 owners discussed above.

- **read:** the read permission gives you the authority to open and read a file. Read permission on a directory gives you the ability to lists its content copy the files from directory.
- **write:** the write permission gives you the authority to modify the contents of a file. The write permission on a directory gives you the authority to add, remove and rename files stored in the directory. Consider a scenario where you have to write permission on file but do not have write permission on the directory where the file is stored. You will be able to modify the file contents. But you will not be able to rename, move or remove the file from the directory.
- **execute:** The execute permission: In Unix/Linux, you cannot run a program unless the execute permission is set. If the execute permission is not set, you might still be able to see/modify the program code (provided read & write permissions are set), but not run it.

Viewing file permissions:

You can view the permissions of file(s)/directory(ies) by using the; `ls -l` or `ll` ;comands.



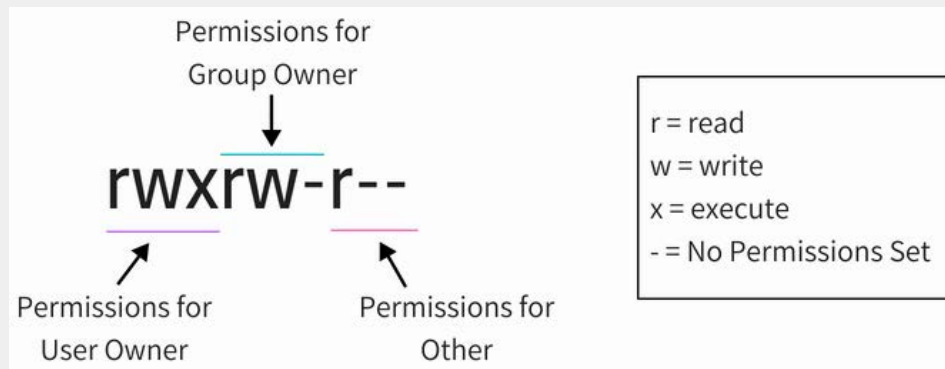
- **File type:** Denotes the type of file. d means directory, – means regular file, l means a symbolic link.
- **Permissions:** This field shows the permission set on a file.
- **Hard link count:** Shows if the file has hard links. Default count is one.
- **User:** The user who owns the files.
- **Group:** The group that has access to this file. Only one group can be the owner of a file at a time.
- **File size:** Size of the file in bytes.
- **Modification time:** The date and time the file was last modified.
- **Filename:** Obviously, the name of the file or directory.



File Permissions



In the previous example, you see the file permissions in three triplets of three characters each. The first triplet shows the owner permissions, the second one group permissions, and the last triplet shows everybody else permissions.



Permissions are always in the order of read, write and execute i.e., rwx. And then these permissions are set for all three kind of owners in the order of User, Group and Other.

Changing file/directory permissions (chmod command):

We can use the **chmod** command which stands for 'change mode', for changing the permissions on a file/directory in Linux. There are two ways to use the chmod command:

1. **Absolute mode**
2. **Symbolic mode**

I will only cover absolute mode in detail because its my preffered method.

Absolute (Numeric) Mode:

In the absolute mode, permissions are represented in numeric form (octal system to be precise). file permissions are represented in a three-digit octal number.

- r (read) = 4
- w (write) = 2
- x (execute) = 1
- - (no permission) = 0

The table below gives numbers for all for permissions types.

Number	Permission Type	Symbol
0	No Permission	---
1	Execute	--x
2	Write	-w-
3	Execute + Write	-wx
4	Read	r--
5	Read + Execute	r-x
6	Read +Write	rw-
7	Read + Write +Execute	rwx

Using chmod to change permissions of a file/directory:

syntax: **chmod <numeric_value#> file**

- **chmod 777 file**



File Permissions



Changing ownership and group (chown command):

To change the ownership of a file/directory, you can use the chown command.

change the user ownership of a file using chown command:

- `chown <username> <file>`

change the user ownership as well as group of a file using chown command:
If you add a colon and groupname after the user you can set both the user and group at the same time.

- `chown <username>:<group_name> <file>`

change the user ownership of directories, their subdirectories and files; (-R option) operate on files and directories recursively.

- `chown -R <username> <dir>`
- `chown -R <username>:<groupname> <dir>`

Change group ownership (chgrp command):

To change the group-owner only, use the chgrp command.

change only the group owner of a file/directory, using chgrp command:

- `chgrp <group> file`



Block devices/Storage devices



Linux block devices, also referred to as storage devices:

lsblk: (list all block devices details or a specific block devices details)
syntax: `lsblk` or `lsblk <specific_device>`

`lsblk` can be used with options (`lsblk <option>`):

- **`lsblk -f` :**

Outputs info about filesystems. This option is equivalent to `-o NAME,FSTYPE,LABEL,UUID,FSAVAIL,FSUSE%,MOUNTPOINT`.

- **`lsblk -f <block_device>`:**

Outputs info about a specific filesystem. This option is equivalent to `-o NAME,FSTYPE,LABEL,UUID,FSAVAIL,FSUSE%,MOUNTPOINT`.

`lsblk` usage examples:

- `lsblk /dev/sdX`
- `lsblk`
- `lsblk -f`
- `lsblk -f /dev/sdX`

blkid: (block id)

syntax: `blkid` or `blkid | grep sdX`

locate/print block device attributes. In order to use `blkid` for a specific device we pipe (`|`) and `grep` the desired device.
e.g.

obtain `blkid` for `/dev/sda`:

- `blkid | grep sda`

fdisk :

syntax: `fdisk -l` or `fdisk -l <device>`

Display or manipulate a disk partition table. you can use `fdisk` to format block devices and to display block devices partition table information among other things.

List information for all block devices (`-l` option):

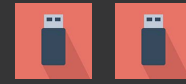
- `fdisk -l`

List information for a specific block device (`-l` option):

- `fdisk -l <device>`
- `fdisk -l /dev/sdX`



Block devices/Storage devices



udevadm info :

syntax : `udevadm info <block_device>`

udevadm info : Query the udev database for device information.

Basic usage example:

- `udevadm info /dev/sdX`

persistent block device naming:

There are four different schemes for persistent naming: **by-label**, **by-uuid**, **by-id** and **by-path**. For those using disks with GUID Partition Table (GPT), two additional schemes can be used **by-partlabel** and **by-partuuid**.

You can also use static device names by using Udev.

The directories in **/dev/disk/** are created and destroyed dynamically, depending on whether there are devices in them.

In order to list a devices, Persistent block device name:

- `ll /dev/disk/by-label`
- `ll /dev/disk/by-uuid/`
- `ll /dev/disk/by-partlabel/`
- `ll /dev/disk/by-partuuid/`

- `ll /dev/disk/by-label | grep sdX`
- `ll /dev/disk/by-uuid/ | grep sdX`
- `ll /dev/disk/by-partlabel/ | grep sdX`
- `ll /dev/disk/by-partuuid/ | grep sdX`

- `ll /dev/disk/by-id/`
- `ll /dev/disk/by-path/`

****NOTE:** **by-id** creates a unique name depending on the hardware serial number, **by-path** depending on the shortest physical path (according to sysfs). Both contain strings to indicate which subsystem they belong to (i.e. pci- for by-path, and ata- for by-id), so they are linked to the hardware controlling the device. This implies different levels of persistence: the **by-path** will already change when the device is plugged into a different port of the controller, the **by-id will change** when the device is plugged into a port of a hardware controller subject to another subsystem.

Thus, **both are not suitable to achieve persistent naming tolerant to hardware changes.(by-id & by-path)**

However, both provide important information to find a particular device in a large hardware infrastructure. For example, if you do not manually assign persistent labels (by-label or by-partlabel) and keep a directory with hardware port usage, by-id and by-path can be used to find a particular device.

BOOTABLE MEDIA

What is Bootable Media?

Bootable Media provides users with an alternative method to boot a system. Bootable media may be any bootable device, such as but not limited to:

- USB drive
- HDD/SSD
- SD card
- CD/DVD

A Bootable Device can be used to run a Live OS or install operating systems.

* [A Bootable Device can be multiboot (many OS in a single USB) or single boot]

Software Used to Create Bootable/Multiboot Media:

Bootable media can be created by Downloading the ISO file of the operating systems or tools you wish to use & then using the program of your choice that creates bootable media.

Bootable Media Creation Tools can be organized in two categories:
Multiboot & SingleBoot

Multiboot:

- - ventoy: (Available for Linux & Windows)
- - yumi: (Available for Windows)
- - easy2boot: (Available for Windows)

Single Boot:

- - etcher: (Available for Linux, Windows, Mac) <https://etcher.io>
- - rufus: (Available for Windows) <https://rufus.ie/>
- * Linux dd command

[and many more]

*You can also extract the iso file to a fat32 partition and it should boot

Boot manager(s):

- * refind boot manager

Booting From Bootable Media:

Turn on the computer and press the key(s) to access the boot menu (sometimes referred to as boot manager)

Normally the key(s) to access the boot menu can be:

F1-F12 (function keys), esc (escape key), backspace or other key(s) combination

*Consult device manual or research device name, brand & model to find the correct key(s) to access boot menu

***If bootable media is not recognized by the system you may need to disable secure boot !!!!**

UEFI native booting: how it works [overview]

EFI encompasses the use of FAT32 file-system partition, and FAT12 or FAT16 for removable media in a GPT (GUID Partition Table) partition table Formated drive. (however I always use FAT32 for everything just fine)

{this FAT boot partition is referred to as ESP (EFI System Partition) }

to see the type of partition table type of a storage device (along with other information) use:

\$ fdisk -l </dev/device>

(disklabel type)

UEFI Directory Structure

An EFI system partition that is present on a hard disk must contain an EFI defined directory in the root directory. This directory is named EFI. All OS loaders and applications will be stored in subdirectories below EFI.

subdirectories below EFI will be of two types: fallback path [BOOT] & vendor defined [any-vendor-name] each with an OS-loader.efi file

UEFI Partition Discovery:

/EFI/BOOT/BOOTX64.EFI : Fallback bootloader Path

This is the only bootloader pathname that the UEFI firmware on 64-bit X86 systems will look for without any pre-existing NVRAM boot settings

Firmware dynamically adds entries in the boot manager NVRAM automatically
{only true if the fallback path exists for that device /EFI/BOOT/BOOTX64.EFI}
once the device is disconnected the entry is deleted

* If theres no fallback path, firmware will not show a boot entry for that partition automatically

if the /EFI/any-vendor/OS-loader.efi file is not asociated to a boot entry in nvram uefi defaults to the fallback path.

```
\EFI
  \<OS Vendor 1 Directory>
    <OS Loader Image>
  \<OS Vendor 2 Directory>
    <OS Loader Image>
  ...
  \<OS Vendor N Directory>
    <OS Loader Image>
  \<OEM Directory>
    <OEM Application Image>
  \<BIOS Vendor Directory>
    <BIOS Vendor Application Image>
  \<Third Party Tool Vendor Directory>
    <Third Party Tool Vendor Application Image>

\BOOT
  BOOT{machine type short name}.EFI          <--- This is the fallback path
```

Use efibootmgr to Modify Boot entries in NVRAM

to list disk information:

```
$ sudo parted -l /dev/nvme0n1
$ sudo fdisk -l /dev/nvme0n1
$ blkid /dev/<partition or device>
```

We use **efibootmgr** command to edit/add boot entries in nvram:

To **add** a boot entry:

```
$ efibootmgr -c -d <Root Device> -p <Partition #> -L <label-for-nvram-entry> -l </EFI/vendor/osloader.efi>
$ efibootmgr -c -d <Root Device> -p <Partition #> -L <label-for-nvram-entry> -l </EFI/BOOT/BOOTX64.EFI>
$ efibootmgr -c -d /dev/nvme0n1 -p 4 -l "\\EFI\\name\\name.EFI" -L "label-nvram"
```

To **delete** a boot entry:

```
$ efibootmgr -b <boot-entry-#-000X> -B
$ efibootmgr -b 0005 -B
```

To change **boot order**:

```
$ efibootmgr -o <boot-entry#,boot-entry#,boot-entry#, ..... boot-entry#>
$ efibootmgr -o 0003
$ efibootmgr -o 0004,0002,0006,0002 ....etc
```

*See `man efibootmgr` for more information on using efibootmgr

Installing rEFInd Boot Manager

It is necessary to install rEFInd to the EFI System Partition (ESP) for it to control the boot process. commands to installing rEFInd include:

- * run:

#installs refind to fallback path of target device esp [for dedicated esp partition since it overwrites default fallback path]... /EFI/BOOT/bootx64.efi

\$ refind-install --usedefault /dev/target-esp-partition

- * for a shared esp partition:

#installs refind to vendor path of target esp [for shared esp partitions].... /EFI/vendor/bootx64.efi

\$ refind-install [with shared target esp mounted to /boot/efi]

or **\$ refind-install --root <mount-point>**

This option is intended to help install rEFInd from a "live CD" or other emergency system. To use it, you should mount your regular installation at /mount-point, including your /boot directory (if it's separate) at /mount-point/boot and (on Linux) your ESP at that location or at /mount-point/boot/efi

other methods (below) may be:

- copying the rEFInd binaries/contents manually from subdirectories of /usr/share/refind to <target esp /EFI/BOOT/>
- run the ./refind-install --usedefault /dev/target from within /usr/share/refind directory
- go to rEFInd's website and download the binary zip file, extract & run ./refind-install --usedefault /dev/target
- go to rEFInd's website and download the iso file. extract it to target esp or copy to ventoy.

For Much more Detailed Documentation on UEFI Booting Keep Reading below:

[UEFI - Booting - Documentation - Extra - Detailed- Following](#)

ArchWiki boot process

Categories: [Boot process](#) | [About Arch](#)

In order to boot Arch Linux, a Linux-capable **boot loader** must be set up. The boot loader is responsible for loading the kernel and **initial ramdisk** before initiating the boot process. The procedure is quite different for **BIOS** and **UEFI** systems. A detailed description is given on this or linked pages.

1 Firmware types

The firmware is the very first program that is executed once the system is switched on.

Tip: The words BIOS and (U)EFI are often used instead of firmware

1.1 BIOS

A **BIOS** or Basic Input-Output System is in most cases stored in a flash memory in the motherboard itself and independent of the system storage. Originally created for the **IBM PC** to handle hardware initialization and the boot process, it has been replaced progressively since 2010 by the **UEFI** which does not suffer from the same technical limitations.

1.2 UEFI

The **Unified Extensible Firmware Interface** has support for reading both the partition table as well as file systems. UEFI does not launch any **boot code from the Master Boot Record (MBR)** whether it exists or not, instead booting relies on boot entries in the **NVRAM**.

The UEFI specification mandates support for the **FAT12, FAT16, and FAT32** file systems (see **UEFI specification version 2.10, section 13.3.1.1**), but any conformant vendor can optionally add support for additional file systems; for example, **HFS+** or **APFS** in some Apple's firmwares. UEFI implementations also support **ISO 9660** for optical discs.

UEFI launches EFI applications, e.g. **boot loaders**, boot managers, **UEFI shell**, etc. These applications are usually stored as files in the **EFI system partition**. Each vendor can store its files in the EFI system partition under the `/EFI/vendor_name` directory. The applications can be launched by adding a boot entry to the NVRAM or from the UEFI shell.

The UEFI specification has support for legacy **BIOS** booting with its **Compatibility Support Module (CSM)**. If CSM is enabled in the UEFI, the UEFI will generate CSM boot entries for all drives. If a CSM boot entry is chosen to be booted from, the UEFI's CSM will attempt to boot from the drive's MBR bootstrap code.

Note: Intel is phasing out support for CSM, relying on the feature may not be feasible in the future. [\[1\]](#)

2 System initialization

2.3 Under BIOS

- System switched on, the **power-on self-test (POST)** is executed.
- After POST, BIOS initializes the hardware required for booting (disk, keyboard controllers etc.).
- BIOS launches the first 440 bytes (**the Master Boot Record bootstrap code area**) of the first disk in the BIOS disk order.
- The boot loader's first stage in the MBR boot code then launches its second stage code (if any) from either:
 - next disk sectors after the MBR, i.e. the so called post-MBR gap (only on a MBR partition table),
 - a partition's or a partitionless disk's **volume boot record (VBR)**,
 - for **GRUB** on a GPT partitioned disk—a GRUB-specific **BIOS boot partition** (it is used in place of the post-MBR gap that does not exist in GPT).
- The actual **boot loader** is launched.
- The boot loader then loads an operating system by either chain-loading or directly loading the operating system kernel.

2.4 Under UEFI

- System switched on, the **power-on self-test (POST)** is executed.
- After POST, UEFI initializes the hardware required for booting (disk, keyboard controllers etc.).
- Firmware reads the boot entries in the NVRAM to determine which EFI application to launch and from where (e.g. from which disk and partition).
 - A boot entry could simply be a disk. In this case the firmware looks for an **EFI system partition** on that disk and tries to find an EFI application in the fallback boot path `\EFI\BOOT\BOOTx64.EFI` (`BOOTIA32.EFI` on **systems with a IA32 (32-bit) UEFI**). This is how UEFI bootable removable media work.
- Firmware launches the EFI application.
 - This could be a **boot loader** or the Arch **kernel** itself using **EFISTUB**.
 - It could be some other EFI application such as the **UEFI shell** or a **boot manager** like **systemd-boot** or **rEFInd**.

If **Secure Boot** is enabled, the boot process will verify authenticity of the EFI binary by signature.

Note: Some UEFI systems can only boot from the fallback boot path.

2.4.1 Multibooting in UEFI

Since each OS or vendor can maintain its own files within the **EFI system partition** without affecting the other, multi-booting using UEFI is just a matter of launching a different EFI application corresponding to the particular operating system's boot loader. This removes the need for relying on the **chain loading** mechanisms of one **boot loader** to load another OS.

See also **Dual boot with Windows** .

3 Boot loader

A boot loader is a piece of software started by the firmware (**BIOS** or **UEFI**). It is responsible for loading the kernel with the wanted **kernel parameters** and any external iniramfs images. In the case of UEFI, the kernel itself can be directly launched by the UEFI using the **EFI boot stub**. A separate boot loader or boot manager can still be used for the purpose of editing kernel parameters before booting.

Warning: A boot loader must be able to access the kernel and iniramfs image(s), otherwise the system will not boot. Thus, in a typical setup, it must support accessing `/boot` . That means it must have support for everything starting from the block devices, stacked block devices (LVM, RAID, dm-crypt, LUKS, etc.) and ending with the file system on which the kernel(s) and iniramfs image(s) reside.

Note: Loading **Microcode** updates requires adjustments in boot loader configuration. [\[2\]](#)

3.5 Feature comparison

Note:

- As GPT is part of the UEFI specification, all UEFI boot loaders support GPT disks. GPT on BIOS systems is possible, using either "hybrid booting" with **Hybrid MBR**, or the new **GPT-only** protocol. This protocol may however cause issues with certain BIOS implementations; see **rodsbooks** for details.
- Encryption mentioned in file system support is **filesystem-level encryption**, it has no bearing on **block-level encryption** .

Name	Firmware		Partition table		Multi-boot	File systems					Notes
	BIOS	UEFI	MBR	GPT		Btrfs	ext4	ReiserFS	VFAT	XFS	
EFISTUB	–	Yes	Yes	Yes	–	–	–	–	Inherited from firmware ¹	–	The kernel is a valid EFI executable which can be loaded directly from the UEFI firmware with efibootmgr , or another boot loader.
Unified kernel image	–	Yes ²	Yes	Yes	–	–	–	–	Inherited from firmware ¹	–	systemd-stub (?), a kernel, iniramfs and kernel command line packed into EFI executable to be loaded directly from UEFI firmware or another boot loader.
											On BIOS/GPT configuration requires a

Name	Firmware		Partition table		Multi-boot	File systems					Notes
	BIOS	UEFI	MBR	GPT		Btrfs	ext4	ReiserFS	VFAT	XFS	
GRUB	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	BIOS boot partition. Supports RAID, LUKS1 and LVM (but not thin provisioned volumes).
Limine	Yes	Yes	Yes	Yes	Yes	No	Without encryption	No	Yes	No	
rEFInd	No	Yes	Yes	Yes	Yes ³	Without encryption	Without encryption	Without tail-packing feature	Inherited from firmware ¹	No	Supports auto-detecting kernels and parameters without explicit configuration, and supports fastboot ^[3] .
Syslinux	Yes	Limited	Yes	Yes	Partial	Without: multi-device volumes, compression, encryption	Without encryption	No	Yes	MBR only; without sparse inodes	No support for certain file system features. ^[4] Does not have file system drivers ^[5] , can only access the file system it was installed to.
systemd-boot	No	Yes ²	Manual install only	Yes	Yes ³	Can be side-loaded ⁴	Can be side-loaded ⁴	Can be side-loaded ⁴	Inherited from firmware ¹	Can be side-loaded ⁴	Cannot launch binaries from partitions other than the ESP or the Extended Boot Loader Partition (XBOOTLDR partition). Supports automatically detecting unified kernel images when they are placed into esp/EFI/Linux.
GRUB Legacy	Yes	No	Yes	No	Yes	No	No	Yes	Yes	XFS v4 only	Discontinued in favor of GRUB.
LILO	Yes	No	Yes	No	Yes	No	Without encryption	Yes	Yes	Yes	Discontinued due to limitations (e.g. with Btrfs, GPT, RAID).

- File system support is inherited from the firmware. The UEFI specification mandates support for the FAT12, FAT16 and FAT32 file systems^[6], but vendors can optionally add support for additional file systems; for example, the firmware in Apple **Macs** supports the HFS+ file system. If the firmware provides an interface for loading **UEFI drivers** on startup, then support for additional file systems can be added by loading (independently acquired) file system drivers.
- Supports mixed mode booting on IA32 UEFI. I.e. It can boot a 64-bit x86_64 Linux kernel on **32-bit IA32 UEFI**.
- A **boot manager**. It can only launch other EFI applications, for example, Linux kernel images built with `CONFIG_EFI_STUB=y` and Windows `bootmgfw.efi`.
- systemd-boot supports loading **UEFI file system drivers**. These are provided by **efifs** and need to be placed into `esp/EFI/systemd/drivers/`.

See also [Wikipedia:Comparison of boot loaders](#).

4 Kernel

The **boot loader** boots the **vmlinux image** containing the **kernel**.

The kernel functions on a low level (*kernel space*) interacting between the hardware of the machine and the programs. The kernel initially performs hardware enumeration and initialization before continuing to userspace. See [Wikipedia:Kernel \(operating system\)](#) and [Wikipedia:Linux kernel](#) for a detailed explanation.

5 initramfs

The root file system at `/` starts out as an empty **rootfs**, which is a special instance of ramfs or tmpfs. This is the temporary root file system where the initramfs (*initial RAM file system*) images will be unpacked to.

The purpose of the initramfs is to bootstrap the system to the point where it can access the root file system (see [FHS](#) for details). It does not need to contain every module one would ever want to use; it should only have modules required for the root device like IDE, SCSI, SATA or USB/FW (if booting from an external drive) and encryption. The majority of modules will be loaded later on by **udev**, during the init process.

First, the kernel unpacks its builtin initramfs into the temporary root. Arch Linux's **official kernels** use an empty archive for the builtin initramfs, which is the default when building Linux. Then, the kernel unpacks external initramfs files specified by the command line passed by the **boot loader**, overwriting any files from the embedded initramfs. These external initramfs images can be generated with **mkinitcpio**, **dracut** or **booste**r, and are Arch's preferred method for setting up for *early userspace*.

6 Early userspace

The *early userspace* stage takes place while the temporary rootfs is mounted, operating on the files provided by the **#initramfs**.

The function of early userspace is **configurable**, but generally does the following:

- systemd-modules-load(8)** loads kernel modules, such as any block device modules needed to mount the real root file system.
- Handle decryption of the real root file system, if applicable.
- Load the DRM module, as **early KMS** is enabled by default for in-tree modules.

At the final stage of early userspace, the real root is mounted at `/sysroot`, and then switched to. The late userspace starts by executing the **init** program from the real root file system.

7 Late userspace

The startup of late userspace is executed by the **init** process. Arch officially uses **systemd** which is built on the concept of units and services, but the functionality described here largely overlaps with other init systems.

7.6 getty

The init process calls **getty** once for each **virtual terminal** (typically six of them). **getty** initializes each terminal and protects it from unauthorized access. When the username and password are provided, **getty** checks them against `/etc/passwd` and `/etc/shadow`, then calls `login(1)`.

7.6.2 Login

The **login** program begins a session for the user by setting environment variables and starting the user's shell, based on `/etc/passwd`. The **login** program displays the contents of `/etc/motd` (*message of the day*) after a successful login, just before it executes the login shell. It is a good place to display your Terms of Service to remind users of your local policies or anything you wish to tell them.

7.6.3 Shell

Once the user's **shell** is started, it will typically run a runtime configuration file, such as **bashrc**, before presenting a prompt to the user. If the account is configured to **start X at login**, the runtime configuration file will call **startx** or **xinit**. Jump to [#Graphical session](#) for the end.

7.7 Display manager

This article or section needs expansion.

Reason: This section only describes the process with **Xorg** but does not explain what happens with **Wayland**. (Discuss in [Talk:Arch boot proces s](#))

Additionally, **init** can be configured to start a **display manager** instead of **getty** on a specific virtual terminal. This requires manually **enabling** its **systemd service file**. The display manager then starts a graphical session.

7.7.4 Graphical session

xinit runs the user's **xinitrc** runtime configuration file, which normally starts a **window manager** or a **desktop environment**. When the user is finished and exits, **xinit**, **startx**, the shell, and login will terminate in that order, returning to **getty** or the display manager.

8 See also

- [Early Userspace in Arch Linux](#)
- [Inside the Linux boot process](#)
- [Wikipedia:Linux startup process](#)
- [Wikipedia:inltrd](#)
- [Boot Linux Grub Into Single User Mode](#)
- [NeoSmart: The BIOS/MBR Boot Process](#)
- [Kernel Newbie Corner: initrd and initramfs](#)
- [Rod Smith - Managing EFI Boot Loaders for Linux](#)

This page was last edited on 14 October 2023, at 10:48.

Content is available under [GNU Free Documentation](#)

[License 1.3](#) or later unless otherwise noted. [Privacy policy](#)

[About ArchWiki](#)

13.3. File System Format

The file system supported by the Extensible Firmware Interface is based on the FAT file system. EFI defines a specific version of FAT that is explicitly documented and testable. Conformance to the EFI specification and its associate reference documents is the only definition of FAT that needs to be implemented to support EFI. To differentiate the EFI file system from pure FAT, a new partition file system type has been defined.

EFI encompasses the use of FAT32 for a system partition, and FAT12 or FAT16 for removable media. The FAT32 system partition is identified by an OStype value other than

that used to identify previous versions of FAT. This unique partition type distinguishes an EFI defined file system from a normal FAT file system. The file system supported by EFI includes support for long file names.

The definition of the EFI file system will be maintained by specification and will not evolve over time to deal with errata or variant interpretations in OS file system drivers or file system utilities. Future enhancements and compatibility enhancements to FAT will not be automatically included in EFI file systems. The EFI file system is a target that is fixed by the EFI specification, and other specifications explicitly referenced by the EFI specification.

For more information about the EFI file system and file image format, visit the web site from which this document was obtained.

13.3.1. System Partition

A System Partition is a partition in the conventional sense of a partition on a legacy system. For a hard disk, a partition is a contiguous grouping of sectors on the disk where the starting sector and size are defined by the Master Boot Record (MBR), which resides on LBA 0 (i.e., the first sector of the hard disk) ([LBA 0 Format](#)), or the GUID Partition Table (GPT), which resides on logical block 1 (the second sector of the hard disk) ([GPT overview](#)). For a diskette (floppy) drive, a partition is defined to be the entire media. A System Partition can reside on any media that is supported by EFI Boot Services.

A System Partition supports backward compatibility with legacy systems by reserving the first block (sector) of the partition for compatibility code. On legacy systems, the first block (sector) of a partition is loaded into memory and execution is transferred to this code. EFI firmware does not execute the code in the MBR. The EFI firmware contains knowledge about the partition structure of various devices, and can understand legacy MBR, GPT, and “El Torito.”

The System Partition contains directories, data files, and UEFI Images. UEFI Images can contain a OS Loader, an driver to extend platform firmware capability, or an application that provides a transient service to the system. Applications written to this specification could include things such as a utility to create partitions or extended diagnostics. A System Partition can also support data files, such as error logs, that can be defined and used by various OS or system firmware software components.

13.3.1.1. File System Format

The first block (sector) of a partition contains a data structure called the BIOS Parameter Block (BPB) that defines the type and location of FAT file system on the drive. The BPB

contains a data structure that defines the size of the media, the size of reserved space, the number of FAT tables, and the location and size of the root directory (not used in FAT32). The first block (sector) also contains code that will be executed as part of the boot process on a legacy system. This code in the first block (sector) usually contains code that can read a file from the root directory into memory and transfer control to it. Since EFI firmware contains a file system driver, EFI firmware can load any file from the file system without the FAT32, FAT16, and FAT12 variants of the EFI file system. What variant of EFI FAT to use is defined by the size of the media. The rules defining the relationship between media size and FAT variants is defined in the specification for the EFI file system.

The EFI firmware must support the FAT32, FAT16, and FAT12 variants of the EFI file system. What variant of EFI FAT to use is defined by the size of the media. The rules defining the relationship between media size and FAT variants is defined in the specification for the EFI file system.

The UEFI system partition FAT32 Data region should be aligned to the physical block boundary and optimal transfer length granularity of the device ([GPT overview](#)). This is controlled by the BPB_RsvdSecCnt field and the applicable BPB_FATSz field (e.g., formatting software may set the BPB_RsvdSecCnt field to a value that results in alignment and/or may set the BPB_FATSz field to a value that ensures alignment).

13.3.1.2. File Names

FAT stores file names in two formats. The original FAT format limited file names to eight characters with three extension characters. This type of file name is called an 8.3, pronounced eight dot three, file name. FAT was extended to include support for long file names (LFN).

FAT 8.3 file names are always stored as uppercase ASCII characters. LFN can either be stored as ASCII or UCS-2 characters and are stored case sensitive. The string that was used to open or create the file is stored directly into LFN. FAT defines that all files in a directory must have a unique name, and unique is defined as a case insensitive match. The following are examples of names that are considered to be the same and cannot exist in a single directory:

- "ThisIsAnExampleDirectory.Dir"
- "thisisanexamppldirectory.dir"
- THISISANEXAMPLEDIRECTORY.DIR
- ThisIsAnExampleDirectory.DIR

Note: Although the FAT32 specification allows file names to be encoded using UTF-16, this specification only recognizes the UCS-2 subset for the purposes of sorting or collation.

13.3.1.3. Directory Structure

An EFI system partition that is present on a hard disk must contain an EFI defined directory in the root directory. This directory is named EFI. All OS loaders and applications will be stored in subdirectories below EFI. Applications that are loaded by other applications or drivers are not required to be stored in any specific location in the EFI system partition. The choice of the subdirectory name is up to the vendor, but all vendors must pick names that do not collide with any other vendor's subdirectory name. This applies to system manufacturers, operating system vendors, BIOS vendors, and third party tool vendors, or any other vendor that wishes to install files on an EFI system partition. There must also only be one executable EFI image for each supported processor architecture in each vendor subdirectory. This guarantees that there is only one image that can be loaded from a vendor subdirectory by the EFI Boot Manager. If more than one executable EFI image is present, then the boot behavior for the system will not be deterministic. There may also be an optional vendor subdirectory called BOOT.

This directory contains EFI images that aid in recovery if the boot selections for the software installed on the EFI system partition are ever lost. Any additional UEFI-compliant executables must be in subdirectories below the vendor subdirectory. The following is a sample directory structure for an EFI system partition present on a hard disk.

```
\EFI
  \<OS Vendor 1 Directory>
    <OS Loader Image>
  \<OS Vendor 2 Directory>
    <OS Loader Image>
  ...
  \<OS Vendor N Directory>
    <OS Loader Image>
  \<OEM Directory>
    <OEM Application Image>
  \<BIOS Vendor Directory>
    <BIOS Vendor Application Image>
  \<Third Party Tool Vendor Directory>
    <Third Party Tool Vendor Application Image>
  \BOOT
    BOOT{machine type short name}.EFI
```

For removable media devices there must be only one UEFI-compliant system partition, and that partition must contain an UEFI-defined directory in the root directory. The directory will be named EFI. All OS loaders and applications will be stored in a subdirectory below EFI called BOOT. There must only be one executable EFI image for each supported processor architecture in the BOOT directory. For removable media to be bootable under EFI, it must be built in accordance with the rules laid out in [Removable Media Boot Behavior](#). This guarantees that there is only one image that can be

automatically loaded from a removable media device by the EFI Boot Manager. Any additional EFI executables must be in directories other than BOOT. The following is a sample directory structure for an EFI system partition present on a removable media device.

```
\EFI
  \BOOT
    BOOT{machine type short name}.EFI
```

13.3.2. Partition Discovery

This specification requires the firmware to be able to parse the [Legacy MBR](#) , GUID Partition Table (GPT)([GPT overview](#)), and El Torito ([ISO-9660](#) and [El Torito](#)) logical device volumes. The EFI firmware produces a logical [EFI_BLOCK_IO_PROTOCOL](#) device for:

- each GUID Partition Entry (see table 16 in 5.3.3) with bit 1 set to zero;
- each El Torito logical device volume; and
- if no GPT is present, each partition found in the legacy MBR partition tables.

LBA zero of the *EFI_BLOCK_IO_PROTOCOL* device will correspond to the first logical block of the partition. See [Nesting of Legacy MBR Partition Records](#). If a GPT Partition Entry has Attribute bit 1 set then a logical *EFI_BLOCK_IO_PROTOCOL* device must not be created.

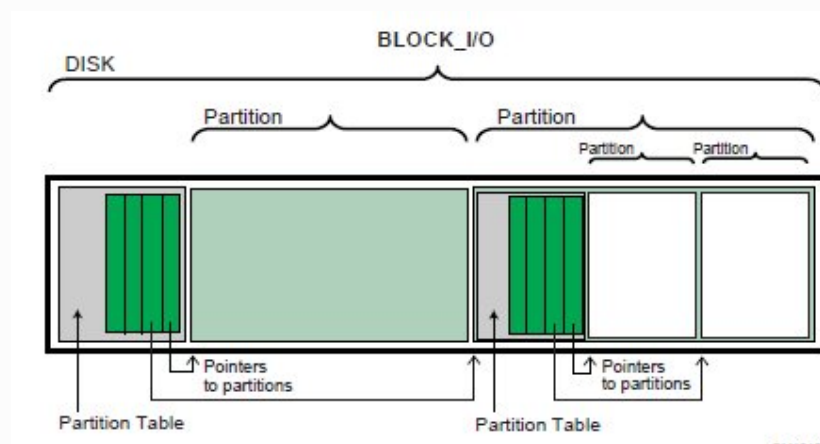


Fig. 13.1 Nesting of Legacy MBR Partition Records

The following is the order in which a block device must be scanned to determine if it contains partitions. When a check for a valid partitioning scheme succeeds, the search terminates.

1. Check for GUID Partition Table Headers.
2. Follow ISO-9660 specification to search for ISO-9660 volume structures on the magic

LBA.

3. Check for an “El Torito” volume extension and follow the “El Torito” CD-ROM specification.
4. If none of the above, check LBA 0 for a legacy MBR partition table.
5. No partition found on device.

If a disk contains a recognized RAID structure (e.g. DDF structure as defined in *The Storage Networking Industry Association Common RAID Disk Data Format Specification* – see Glossary), the data on the disk must be ignored, unless the driver is using the RAID structure to produce a logical RAID volume.

EFI supports the nesting of legacy MBR partitions, by allowing any legacy MBR partition to contain more legacy MBR partitions. This is accomplished by supporting the same partition discovery algorithm on every logical block device. It should be noted that the GUID Partition Table does not allow nesting of GUID Partition Table Headers. Nesting is not needed since a GUID Partition Table Header can support an arbitrary number of partitions (the addressability limits of a 64-bit LBA are the limiting factor).

13.3.2.1. ISO-9660 and El Torito

ISO-9660 is the industry standard low level format used on CD-ROM and DVD-ROM. The CD-ROM format is completely described by the “El Torito” Bootable CD-ROM Format Specification Version 1.0. To boot from a CD-ROM or DVD-ROM in the boot services environment, an EFI System partition is stored in a “no emulation” mode as defined by the “El Torito” specification. A Platform ID of 0xEF indicates an EFI System Partition. The Platform ID is in either the Section Header Entry or the Validation Entry of the Booting Catalog as defined by the “El Torito” specification. EFI differs from “El Torito” “no emulation” mode in that it does not load the “no emulation” image into memory and jump to it. EFI interprets the “no emulation” image as an EFI system partition. EFI interprets the Sector Count in the Initial/Default Entry or the Section Header Entry to be the size of the EFI system partition. If the value of Sector Count is set to 0 or 1, EFI will assume the system partition consumes the space from the beginning of the “no emulation” image to the end of the CD-ROM.

A DVD-ROM image formatted as required by the UDF 2.0 specification (OSTA Universal Disk Format Specification, Revision 2.0) shall be booted by UEFI if:

- the DVD-ROM image conforms to the “UDF Bridge” format defined in the UDF 2.0 specification, and
- the DVD-ROM image contains exactly one ISO-9660 file system, and
- the ISO-9660 file system conforms to the “El Torito” Bootable CD-ROM Format Specification.

Booting from a DVD-ROM that satisfies the above requirements is accomplished using the same methods as booting from a CD-ROM: the ISO-9660 file system shall be booted.

Since the EFI file system definition does not use the same Initial/Default entry as a legacy CD-ROM it is possible to boot personal computers using an EFI CD-ROM or DVD-ROM. The inclusion of boot code for personal computers is optional and not required by EFI.

13.3.3. Number and Location of System Partitions

UEFI does not impose a restriction on the number or location of System Partitions that can exist on a system. System Partitions are discovered when required by UEFI firmware by examining the partition GUID and verifying that the contents of the partition conform to the FAT file system as defined in [File System Format](#). Further, UEFI implementations may allow the use of conforming FAT partitions which do not use the ESP GUID. Partition creators may prevent UEFI firmware from examining and using a specific partition by setting bit 1 of the Partition Attributes (see 5.3.3) which will exclude the partition as a potential ESP.

Software installation may choose to create and locate an ESP on each target OS boot disk, or may choose to create a single ESP independent of the location of OS boot disks and OS partitions. It is outside of the scope of this specification to attempt to coordinate the specification of size and location of an ESP that can be shared by multiple OS or Diagnostics installations, or to manage potential namespace collisions in directory naming in a single (central) ESP.

13.3.4. Media Formats

This section describes how booting from different types of removable media is handled. In general the rules are consistent regardless of a media's physical type and whether it is removable or not.

13.3.4.1. Removable Media

Removable media may contain a standard FAT12, FAT16, or FAT32 file system.

Booting from a removable media device can be accomplished the same way as any other boot. The boot file path provided to the boot manager can consist of a UEFI application image to load, or can merely be the path to a removable media device. In the first case, the path clearly indicates the image that is to be loaded. In the later case, the boot manager implements the policy to load the default application image from the device.

For removable media to be bootable under EFI, it must be built in accordance with the rules laid out in [Removable Media Boot Behavior](#)

13.3.4.2. Diskette

EFI bootable diskettes follow the standard formatting conventions used on personal computers. The diskette contains only a single partition that complies to the EFI file system type. For diskettes to be bootable under EFI, it must be built in accordance with the rules laid out in [Removable Media Boot Behavior](#).

Since the EFI file system definition does not use the code in the first block of the diskette, it is possible to boot personal computers using a diskette that is also formatted as an EFI bootable removable media device. The inclusion of boot code for personal computers is optional and not required by EFI.

Diskettes include the legacy 3.5-inch diskette drives as well as the newer larger capacity removable media drives such as an Iomega* Zip,* Fujitsu MO, or MKE LS-120/SuperDisk*.

13.3.4.3. Hard Drive

Hard drives may contain multiple partitions as defined in See [Partition Discovery](#) on partition discovery. Any partition on the hard drive may contain a file system that the EFI firmware recognizes. Images that are to be booted must be stored under the EFI subdirectory as defined in [System Partition](#) and [Partition Discovery](#).

EFI code does not assume a fixed block size.

Since EFI firmware does not execute the MBR code and does not depend on the BootIndicator field in the legacy MBR partition records, the hard disk can still boot and function normally.

13.3.4.4. CD-ROM and DVD-ROM

A CD-ROM or DVD-ROM may contain multiple partitions as defined in [System Partition](#) and [Partition Discovery](#) and in the “El Torito” specification.

EFI code does not assume a fixed block size.

Since the EFI file system definition does not use the same Initial/Default entry as a legacy CD-ROM, it is possible to boot personal computers using an EFI CD-ROM or DVD-ROM. The inclusion of boot code for personal computers is optional and not required by EFI.

UEFI boot: how does that actually work, then?

Adam Williamson — 2014-01-25 21:07 — [Source](#)

It's AdamW Essay Time again! If you're looking for something short and snappy, look elsewhere.

[Kamil Paral](#) kindly informs me I'm a chronic sufferer of [Graphomania](#). Always nice to know what's wrong with you.

IMPORTANT NOTE TO INDUSTRY FOLKS: This blog post is aimed at regular everyday folks; it's intended to dispel a few common myths and help regular people understand UEFI a bit better. It is not a low-level fully detailed and 100% technically accurate explanation, and I'm not a professional firmware engineer or anything like that. If you're actually building an operating system or hardware or something, please don't rely on my simplified explanations or ask me for help; I'm just an idiot on the internet. If you're doing that kind of thing and you have money, join the UEFI Forum or ask your suppliers or check your reference implementation or whatever. If you don't have money, try asking your peers with more experience, nicely. **END IMPORTANT NOTE**

You've probably read a lot of stuff on the internet about UEFI. Here is something important you should understand: 95% of it was probably garbage. If you think you know about UEFI, and you derived your knowledge anywhere other than the UEFI specifications, [mjpg59's blog](#) or one of a few other vaguely reliable locations/people - [Rod Smith](#), or [Peter Jones](#), or Chris Murphy, or the documentation of the relatively few OSes whose developers actually know what the hell they're doing with UEFI - what you think you know is likely a toxic mix of misunderstandings, misconceptions, half-truths, propaganda and downright lies. So you should probably forget it all.

Good, now we've got that out of the way. What I mostly want to talk about is bootloading, because that's the bit of firmware that matters most to most people, and the bit news sites are always banging on about and wildly misunderstanding.

Terminology

First, let's get some terminology out of the way. Both [BIOS](#) and [UEFI](#) are types of [firmware](#) for computers. BIOS-style firmware is (mostly) only ever found on [IBM PC compatible computers](#). UEFI is meant to be more generic, and can be found on systems which are not in the 'IBM PC compatible' class.

You do not have a 'UEFI BIOS'. No-one has a 'UEFI BIOS'. Please don't ever say 'UEFI BIOS'. BIOS is not a generic term for all PC firmware, it is a particular type of PC firmware. Your computer has a firmware. If it's an IBM PC compatible computer, it's almost certainly either a BIOS or a UEFI firmware. If you're running [Coreboot](#), congratulations, Mr./Ms. Exception. You may be proud of yourself.

[Secure Boot](#) is not the same thing as UEFI. Do not ever use those terms interchangeably. Secure Boot is a single effectively optional element of the UEFI specification, which was added in version 2.2 of the UEFI specification. We will talk about precisely what it is later, but for now, just remember it is not the same thing about UEFI. You need to understand what Secure Boot is, and what UEFI is, and which of the two you are actually talking about at any given time. We'll talk about UEFI first, and then we'll talk about Secure Boot as an 'extension' to UEFI, because that's basically what it is.

Bonus Historical Note: UEFI was not invented by, is not controlled by, and has never been controlled by Microsoft. Its predecessor and basis, EFI, was developed and published by Intel. UEFI is managed by the [UEFI Forum](#). Microsoft is a member of the UEFI forum. So is Red Hat, and so is Apple, and so is just about every major PC manufacturer, Intel (obviously), AMD, and a [laundry list of other major and minor hardware, software and firmware companies and organizations](#). It is a broad consensus specification, with all the messiness that entails, some of which we'll talk about specifically later. It is no one company's Evil Vehicle Of Evilness.

References

If you really want to understand UEFI, it's a really good idea to go and read the UEFI specification. You can do this. It's very easy. You don't have to pay anyone any money. I am not going to tell you that reading it will be the most fun you've ever had, because it won't. But it won't be a waste of your time. You can find it [right here on the official UEFI site](#). You have to check a couple of boxes, but you are not signing your soul away to Satan, or anything. It's fine. As I write this, the current version of the spec is [2.4 Errata A](#), and that's the version this post is written with regard to.

There is no BIOS specification. BIOS is a *de facto* standard - it works the way it worked on actual IBM PCs, in the 1980s. That's kind of one of the reasons UEFI exists.

Now, to keep things simple, let's consider two worlds. One is the world of IBM PC compatible computers - hereafter referred to just as PCs - before UEFI and GPT (we'll come to GPT) existed. This is the world a lot of you are probably familiar with and may understand quite well. Let's talk about how booting works on PCs with BIOS firmware.

BIOS booting

It works, in fact, in a very, very simple way. On your bog-standard old-skool BIOS PC, you have one or more disks which have an [MBR](#). The MBR is another *de facto* standard; basically, the very start of the disk describes the partitions on the disk in a particular format, and contains a 'boot loader', a very small piece of code that a BIOS firmware knows how to execute, whose job it is to boot the operating system(s). (Modern bootloaders frequently are much bigger than can be contained in the MBR space and have to use a multi-stage design where the bit in the MBR just knows how to load the next stage from somewhere else, but that's not important to us right now).

All a BIOS firmware knows, in the context of booting the system, is what disks the system contains. You, the owner of this BIOS-based computer, can tell the BIOS firmware which disk you want it to boot the system from. The firmware has no knowledge of anything beyond that. It executes the bootloader it finds in the MBR of the specified disk, and that's it. The firmware is no longer involved in booting.

In the BIOS world, absolutely all forms of multi-booting are handled above the firmware layer. The firmware layer doesn't really know what a bootloader is, or what an operating system is. Hell, it doesn't know what a partition is. All it can do is run the boot loader from a disk's MBR. You also cannot configure the boot process from outside of the firmware.

UEFI booting: background

OK, so we have our background, the BIOS world. Now let's look at how booting works on a UEFI system. Even if you don't grasp the details of this post, grasp this: *it is completely different*. Completely and utterly different from how BIOS booting works. You cannot apply any of your understanding of BIOS booting to native UEFI booting. You cannot make a little tweak to a system designed for the world of BIOS booting and apply it to *native* UEFI booting. You need to understand that it is a completely different world.

Here's another important thing to understand: many UEFI firmwares implement some kind of *BIOS compatibility mode*, sometimes referred to as a *CSM*. Many UEFI firmwares can boot a system just like a BIOS firmware would - they can look for an MBR on a disk, and execute the boot loader from that MBR, and leave everything subsequently up to that bootloader. People sometimes *incorrectly* refer to using this feature as 'disabling UEFI', which is *linguistically* nonsensical. You cannot 'disable' your system's firmware. It's just a stupid term. Don't use it, but understand what people really mean when they say it. They are talking about using a UEFI firmware's ability to boot the system 'BIOS-style' rather than native UEFI style.

What I'm going to describe is *native* UEFI booting. If you have a UEFI-based system whose firmware has the BIOS compatibility feature, and you decide to use it, and you apply this decision consistently, then as far as booting is concerned, you can pretend your system is BIOS-based, and just do everything the way you did with BIOS-style booting. If you're going to do this, though, just make sure you *do* apply it consistently. I really can't recommend strongly enough that you do *not* attempt to mix UEFI-native and BIOS-compatible booting of permanently-installed operating systems on the same computer, and *especially not* on the same disk. It is a terrible terrible idea and will cause you heartache and pain. If you decide to do it, don't come crying to me.

For the sake of sanity, I am going to assume the use of disks with a [GPT](#) partition table, and EFI FAT32 [EFI system partitions](#). Depending on how deep you're going to dive into this stuff you *may* find out that it's not strictly speaking the case that you can *always* assume you'll be dealing with GPT disks and EFI FAT32 ESPs when dealing with UEFI native boot, but the UEFI specification is quite strongly tied to GPT disks and EFI FAT32 ESPs, and this is what you'll be dealing with in 99% of cases. Unless you're dealing with Macs, and quite frankly, *screw* Macs.

Edit note: the following sections (up to *Implications and Complications*) were heavily revised on 2014-01-26, a few hours after the initial version of this post went up, based on feedback from Peter Jones. Consider this to be v2.0 of the post. An earlier version was written in a somewhat less accurate and more confusing way.

UEFI native booting: how it actually works - background

OK, with that out of the way, let's get to the meat. This is how native UEFI booting actually works. It's probably helpful to go into this with a bit of high-level background.

UEFI *provides much more* infrastructure at the firmware level for handling system boot. It's nowhere near as simple as BIOS. Unlike BIOS, UEFI certainly does understand, to varying degrees, the concepts of 'disk partitions' and 'bootloaders' and 'operating systems'.

You can sort of look at the BIOS boot process, and look at the UEFI process, and see how the UEFI process extends various bits to address specific problems.

The BIOS/MBR approach to finding the bootloader is pretty janky, when you think about it. It's very 'special sauce': this particular tiny space at the front of the disk contains magic code that only really makes much sense to the system firmware and special utilities for writing it. There are several problems with this approach.

- It's inconvenient to deal with - you need special utilities to write the MBR, and just about the only way to find out what's in one is to dd the contents out and examine them.
- As noted above, the MBR itself is not big enough for many modern bootloaders. What they do is install a small part of themselves to the MBR proper, and

- the rest to the empty space on the disk between where the conventional MBR ends and the first partition begins. There's a rather big problem with this (well, the whole design is a big problem, but never mind), which is that there's no reliable convention for where the first partition should begin, so it's difficult to be sure there'll be enough space. One thing you usually can rely on is that there won't be *enough* space for some bootloader configurations.
- The design doesn't provide any standardized layer or mechanism for selecting boot targets other than disks...but people *want* to select boot targets other than disks. i.e. they want to have multiple bootable 'things' - usually operating systems - per disk. The only way to do this, in the BIOS/MBR world, is for the bootloaders to handle it; but there's no widely accepted convention for the right way to do this. There are many many different approaches, none of which is particularly interoperable with any of the others, none of which is a widely accepted standard or convention, and it's very difficult to write tooling at the OS / OS installation layer that handles multiboot cleanly. It's just a very messy design.
 - The design doesn't provide a standard way of booting from anything *except* disks. We're not going to really talk about that in this article, but just be aware it's another advantage of UEFI booting: it provides a standard way for booting from, for instance, a remote server.
 - There's no mechanism for levels above the firmware to configure the firmware's boot behaviour.

So you can imagine the UEFI Elves sitting around and considering this problem, and coming up with a solution. Instead of the firmware only knowing about disks and one 'magic' location per disk where bootloader code might reside, UEFI has much more infrastructure at the firmware layer for handling boot loading. Let's look at all the things it defines that are relevant here.

EFI executables

The EFI spec defines an executable format and requires all UEFI firmwares be capable of executing code in this format. When you write a bootloader for native UEFI, you write in this format. This is pretty simple and straightforward, and doesn't need any further explanation: it's just a Good Thing that we now have a firmware specification which actually defines a common format for code the firmware can execute.

The GPT (GUID partition table) format

The [GUID Partition Table](#) format is very much tied in with the UEFI specification, and again, this isn't something particularly complex or in need of much explanation, it's just a good bit of groundwork the spec provides. GPT is just a standard for doing partition tables - the information at the start of a disk that defines what partitions that disk contains. It's a better standard for doing this than MBR/MS-DOS' partition tables were in many ways, and the UEFI spec requires that UEFI-compliant firmwares be capable of interpreting GPT (it also requires them to be capable of interpreting MBR, for backwards compatibility). All of this is useful groundwork: what's going on here is the spec is establishing certain capabilities that everything above the firmware layer can rely on the firmware to have.

EFI system partitions

I actually really wrapped my head around the EFI system partition concept while revising this post, and it was a great 'ah!' moment. Really, the concept of 'EFI system partitions' is just an answer to the problem of the 'special sauce' MBR space. The concept of some undefined amount of empty space at the start of a disk being 'where bootloader code lives' is a pretty crappy design, as we saw above. EFI system partitions are just UEFI's solution to that. ¹

The solution is this: we require the firmware layer to be capable of reading some specific types of filesystem. The UEFI spec requires that compliant firmwares be capable of reading the FAT12, FAT16 and FAT32 variants of the [FAT](#) format, in essence. In fact what it does is codify a particular interpretation of those formats as they existed at the point UEFI was accepted, and say that UEFI compliant firmwares must be capable of reading those formats. As the spec puts it:

"The file system supported by the Extensible Firmware Interface is based on the FAT file system. EFI defines a specific version of FAT that is explicitly documented and testable. Conformance to the EFI specification and its associate reference documents is the only definition of FAT that needs to be implemented to support EFI. To differentiate the EFI file system from pure FAT, a new partition file system type has been defined."

An 'EFI system partition' is really just any partition formatted with one of the UEFI spec-defined variants of FAT and given a specific GPT partition type to help the firmware find it. And the purpose of this is just as described above: allow everyone to rely on the fact that the firmware layer will definitely be able to read data from a pretty 'normal' disk partition. Hopefully it's clear why this is a better design: instead of having to write bootloader code to the 'magic' space at the start of an MBR disk, operating systems and so on can just create, format and mount partitions in a widely understood format and put bootloader code and anything else that they might want the firmware to read there.

The whole ESP thing seemed a bit bizarre and confusing to me at first, so I hope this section explains why it's actually a very sensible idea and a good design - the bizarre and confusing thing is really the BIOS/MBR design, where the only way for you to write something from the OS layer that you knew the firmware layer could consume was to write it into some (but you didn't know how much) Magic Space at the start of a disk, a convention which isn't actually codified anywhere. That really *isn't* a very sensible or understandable design, if you step back and take a look at it.

As we'll note later, the UEFI spec tends to take a 'you must at least do these things' approach - it rarely prohibits firmwares from doing anything else. It's not against the spec to write a firmware that can execute code in other formats, read other types of partition table, and read partitions formatted with filesystems other than the UEFI variants of FAT. But a UEFI compliant firmware must *at least* do all these things, so if you are writing an OS or something else that you want to run on *any* UEFI compliant firmware, this is why the EFI system partition concept is so important: it gives you (at least in theory) 100% confidence that you can put an EFI executable on a partition formatted with the UEFI FAT implementation and the correct GPT partition type, and the system firmware will be able to read it. This is the thing you can take to the bank, like 'the firmware will be able to execute some bootloader code I put in the MBR space' was in the BIOS world.

So now we have three important bits of groundwork the UEFI spec provides: thanks to these requirements, any other layer can confidently rely on the fact that the firmware:

- Can read a partition table
- Can access files in some specific filesystems
- Can execute code in a particular format

This is much more than you can rely on a BIOS firmware being capable of. However, in order to complete the vision of a firmware layer that can handle booting multiple targets - not just disks - we need one more bit of groundwork: there needs to be a mechanism by which the firmware *finds* the various possible boot targets, and a way to configure it.

The UEFI boot manager

The UEFI spec defines something called the *UEFI boot manager*. (Linux distributions contain a tool called [efibootmgr](#) which is used to manipulate the configuration of the UEFI boot manager). As a sample of what you can expect to find if you do read the UEFI spec, it defines the UEFI boot manager thusly:

"The UEFI boot manager is a firmware policy engine that can be configured by modifying architecturally defined global NVRAM variables. The boot manager will attempt to load UEFI drivers and UEFI applications (including UEFI OS boot loaders) in an order defined by the global NVRAM variables."

Well, that's that cleared up, let's move on. :) No, not really. Let's translate that to Human. With only a reasonable degree of simplification, you can think of the UEFI boot manager as being a boot menu. With a BIOS firmware, your firmware level 'boot menu' is, necessarily, the disks connected to the system at boot time - no more, no less. This is not true with a UEFI firmware.

The UEFI boot manager can be configured - simply put, you can add and remove entries from the 'boot menu'. The firmware can also (it fact the spec requires it to, in various cases) effectively 'generate' entries in this boot menu, according to the disks attached to the system and possibly some firmware configuration settings. It can also be examined - you can look at what's in it.

One rather great thing UEFI provides is a mechanism for doing this *from other layers*: you can configure the system boot behaviour from a booted operating system. You can do all this by using the [efibootmgr](#) tool, once you have Linux booted via UEFI somehow. There are Windows tools for it too, but I'm not terribly familiar with them. Let's have a look at some typical [efibootmgr](#) output, which I stole and slightly tweaked from the Fedora forums:

```
[root@system directory]# efibootmgr -v
BootCurrent: 0002
Timeout: 3 seconds
BootOrder: 0003,0002,0000,0004
Boot0000* CD/DVD Drive BIOS(3,0,00)
Boot0001* Hard Drive HD(2,0,00)
Boot0002* Fedora HD(1,800,61800,6d98f360-cb3e-4727-8fed-5ce0c840365d)File(\EFI\fedora\grubx64.efi)
Boot0003* opensuse HD(1,800,61800,6d98f360-cb3e-4727-8fed-5ce0c840365d)File(\EFI\opensuse\grubx64.efi)
Boot0004* Hard Drive BIOS(2,0,00)P0: ST15000R003-9YN166 .
[root@system directory]#
```

This is a nice clean example I stole and slightly tweaked from the Fedora forums. We can see a few things going on here.

The first line tells you which of the 'boot menu' entries you are *currently* booted from. The second is pretty obvious (if the firmware presents a boot menu-like interface to the UEFI boot manager, that's the timeout before it goes ahead and boots the default entry). The BootOrder is the order in which the entries in the list will be tried. The rest of the output shows the actual boot entries. We'll describe what they actually do later.

If you boot a UEFI firmware entirely normally, without doing any of the tweaks we'll discuss later, what it ought to do is try to boot from each of the 'entries' in the 'boot menu', in the order listed in *BootOrder*. So on this system it would try to boot the entry called 'opensuse', then if that failed, the one called 'Fedora', then 'CD/DVD Drive', and then the second 'Hard Drive'.

UEFI native booting: how it actually works - boot manager entries

What does these entries actually *mean*, though? There's actually a *huge* range of possibilities that makes up rather a large part of the complexity of the UEFI spec all by itself. If you're reading the spec, pour yourself an extremely large shot of gin and turn to the EFI_DEVICE_PATH_PROTOCOL section, but note that this is a generic protocol that's used for other things than booting - it's UEFI's Official Way Of Identifying Devices For All Purposes, used for boot manager entries but also for all sorts of other purposes. Not every possible EFI device path makes sense as a UEFI boot manager entry, for obvious reasons (you're probably not going to get too far trying to boot from your video adapter). But you can certainly have an entry that points to, say, a PXE server, not a disk partition. The spec has lots of bits defining valid non-disk boot targets that can be added to the UEFI boot manager configuration.

For our purposes, though, lets just consider fairly normal disks connected to the system. In this case we can consider three types of entry you're likely to come across.

BIOS compatibility boot entries

Boot0000 and Boot0004 in this example are actually BIOS compatibility mode entries, not UEFI native entries. They have not been added to the UEFI boot manager configuration by any external agency, but generated by the firmware itself - this is a common way for a UEFI firmware to implement BIOS compatibility booting, by generating UEFI boot manager entries that trigger a BIOS-compatible boot of a given device. How they *present this to the user* is a different question, as we'll see later. Whether you see any of these entries or not will depend on your particular firmware, and its configuration. Each of these entries just gives a name - 'CD/DVD Drive', 'Hard Drive' - and says "if this entry is selected, boot this disk (where 'this disk' is `3,0,00` for Boot0000 and `2,0,00` for Boot0004) in BIOS compatibility mode".

'Fallback path' UEFI native boot entries

Boot0001 is an entry (fictional, and somewhat unlikely, but it's for illustrative purposes) that tells the firmware to try and boot from a particular disk, and in UEFI mode not BIOS compatibility mode, but doesn't tell it anything more. It doesn't specify a particular boot target on the disk - it just says to boot the disk.

The UEFI spec defines a sort of 'fallback' path for booting this kind of boot manager entry, which works in principle somewhat like BIOS drive booting: it looks in a standard location for some boot loader code. The details are different, though.

What the firmware will actually do when trying to boot in this way is reasonably simple. The firmware will look through each EFI system partition on the disk in the order they exist on the disk. Within the ESP, it will look for a file with a specific name and location. On an x86-64 PC, it will look for the file `\EFI\BOOT\BOOTx64.EFI`. What it actually looks for is `\EFI\BOOT\BOOT(machine_type short-name).EFI` - 'x64' is the "machine type short-name" for x86-64 PCs. The other possibilities are `BOOTIA32.EFI` (x86-32), `BOOTIA64.EFI` (Itanium), `BOOTARM.EFI` (AArch32 - that is, 32-bit ARM) and `BOOTAA64.EFI` (AArch64 - that is, 64-bit ARM). It will then execute the first qualifying file it finds (obviously, the file needs to be in the executable format defined in the UEFI specification).

This mechanism is not designed for booting permanently-installed OSes. It's more designed for booting hotpluggable, device-agnostic media, like live images and OS install media. And this is indeed what it's usually used for. If you look at a UEFI-capable live or install medium for a Linux distribution or other OS, you'll find it has a GPT partition table and contains a FAT-formatted partition at or near the start of the device, with the GPT partition type that identifies it as an EFI system partition. Within that partition there will be a `\EFI\BOOT` directory with at least one of the specially-named files above. When you boot a Fedora live or install medium in UEFI-native mode, this is the mechanism that is used. The `BOOTx64.EFI` (or whatever) file handles the rest of the boot process from there, booting the actual operating system contained on the medium.

Full UEFI native boot entries

Boot0002 and Boot0003 are 'typical' entries for operating systems permanently installed to permanent storage devices. These entries show us the full power of the UEFI boot mechanism, by not just saying "boot from this disk", but "boot this specific bootloader in this specific location on this specific disk", using all the 'groundwork' we talked about above.

Boot0002 is a boot entry produced by a UEFI-native Fedora installation. Boot0003 is a boot entry produced by a UEFI-native OpenSUSE installation. As you may be able to tell, all they're saying is "load this file from this partition". The partition is the `HD(1,800,61800,6d98f360-cb3e-4727-8fed-5c00c040365d)` bit: that's referring to a specific partition (using the `EFI_DEVICE_PATH_PROTOCOL`, which I'm really not going to attempt to explain in any detail - you don't necessarily need to know it, if you interact with the boot manager via the firmware interface and `efibootmgr`). The file is the `File(\EFI\opensuse\grubx64.efi)` bit: that just means "load the file in this location on the partition we just described". The partition in question will almost always be one that qualifies as an EFI system partition, because of the considerations above: that's the type of partition we can trust the firmware to be able to access.

This is the mechanism the UEFI spec provides for operating systems to make themselves available for booting: the operating system is intended to install a bootloader which loads the OS kernel and so on to an EFI system partition, and add an entry to the UEFI boot manager configuration with a name - obviously, this will usually be derived from the operating system's name - and the location of the bootloader (in EFI executable format) that is intended for loading that operating system.

Linux distributions use the `efibootmgr` tool to deal with the UEFI boot manager. What a Linux distribution actually does, so far as bootloading is concerned, when you do a UEFI native install is really pretty simple: it creates an EFI system partition if one does not already exist, installs an EFI boot loader with an appropriate configuration - often `grub2-efi`, but there are others - into a correct path in the EFI system partition, and calls `efibootmgr` to add an appropriately-named UEFI boot manager entry pointing to its boot loader. Most distros will use an existing EFI system partition if there is one, though it's perfectly valid to create a new one and use that instead: as we've noted, UEFI is a permissive spec, and if you follow the design logically, there's really no problem with having just as many EFI system partitions as you want.

Configuring the boot process (the firmware UI)

The above describes the basic mechanism the UEFI spec defines that manages the UEFI boot process. It's important to realize that your firmware user interface may well not represent this mechanism very clearly. Unfortunately, the spec intentionally refrains from defining how the boot process should be represented to the user or how the user should be allowed to configure it, and what that means - since we're dealing with [firmware engineers](#) - is that every firmware does it differently, and some do it insanely.

Many firmwares do have fairly reasonable interfaces for boot configuration. A good firmware design will at least show you the boot order, with a reasonable representation of the entries on it, and let you add or remove entries, change the order, or override the order for a specific boot (by changing it just for that boot, or directly instructing the firmware to boot a particular menu entry, or even giving you the option to simply say "boot this disk", either in BIOS compatibility mode or UEFI 'fallback' mode - my firmware does this). Such an interface will often show 'full' UEFI native boot entries (like the Fedora and openSUSE examples we saw earlier) only by their name; you have to examine the `efibootmgr -v` output to know precisely what these entries will actually try and *do* when invoked.

Some firmwares try to abstract and simplify the configuration, and may do a good or a bad job of it. For instance, if you have an option to 'enable or disable' BIOS compatibility mode, what it'll really likely *do* is configure whether the firmware adds BIOS compatibility entries for attached drives to the UEFI boot manager configuration or not. If you have an option to 'enable or disable' UEFI native booting, what likely really happens when you 'disable' it is that the firmware changes the UEFI boot manager configuration to leave all UEFI-native entries out of the `BootOrder`.

The key point to remember is that any configuration option inside your firmware interface which is to do with booting is really, behind the scenes, configuring the behaviour of the UEFI boot manager. If you understand all the stuff we've discussed above, you may well find it easier to figure out what's *really* happening when you twiddle the knobs your firmware interface exposes.

In the BIOS world, you'll remember, you don't always find that systems are configured to try and boot from removable drives - CD, USB - before booting from permanent drives. Some are, and some aren't. Some will try CD before the hard disks, but not USB. People have got fairly used to having to check the BIOS configuration to ensure the boot order is 'correct' when trying to install a new operating system.

This applies to the UEFI world too, but because of the added flexibility/complexity of the UEFI boot manager mechanism, it can look unfamiliar and scary.

If you want to ensure that your system tries to boot from removable devices using the 'fallback' mechanism before it tries to boot 'permanent' boot entries - as you will want to do if you want to, say, install Fedora - you need this to be the default for your firmware, or you need to be able to tell the firmware this. Depending on your firmware's interface, you may find there is a 'menu entry' for each attached removable device and you just have to adjust the boot order to put it at the top of the list, or you may find that there is the mechanism to directly request 'UEFI fallback boot of this particular disk', or you may find that the firmware tries to abstract the configuration somehow. We just don't know, and that makes writing instructions for this quite hard. But now you broadly understand how things work behind the scenes, you may find it easier to understand your firmware user interface's representation of that.

Configuring the boot process (from an operating system)

As we've noted above, unlike in the BIOS world, you can actually configure the UEFI boot process from the operating system level. If you have an insane firmware, you may *have* to do this in order to achieve what you want.

You can use the `efibootmgr` tool mentioned earlier to add, delete and modify entries in the UEFI boot manager configuration, and actually do quite a lot of other stuff with it too. You can change the boot order. You can tell it to boot some particular entry in the list on the next boot, instead of using the *BootOrder* list (if you or some other tool has configured this to happen, your `efibootmgr -v` output will include a *BootNext* item stating which menu entry will be loaded on the next boot). There are tools for Windows that can do this stuff from Windows, too. So if you're really struggling to manage to do whatever it is you want to do with UEFI boot configuration from your firmware interface, but you *can* boot a UEFI native operating system of some kind, you may want to consider doing your boot configuration from that operating system rather than from the firmware UI.

So to recap:

- Your UEFI firmware contains something very like what you think of as a boot menu.
- You can query its configuration with `efibootmgr -v`, from any UEFI-native boot of a Linux OS, and also *change* its configuration with `efibootmgr` (see the [man page](#) for details).
- This 'boot menu' can contain entries that say 'boot this disk in BIOS compatibility mode', 'boot this disk in UEFI native mode via the fallback path' (which will use the 'look for BOOT(something).EFI' method described above), or 'boot the specific EFI format executable at this specific location (almost always on an EFI system partition)'.
- The nice, clean design that the UEFI spec is trying to imply is that all operating systems should install a bootloader of their own to an EFI system partition, add entries to this 'boot menu' that point to themselves, and butt out from trying to take control of booting anything else.
- Your firmware UI has free rein to represent this mechanism to you in whatever way it wants, and it may do this well, or it may do this poorly.

Installing operating systems to UEFI-based computers

Let's have a quick look at some specific consequences of the above that relate to installing operating systems on UEFI computers.

UEFI native and BIOS compatibility booting

Here's a very very simple one which people sometimes miss:

- If you boot the installation medium in 'UEFI native' mode, it will do a UEFI native install of the operating system: it will try to write an EFI-format bootloader to an EFI system partition, and attempt to add an entry to the UEFI boot manager 'boot menu' which loads that bootloader.
- If you boot the installation medium in 'BIOS compatibility' mode, it will do a BIOS compatible install of the operating system: it will try to write an MBR-type bootloader to the magic MBR space on a disk.

This applies (with one minor caveat I'm going to paper over for now) to all OSes of which I'm aware. So you probably want to make sure you understand how, in your firmware, you can choose to boot a removable device in UEFI native mode and how you can choose to boot it in BIOS compatibility mode, and make sure you pick whichever one you actually want to use for your installation.

You really cannot do a completely successful UEFI-native installation of an OS if you boot its installation medium in BIOS compatibility mode, because the installer will not be able to configure the UEFI boot manager (this is only possible when booted UEFI-native).

It is theoretically possible for an OS installer to install the OS in the BIOS style - that is, write a bootloader to a disk's MBR - after being booted in UEFI native mode, but most of them won't do this, and that's probably sensible.

Finding out which mode you're booted in

It is possible that you might find yourself with your operating system installer booted, and not sure whether it's actually booted in UEFI native mode or BIOS compatibility mode. Don't panic! It's pretty easy to find out which, in a few different ways. One of the easiest is just to try and talk to the UEFI boot manager. If what you have booted is a Linux installer or environment, and you can get to a shell (ctrl-alt-f2 in the Fedora installer, for instance), run `efibootmgr -v`. If you're booted in UEFI native mode, you'll get your UEFI boot manager configuration, as shown above. If you're booted in BIOS compatibility mode, you'll get something like this:

```
Fatal: Couldn't open either sysfs or procfs directories for accessing EFI variables.
Try 'modprobe efivars' as root.
```

If you've booted some other operating system, you can try running a utility native to that OS which tries to talk to the UEFI boot manager, and see if you get sensible output or a similar kind of error. Or you can examine the system logs and search for 'efi' and/or 'uefi', and you'll probably find some kind of indication.

Enabling UEFI native boot

To be bootable in UEFI native mode, your OS installation medium must obviously actually comply with all this stuff we've just described: it's got to have a GPT partition table, and an EFI system partition with a bootloader in the correct 'fallback' path - \EFI\BOOT\BOOTx64.EFI (or the other names for the other platforms). If you're having trouble doing a UEFI native boot of your installation medium and can't figure out why, check that this is actually the case. Notably, when using the `livecd-iso-to-disk` tool to write a Fedora image to a USB stick, you must pass the `--efi` parameter to configure the stick to be UEFI bootable.

Forcing BIOS compatibility boot

If your firmware seems to make it very difficult to boot from a removable medium in BIOS compatibility mode, but you really want to do that, there's a handy trick you can use: just make the medium not UEFI native bootable at all. You can do this pretty easily by just wiping all the EFI system partitions. (Alternatively, if using `livecd-iso-to-disk` to create a USB stick from a Fedora image, you can just leave out the `--efi` parameter and it won't be UEFI bootable). If at that point your firmware refuses to boot it in BIOS compatibility mode, commence swearing at your firmware vendor (if you didn't already).

Disk formats (MBR vs. GPT)

Here's another very important consideration:

- If you want to do a 'BIOS compatibility' type installation, you probably want to install to an MBR formatted disk.
- If you want to do a UEFI native installation, you probably want to install to a GPT formatted disk.

Of course, to make life complicated, many firmwares *can* boot BIOS-style from a GPT formatted disk. UEFI firmwares are in fact technically *required* to be able to boot UEFI-style from an MBR formatted disk (though we are not particularly confident that they all really can). But you really should avoid this if at all possible. This consideration is quite important, as it's one that trips up quite a few people. For instance, it's a bad idea to boot an OS installer in UEFI native mode and then attempt to install to an MBR formatted disk without reformatting it. This is very likely to fail. Most modern OS installers will automatically reformat the disk in the correct format if you allow them to completely wipe it, but if you try and tell the installer 'do a UEFI native installation to this MBR formatted disk and don't reformat it because it has data on it that I care about', it's very likely to fail, even though this configuration is technically covered in the UEFI specification. Specifically, Windows and Fedora at least explicitly disallow this configuration.

Checking the disk format

You can use the `parted` utility to check the format of a given disk:

```
[adam@adan Downloads]$ sudo parted /dev/sda
GNU Parted 3.1
Using /dev/sda
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted) p
Model: ATA C300-CTFDDAC128M (scsi)
Disk /dev/sda: 128GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      1049KB  525MB   524MB   primary  ext4         boot
 2      525MB   128GB   128GB   primary                lvm

(parted)
```

See that **Partition table: msdos** ? This is an MBR/MS-DOS formatted disk. If it was GPT-formatted, that would say `gpt`. You can reformat the disk with the other type of partition table by doing `mklabel gpt` or `mklabel msdos` from within parted. **This will destroy the contents of the disk.**

With most OS installers, if you pick a disk configuration that blows away the entire contents of the target disk, the installer will automatically reformat it using the most appropriate configuration for the type of installation you're doing, but if you want to use an existing disk without reformatting it, you're going to have to check how it's formatted and take this into account.

Handling EFI system partition if doing manual partitioning

I can only give authoritative advice for Fedora here, but the gist may be useful for other distros / OSes.

If you allow Fedora to handle partitioning for you when doing a UEFI native installation - and you use a GPT-formatted disk, or allow it to reformat the disk (by deleting all existing partitions) - it will handle the EFI system partition stuff for you.

If you use custom partitioning, though, it will expect you to provide an EFI system partition for the installer to use. If you don't do this, the installer will complain (with a somewhat confusing error message) and refuse to let you start the installation.

So if you're doing a UEFI native install and using custom partitioning, you need to ensure that a partition of the 'EFI system partition' type is mounted at `/boot/efi` - this is where Fedora expects to find the EFI system partition it's using. If there is an existing EFI system partition on the system, just set its mount point to `/boot/efi`. If there is not an EFI system partition yet, create a partition, set its type to *EFI system partition*, make it at least 200MB big (500MB is good), and set its mount point to `/boot/efi`.

A specific example

To boil down the above: if you bought a Windows 8 or later system, you *almost* certainly have a UEFI native install of Windows to a GPT-formatted disk. This means that if you want to install another OS alongside that Windows install, you almost certainly want to do a UEFI-native installation of your other OS. If you don't like all this UEFI nonsense and want to go back to the good old world you're familiar with, you will, I'm afraid, have to blow away the UEFI-native Windows installation, and it would be a good idea to reformat the disk to MBR.

Implications and Complications

So, that's how UEFI booting works, at least a reasonable approximation. When I describe it like that, it almost all makes sense, right?

However, all is not sweetness and light. There are problems. There always are.

Attentive readers may have noticed that I've talked about the UEFI spec *providing* a mechanism. This is accurate, and important. As the UEFI spec is a 'broad consensus' sort of thing, one of its major shortcomings (looked at from a particular perspective) is that it's nowhere near prescriptive enough.

If you read the UEFI spec critically, its basic approach is to define a set of functions that UEFI compliant firmwares must support. What it doesn't do a lot of at all is strictly requiring things to be done in any particular way, or not done in any particular way.

So: the spec says that a system firmware must do all the stuff I've described above, in order to be considered a UEFI-compliant firmware. The spec, however, doesn't talk about what operating systems 'should' or 'must' do at all, and it doesn't say that firmwares must *not* support (or no-one may expect them to support, or whatever)...anything at all. If you're making a UEFI firmware, in other words, you have to support GPT formatted disks, and FAT-formatted EFI system partitions, and you must read UEFI boot manager entries in the standard format, and you must do this and that and the other - but you can also do any *other* crap you like.

It's pretty easy to read certain implications from the spec - it carefully sets up this nice mechanism for handling OS (or other 'bootable thing') selection at the

firmware level, for instance, with the clear implication "hey, it'd be great if all OSes were written to this mechanism". But the UEFI spec doesn't *require* that, and neither does any other widely-respected specification.

So, what happens in the real world is that we wind up with really dumb crap. Apple, for instance, ships at least *some* Macs with their bootloaders in an HFS+ partition. The spec says a UEFI-compliant firmware must support UEFI FAT partitions with the specific GPT partition type that identifies them as an "EFI system partition", but it doesn't say the firmware can't *also* recognize some other filesystem type and load a bootloader from that. (Whether you consider such a partition to be an "EFI system partition" or not is an interesting philosophical conundrum, but let's skate right over that for now).

The world would pretty clearly be a better place if everyone just damn well used the EFI system partition format the spec goes to such great pains to define, but Apple is Apple and we can't have nice things, so Apple went right ahead and wrote firmwares that *also* can read and load code from HFS+ partitions, and now everyone else has to deal with that or tell Macs to go and get boned. Apple also goes quite a long way beyond the spec in its boot process design, and if you want your alternative OS to show up on its graphical boot menu with a nice icon and things, you have to do more than what the UEFI spec would suggest.

There are various similar incredibly annoying corner cases we've come across, but let's not go into them all right now. This post is long enough.

Also, as we noted earlier, the spec makes no requirements as to how the mechanism should be represented to the user. So if a couple of software companies write OSes to behave 'nicely' according to the conventions the spec is clearly designed to back, and install EFI boot loaders and define EFI boot manager entries with nice clear names - like, oh say, "Fedora" and "Windows" - they are implicitly relying on the firmware to then give the user *some* kind of sane interface *somewhere* relatively discoverable that lets them choose to boot "Windows" or "Fedora". The more firmwares don't do a good job of this, the less willing OS engineers will be to rely on the 'proper' conventions, and the more likely they'll be to start rebuilding ugly hacks above the firmware level.

To be fair, we could do somewhat more at the OS level. We could present all those neat efibootmgr capabilities rather more obviously - we can use that 'don't respect BootOrder on the next boot, but instead boot *this*' capability, for instance, and have 'Reboot to Windows' as an option. It'd be kinda nice if someone looked at exposing all this functionality somewhere more obvious than efibootmgr. Windows 8 systems do use this, to some extent - you can reboot your system to the firmware UI from the Windows 8 settings menus, for instance. But still.

All this is really incredibly frustrating, because UEFI is so *close* to making things really a lot better. The BIOS approach doesn't provide any kind of convention or standard for multibooting at all - it *has* to be handled entirely above the firmware level. We (the industry) could have come up with some sort of convention for handling multiboot, but we never did, so it just became a multiple-decade epic fail, where each operating system came up with its own approach and lots of people wrote their own bootloaders which tried to subsume all the operating systems and all the operating systems and independent bootloaders merrily fought like cats in a sack. I mean, pre-UEFI multibooting is such a cluster*^k it's not even worth going into, it's broken sixteen ways from Sunday by definition.

If UEFI - or a spec built on top of it - had just *mandated* that everybody follow the conventions UEFI carefully establishes, and *mandated* that firmwares *provide* a sensible user interface, the win would have been epic. But it doesn't, so it's entirely possible that in a UEFI world things will be *even worse than they were in the BIOS world*. If many more firmwares show up that don't present a good UI for the UEFI boot manager mechanism, what could happen is that OS vendors give up on the UEFI boot manager mechanism (or decide to support it *and* alternatives, because choice!) and just reinvent the entire goddamn nightmare of BIOS multibooting *on top of UEFI* - and we'll all have to deal with all of that, *plus* the added complication of the UEFI boot manager layer. You'll have multiple bootloaders fighting to load multiple operating systems all on top of the whole UEFI boot manager mechanism which is just throwing a whole bunch of other variables into the equation.

This is not a prospect filling the mind of anyone who's had to think about it with joy.

Still, it's important to recognize that the sins of UEFI in this area are sins of omission - they are not sins of commission, and they're not really the result of evil intent on anyone's part. The entity you should really be angry with if you have an idiotic system firmware that doesn't give you good access to the UEFI boot manager mechanism is not the UEFI forum, or Microsoft, and it *certainly* isn't Fedora and even more *certainly* isn't me :). The entity you should be angry at is your system/motherboard manufacturer and the goddamn incompetents they hired to write the firmware, because the UEFI spec makes it really damn clear to anyone with two brain cells to rub together that it would be a very good idea to provide some kind of useful user interface to the UEFI boot manager, and any firmware which doesn't do so is crap code by definition. Yes, the UEFI forum should've realized that firmware engineers couldn't code their way out of a goddamned paper bag and just ordered them to do so, but still, it's ultimately the firmware engineers who should be lined up against the nearest wall.

Wait, we can simplify that. "Any firmware is crap code". Usually pretty accurate.

Secure Boot

So now we come, finally, to Secure Boot.

Secure Boot is not magic. It's not complicated. OK, that's a lie, it's incredibly complicated, but the *theory* isn't very complicated. And no, Secure Boot itself is not evil. I am entirely comfortable stating this as a fact, and you should be too, unless you think GPG is evil.

Secure Boot is defined in chapter 28 of the UEFI spec (2.4a, anyway). It's actually a pretty clever mechanism. But what it does can be described very, very simply. It says that the firmware can contain a set of signatures, and refuse to run any EFI executable which is not signed with one of those signatures.

That's it. Well, no, it really isn't, but that's a reasonably acceptable simplification. Security is hard, so there are all kinds of wibbly bits to implementing a really secure bootchain using Secure Boot, and mjb59 can tell you all about them, or you can pour another large shot of gin and read the whole of chapter 28. But that's the basic idea.

Using public key cryptography to verify the integrity of something is hardly a radical or evil concept. Pretty much all Linux distributions depend on it - we sign our packages and have our package managers go AWOOGA AWOOGA if you try to install a package which isn't signed with one of our keys. This isn't us being evil, and I don't think anyone's ever accused an OS of being evil for using public key cryptographic signing to establish trust in this way. Secure Boot is literally this exact same perfectly widely accepted mechanism, applied to the boot chain. Yet because a bunch of journalists *wildly* grasped the wrong end of the stick, it's widely considered to be slightly more evil than Hitler.

Secure Boot, as defined in the UEFI spec, says nothing at all about what the keys the firmware trusts should be, or where they should come from. I'm not going to go into all the goddamn details, because it gets stultifyingly boring and this post is too long already. But the executive summary is that the spec is utterly and entirely about defining a *mechanism* for doing cryptographic verification of a boot chain. It does not really even consider any kind of icky questions about the *policy* for doing so. It does nothing evil. It is as flexible as it could reasonably be, and takes care to allow for all the mechanisms involved to be configurable at multiple levels. The word 'Microsoft' is not mentioned. It is not in any way, shape, or form a secret agenda for Microsoft's domination of the world. If you doubt this, at the very bloody least, go and *read* it. I've given you all the necessary pointers. There is literally not a single legitimate reason I can think of for anyone to be *angry* with the idea "hey, it'd be neat if there was a mechanism for optional cryptographic verification of bootloader code in this firmware specification". None. Not one.

Secure Boot in the real world

Most of the unhappiness about Secure Boot is not really about Secure Boot the mechanism - whether the people expressing that unhappiness *think* it is or not - but about specific implementations of Secure Boot in the real world.

The only one we really care about is Secure Boot as it's implemented on PCs shipped with Microsoft Windows 8 or higher pre-installed.

Microsoft has these things called the [Windows Hardware Certification Requirements](#) . There they are. They are not Top Secret, Eyes Only, You Will Be Fed To Bill Gates' Sharks After Reading - they're right there on the Internet for anyone to read.

If you want to get cheap volume licenses of Windows from Microsoft to pre-install on your computers and have a nice "reassuring" "Microsoft Approved!" sticker or whatever on the case, you have to comply with these requirements. That's all the force they have: they are not *actually* a part of the law of the United States or any other country, whatever some people seem to believe. Bill Gates cannot feed you to his sharks if you sell a PC that doesn't comply with these requirements, so long as you don't want a cheap copy of Windows to pre-install and a nice sticker. There is literally no requirement for a PC sold *outside* the Microsoft licensing program to configure Secure Boot in any particular way, or include Secure Boot at all. A PC that claims to have a UEFI 2.2 or later compliant firmware must implement Secure Boot, but can ship with it configured in literally absolutely any way it pleases (including turned off).

If you're going to have very loud opinions about Secure Boot, you have zero excuse for not going and reading the Microsoft certification requirements. Right now. I'll wait. You can search for "Secure Boot" to get to the relevant bit. It starts at "System.Fundamentals.Firmware.UEFI.SecureBoot".

You should read it. But here is a summary of what it says.

Computers complying with the requirements must:

- Ship with Secure Boot turned on (except for servers)
- Have Microsoft's key in the list of keys they trust
- Disable BIOS compatibility mode when Secure Boot is enabled (actually the UEFI spec requires this too, if I read it correctly)
- Support signature blacklisting

x86 computers complying with the requirements must additionally:

- **Allow** a physically present person to disable Secure Boot
- **Allow** a physically present person to enable Custom Mode, and modify the list of keys the firmware trusts

ARM computers complying with the requirements must additionally:

- **NOT allow** a physically present person to disable Secure Boot
- **NOT allow** a physically present person to enable Custom Mode, and modify the list of keys the firmware trusts

Yes. You read that correctly. The Microsoft certification requirements, for x86 machines, *explicitly require implementers to give a physically present user complete control over Secure Boot* - turn it off, or completely control the list of keys it trusts. Another important note here is that while the certification requirements state that the out-of-the-box list of trusted keys must *include* Microsoft's key, they don't say, for e.g., that it must *not include* any other keys. The requirements explicitly

and intentionally allow for the system to ship with any number of other trusted keys, too.

These requirements aren't present entirely out of the goodness of Microsoft's heart, or anything - they're present in large part because other people explained to Microsoft that if they weren't present, it'd have a hell of a lawsuit on its hands - but they are present. Anyone who actually understands UEFI and Secure Boot cannot possibly read the requirements any other way, they are extremely clear and unambiguous. They both clearly *intend to and succeed in* ensuring the owner of a certified system has complete control over Secure Boot.

If you have an x86 system that claims to be Windows certified but does not allow you to disable Secure Boot, it is in direct violation of the certification requirements, and you should certainly complain very loudly to someone. If a lot of these systems exist then we clearly have a problem and it might be time for that giant lawsuit, but so far I'm not aware of this being the case. All the x86-based, Windows-certified systems I've seen *have* had the 'disable Secure Boot' option in their firmwares.

Now, for ARM machines, the requirements are significantly more evil: they state exactly the opposite, that it must *not* be possible to disable Secure Boot and it must *not* be possible for the system owner to change the trusted keys. This is bad and wrong. It makes Microsoft-certified ARM systems into a closed shop. But it's worth noting it's no *more* bad or wrong than most other major ARM platforms. Apple locks down the bootloader on all iDevices, and most Android devices also ship with locked bootloaders.

If you're planning to buy a Microsoft-certified ARM device, be aware of this, and be aware that you will not be in control of what you can boot on it. If you don't like this, don't buy one. But also don't buy an iDevice, or an Android device with a locked bootloader (you can buy Android devices with unlocked or unlockable bootloaders, still, but you have to do your research).

As far as x86 devices go, though, right now, Microsoft's certification requirements actually *explicitly protect* your right to determine what can boot on your system. This is good.

Recommendations

The following are AdamW's General Recommendations On Managing System Boot, offered with absolutely no guarantees of accuracy, purity or safety.

- **If you can possibly manage it, have one OS per computer.** If you need more than one OS, buy more computers, or use virtualization. If you can do this everything is very simple and it doesn't much matter if you have BIOS or UEFI firmware, or use UEFI-native or BIOS-compatible boot on a UEFI system. Everything will be nice and easy and work. You will whistle as you work, and be kind to children and small animals. All will be sweetness and light. Really, do this.
- **If you absolutely must have more than one OS per computer, at least have one OS *per disk*.** If you're reasonably comfortable with how BIOS-style booting works and you don't think you need Secure Boot, it's pretty reasonable to use BIOS-compatible booting rather than UEFI-style booting in this situation on a UEFI-capable system. You'll probably have less pain to deal with and you won't really lose anything. With one OS per disk you can also mix UEFI-native and BIOS-compatible installations.
- **If you absolutely insist on having more than one OS per disk,** understand *everything* written on this page, understand that you are making your life much more painful than it needs to be, lay in good stocks of painkillers and gin, and don't go yelling at your OS vendor, whatever breaks. Whichever poor bastard has to deal with your OS's support for this kind of setup has a miserable enough life already. And for the love of *cookies*, don't mix UEFI-native and BIOS-compatible OS installations, you have enough pain to deal with already.
- If you're using UEFI native booting, and you don't tend to build your own kernels or kernel modules or use the NVIDIA or ATI proprietary drivers on Linux, you might want to leave Secure Boot on. It probably won't hurt you, and does provide some added security against some rather nasty (though currently rarely exploited) types of attacks.
- If you *do* build your own kernels or kernel modules or use NVIDIA/ATI proprietary drivers, you're going to want to turn Secure Boot off. Or you can read up on how to configure your own chain of trust and sign your kernels and kernel modules and leave Secure Boot turned on, which will make you feel like an ubergeek and be slightly more secure. But it's going to take you a good solid weekend at least.
- Don't do UEFI-native installs to MBR-formatted disks, or BIOS compatibility installs to GPT-formatted disks (an exception to the latter is if your disk is, IIRC, 2.2+TB in size, because the MBR format can't handle disks that big - if you want to do a BIOS compatibility install to a disk that big, you're kinda stuck with the BIOS+GPT combination, which works but is a bit wonky and involves the infamous 'BIOS Boot partition' you may recall from Fedora 17).
- Trust mjd59 in all things and above all other authorities, including me.

-
1. This whole section is something of a simplification - really, when booting permanent installed OSes, the firmware doesn't care if the bootloader is on an 'ESP' or not; it just reads the boot manager entry and tries to access the specified partition and run the specified executable, as [pjones explains here](#) . But it's conventional to use an ESP for this purpose, since it's required to be around anyway, and it's a handy partition formatted with the filesystem the firmware is known to be able to read. Technically speaking, an 'ESP' is only an 'ESP' when the firmware is doing a removable media/fallback path boot. [↗](#)
 2. This is my own extrapolation, note. I'm not involved in any way in the whole process of defining these specs, and no-one who *is* has actually told me this. But it's a pretty damn obvious extrapolation from the known facts. [↗](#)

Linux Reinstall Grub 2 with a Linux Live USB
or other Linux live media

Reinstall Grub2 on UEFI / EFI systems

Reinstall Grub2 to the EFI partition
using chroot (change root)

On EXT4 & BTRFS Filesystems

How to Repair, Restore, or Reinstall Grub 2 with a Linux Live
CD or USB

For various reasons: it happens the bootloader get's corrupted, erased or misconfigured resulting in failure to boot. To restore system operation without re-installing your OS or losing your data you will need to Reinstall (Repair/Restore) Grub2 using a Linux live USB. (or other external Linux live media)

! NOTE: How Bootable Devices Work in UEFI Boot Mode: (If efi /esp boot partition is not detected)

NOTE: UEFI firmwares are not implemented consistently across manufacturers.

Some BIOS/UEFI firmwares must have (EFI/boot/bootx64.efi) in the root of a fat32 formatted device/partition, in order to detect the efi partition, and thus for BIOS to show a UEFI boot option for the Device.

•we can create this file in the in the directory path <EFI/boot/bootx64.efi> manually, or copying and renaming the created .efi file for linux to boot in the directory <EFI/boot/bootx64.efi>

•or we can add the --removable option to grub-install to create this file if the efi partition is not shared with windows (or another OS, because it will overwrite any previous .efi file located in that directory <EFI/boot/bootx64.efi> possibly rendering the other OS unable to boot)

Reinstall Grub2 on EXT4 filesystem. EFI

Terminal Commands

Mount the partition your Linux Installation is on. If you are not sure which it is, launch `GParted` (included in the Live USB) or `lsblk` or `lsblk -f` or `fdisk -l` commands. Replace the XY with the drive letter, and partition number, for example:
`sudo mount /dev/sda2 /mnt.`

```
sudo mount /dev/sdXY /mnt
```

mount the efi partition to the `/boot/efi` directory of the linux installation

```
sudo mount /dev/sdXY /mnt/boot/efi
```

Now bind the directories that grub needs access to to detect other operating systems, like so.

```
sudo mount --bind /dev /mnt/dev
sudo mount --bind /dev/pts /mnt/dev/pts
sudo mount --bind /proc /mnt/proc
sudo mount --bind /sys /mnt/sys
```

copy and paste: `sudo mount -B /dev /mnt/dev && sudo mount -B /dev/pts /mnt/dev/pts && sudo mount -B /proc /mnt/proc && sudo mount -B /sys /mnt/sys`
* you can use `-B` instead of `--bind` both are equivalent

Now we jump into that using `chroot`.

```
sudo chroot /mnt
```

Now install, check, and update grub.

Now you install grub to the efi partition, for example: `grub-install /dev/sda1` `grub-install --recheck /dev/sda`.

caution then you may want to use the `--removable` option: `grub-install <efi partition> --removable`

```
grub-install /dev/sdXY
```

```
grub-install --recheck /dev/sdXY
```

```
update-grub
```

Now grub is back, all that is left is to exit the chrooted system and unmount everything.

```
exit
sudo umount /mnt/sys
sudo umount /mnt/proc
sudo umount /mnt/dev/pts
sudo umount /mnt/dev
sudo umount /mnt
```

copy and paste: `sudo umount /mnt/sys; sudo umount /mnt/proc; sudo umount /mnt/dev/pts; sudo umount /mnt/dev; sudo umount /mnt`

Shut down and turn your computer back on, and you will be met with the default Grub2 screen.

**You may need to Remove/Edit the previous EFI partition mount point in /etc/fstab
If you still encounter problems booting/entering into your Linux System**

**You may need to Edit the /etc/crypttab if the linux system was installed in a partition encrypted with
LUKS/cryptsetup/dm-crypt:
Edit the /etc/crypttab with Linux installation's filesystem uuid**

You may want to update grub or re-install within in the Linux system however you like it
Congratulations, you have just Repaired/Restored/Reinstalled Grub 2 with a Linux Live USB!

Reinstall Grub2 on BTRFS Filesystem EFI

BTRFS Filesystem with /@ subvolume

**:bind mounting subvolumes BTRFS (if @ subvolume
is present) & reinstall grub2**

On a BTRFS filesystem where the OS is installed in a subvolume (looks like a directory, instead of being in the root of the filesystem), usually this is /@. we need to **bind mount** the subvolume to /mnt,
or specify the subvol option in the mount command: `mount -o subvol=/@ <path-to-mount-to>`

A **bind mount** takes an existing directory tree and replicates it under a different mount point. The directories and files in the bind mount are the same as the original. Any modification on one side is immediately reflected on the other side, since the two views show the same data.

The **subvol** mount option is used to mount the subvolume of a Btrfs filesystem using its relative path.

Terminal Commands (for BTRFS with system in a subvolume)

Bind Mount the partition your Linux Installation is on (if within a subvolume). If you are not sure which it is, launch GParted (included in the Live USB) or `lsblk` or `lsblk -f` commands. Replace the XY with the drive letter, and partition number, for example: `sudo mount /dev/sda2 /mnt`.

```
sudo mount /dev/sdXY -o subvol=@ /mnt
```

* or `sudo mount --bind </dev/sdXY/@`s mount point> /mnt`

mount the efi partition to the `/boot/efi` directory of the linux installation

```
sudo mount /dev/sdXY /mnt/boot/efi
```

Now bind the directories that grub needs access to to detect other operating systems, like so.

```
sudo mount --bind /dev /mnt/dev
sudo mount --bind /dev/pts /mnt/dev/pts
sudo mount --bind /proc /mnt/proc
sudo mount --bind /sys /mnt/sys
```

copy and paste: `sudo mount -B /dev /mnt/dev && sudo mount -B /dev/pts /mnt/dev/pts && sudo mount -B /proc /mnt/proc && sudo mount -B /sys /mnt/sys`

* you can use `-B` instead of `--bind` both are equivalent

Now we jump into that using `chroot`.

```
sudo chroot /mnt
```

Now install, check, and update grub.

Now you install grub to the efi partition, for example: `grub-install /dev/sda1`
`grub-install --recheck /dev/sda`.

caution then you may want to use the `--removable` option: `grub-install <efi partition> --removable`

```
grub-install /dev/sdXY
```

```
grub-install --recheck /dev/sdXY
```

```
update-grub
```

Now grub is back, all that is left is to exit the chrooted system and unmount everything.

```
exit
```

```
sudo umount /mnt/sys
```

```
sudo umount /mnt/proc
```

```
sudo umount /mnt/dev/pts
```

```
sudo umount /mnt/dev
```

```
sudo umount /mnt
```

copy and paste: `sudo umount /mnt/sys; sudo umount /mnt/proc; sudo umount /mnt/dev/pts; sudo umount /mnt/dev; sudo umount /mnt`

Shut down and turn your computer back on, and you will be met with the default Grub2 screen.

**You may need to Remove/Edit the previous EFI partition mount point in /etc/fstab
If you still encounter problems booting/entering into your Linux System**

**You may need to Edit the /etc/crypttab if the linux system was installed in a partition encrypted with
LUKS/cryptsetup/dm-crypt:
Edit the /etc/crypttab with Linux installation's filesystem uuid**

You may want to update grub or re-install within in the Linux system however you like it

Congratulations, you have just Repaired/Restored/Reinstalled Grub 2 with a Linux Live USB!

HARDWARE

Commands to View System Hardware Information:

```
- $ inxi -Fx
- $ dmidecode
- $ dmidecode -t memory
- $ sudo lshw
- $ hwdmidecode
- $ lspci
- $ lspci -v
- $ lspci
- $ lsusb
- $ lsusb -v
```

Commands To View Filesystem Information:

```
- $ lsblk
- $ lsblk -f
- $ blkid
- $ fdisk -l
- $ fdisk -l </dev/device>
- $ parted -l
- $ parted -l </dev/device>
```

Commands To View System Information:

```
- $ htop
- $ btop
- $ i7z
- $ uname -a
- $ uname -r
- $ neofetch
- $ screenfetch
```

* See man pages for manual pages for the above commands:
more information: `man <command>`

* you may cat the results of the above commands to a
file: `<command> | cat > filename.txt`

VIRTUALIZATION

Virtualization is the process of running a virtual instance of a computer system in a layer abstracted from the actual hardware.

Most commonly, it refers to running multiple operating systems on a computer system simultaneously.

Uses Of Virtualization:

virtualization offers the ability o run different operating systems simultaneously and/or run applications meant for a different operating system without having to switch computers or reboot into a different system.

It also allows for isolation, keeping programs running inside of a virtual machine safe from the processes taking place in another virtual machine on the same host.

Hypervisors

A hypervisor is a program for creating and running virtual machines. Hypervisors have traditionally been split into two classes:

type 1: bare metal

type 2: hosted

Type 1: or “Bare Metal”

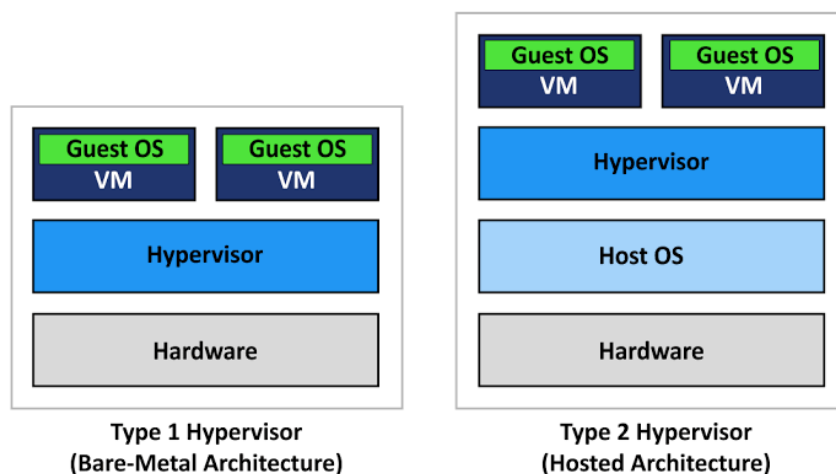
Type one, or "bare metal" hypervisors that run guest virtual machines directly on a system's hardware, essentially behaving as an operating system.

Bare Metal is just another way of saying “Dedicated Server.”

This is a single tenant environment with direct access to underlying hardware technology without any hypervisor overhead.

Type 2: or “Hosted”

Type two, or "hosted" hypervisors behave more like traditional applications that can be started and stopped like a normal program. In modern systems, this split is less prevalent, particularly with systems like KVM. KVM, short for kernel-based virtual machine, is a part of the Linux kernel that can run virtual machines directly, although you can still use a system running KVM virtual machines as a normal computer itself



We will be focusing on Type 2 Hypervisors: Hosted

[Software Covered Here For Virtualization Type 2: Hosted]

- Virtualbox
- KVM Qemu virt-manager

For correct Virtualization you will need to add user to a few groups:

[Virtualbox]

you need to add user to groups: vboxusers, disk

- sudo usermod -a -G vboxusers \$USER
- sudo usermod -a -G disk \$USER

[kvm qemu virt-manager]

you need to add user to groups: libvirt, libvirt-qemu, kvm

- sudo usermod -a -G libvirt \$USER
- sudo usermod -a -G libvirt-qemu \$USER
- sudo usermod -a -G kvm \$USER

You can add a users to a group auto automatically by editing the **/etc/adduser.conf** file:

uncomment the following lines and add groups you want users to be automatically added to::

```
#EXTRA_GROUPS=" * "  
to:  
EXTRA_GROUPS="vboxusers disk libvirt libvirt-qemu kvm"  
  
#ADD_EXTRA_GROUPS=1  
to:  
ADD_EXTRA_GROUPS=1
```

* to enable kvm qemu virt-manager pcie passthrough you must add
intel_iommu=on
to kernel boot paramaters for intel cpus

VIRTUALBOX RAW DISK ACCESS [For Version 6.1]

2.7. Advanced Storage Configuration

2.7.1. Using a Raw Host Hard Disk From a Guest

As an alternative to using virtual disk images as described in [Virtual Storage](#), Oracle VM VirtualBox can also present either entire physical hard disks or selected partitions as virtual disks to virtual machines.

With Oracle VM VirtualBox, this type of access is called *raw hard disk access*. It enables a guest operating system to access its virtual hard disk without going through the host OS file system. The actual performance difference for image files compared to raw disk varies greatly depending on the overhead of the host file system, whether dynamically growing images are used, and on host OS caching strategies. The caching indirectly also affects other aspects such as failure behavior. For example, whether the virtual disk contains all data written before a host OS crash. Consult your host OS documentation for details on this.



Warning

Raw hard disk access is for expert users only. Incorrect use or use of an outdated configuration can lead to **total loss of data** on the physical disk. Most importantly, *do not* attempt to boot the partition with the currently running host operating system in a guest. This will lead to severe data corruption.

Raw hard disk access, both for entire disks and individual partitions, is implemented as part of the VMDK image format support. As a result, you will need to create a special VMDK image file which defines where the data will be stored. After creating such a special VMDK image, you can use it like a regular virtual disk image. For example, you can use the VirtualBox Manager, see [The Virtual Media Manager](#), or [VBoxManage](#) to assign the image to a virtual machine.

2.7.1.1. Access to Entire Physical Hard Disk

While this variant is the simplest to set up, you must be aware that this will give a guest operating system direct and full access to an *entire physical disk*. If your *host* operating system is also booted from this disk, please take special care to not access the partition from the guest at all. On the positive side, the physical

disk can be repartitioned in arbitrary ways without having to recreate the image file that gives access to the raw disk.

On a Linux host, to create an image that represents an entire physical hard disk which will not contain any actual data, as this will all be stored on the physical disk, use the following command:

```
$ VBoxManage internalcommands createrawvmdk -filename \  
path-to-file.vmdk -rawdisk /dev/sda
```

This creates the `/path-to-file.vmdk` file image that must be an absolute path. All data is read and written from `/dev/sda`.

On a Windows host, instead of the above device specification, for example use `\\.\PhysicalDrive0`. On a Mac OS X host, instead of the above device specification use for example `/dev/disk1`. Note that on Mac OS X you can only get access to an entire disk if no volume is mounted from it.

Creating the image requires read/write access for the given device. Read/write access is also later needed when using the image from a virtual machine. On some host platforms, such as Windows, raw disk access may be restricted and not permitted by the host OS in some situations.

Just like with regular disk images, this does not automatically attach the newly created image to a virtual machine. This can be done as follows:

```
$ VBoxManage storageattach WindowsXP --storagectl "IDE Controller" \  
--port 0 --device 0 --type hdd --medium path-to-file.vmdk
```

When this is done the selected virtual machine will boot from the specified physical disk.

2.7.1.2. Access to Individual Physical Hard Disk Partitions

This *raw partition support* is quite similar to the full hard disk access described above. However, in this case, any partitioning information will be stored inside the VMDK image. This means that you can install a different boot loader in the virtual hard disk without affecting the host's partitioning information. While the guest will be able to see all partitions that exist on the physical disk, access will be filtered in that reading from partitions for which no access is allowed the partitions will only yield zeroes, and all writes to them are ignored.

To create a special image for raw partition support, which will contain a small amount of data, on a Linux host, use the command:

```
$ VBoxManage internalcommands createrawvmdk -filename \  
path-to-file.vmdk -rawdisk /dev/sda -partitions 1,5
```

The command is identical to the one for full hard disk access, except for the additional `-partitions` parameter. This example would create the image `/path-to-file.vmdk`, which must be absolute, and partitions 1 and 5 of `/dev/sda` would be made accessible to the guest.

Oracle VM VirtualBox uses the same partition numbering as your Linux host. As a result, the numbers given in the above example would refer to the first primary partition and the first logical drive in the extended partition, respectively.

On a Windows host, instead of the above device specification, use for example `\\.\PhysicalDrive0`. On a Mac OS X host, instead of the above device specification use `/dev/disk1`, for example. Note that on OS X you can only use partitions which are not mounted. Eject the respective volume first. Partition numbers are the same on Linux, Windows, and Mac OS X hosts.

The numbers for the list of partitions can be taken from the output of the following command:

```
$ VBoxManage internalcommands listpartitions -rawdisk /dev/sda
```

The output lists the partition types and sizes to give the user enough information to identify the partitions necessary for the guest.

Images which give access to individual partitions are specific to a particular host disk setup. You cannot transfer these images to another host. Also, whenever the host partitioning changes, the image *must be recreated*.

Creating the image requires read/write access for the given device. Read/write access is also later needed when using the image from a virtual machine. If this is not feasible, there is a special variant for raw partition access, currently only available on Linux hosts, that avoids having to give the current user access to the entire disk. To set up such an image, use:

```
$ VBoxManage internalcommands createrawvmdk -filename \  
path-to-file.vmdk -rawdisk /dev/sda -partitions 1,5 -relative
```

When used from a virtual machine, the image will then refer not to the entire disk, but only to the individual partitions. In this example, `/dev/sda1` and `/dev/sda5`. As a consequence, read/write access is only required for the affected partitions, not for the entire disk. During creation however, read-only access to the entire disk is required to obtain the partitioning information.

In some configurations it may be necessary to change the MBR code of the created image. For example, to replace the Linux boot loader that is used on the host by another boot loader. This enables for example the guest to boot directly to Windows, while the host boots Linux from the "same" disk. For this purpose the `-mbr` option is provided. It specifies a file name from which to take the MBR code. The partition table is not modified at all, so a MBR file from a system with totally different partitioning can be used. An example of this is:

```
$ VBoxManage internalcommands createrawvmdk -filename \  
path-to-file.vmdk -rawdisk /dev/sda -partitions 1,5 -mbr winxp.mbr
```

The modified MBR will be stored inside the image, not on the host disk.

The created image can be attached to a storage controller in a VM configuration as usual.

VIRTUALBOX RAW DISK ACCESS [For Version 7]

Advanced Storage Configuration

Using a Raw Host Hard Disk From a Guest

As an alternative to using virtual disk images as described in [Virtual Storage](#), Oracle VM VirtualBox can also present either entire physical hard disks or selected partitions as virtual disks to virtual machines.

With Oracle VM VirtualBox, this type of access is called *raw hard disk access*. It enables a guest operating system to access its virtual hard disk without going through the host OS file system. The actual performance difference for image files compared to raw disk varies greatly depending on the overhead of the host file system, whether dynamically growing images are used, and on host OS caching strategies. The caching indirectly also affects other aspects such as failure behavior. For example, whether the virtual disk contains all data written before a host OS crash. Consult your host OS documentation for details on this.

▲ Caution:

Raw hard disk access is for expert users only. Incorrect use or use of an outdated configuration can lead to **total loss of data** on the physical disk. Most importantly, *do not* attempt to boot the partition with the currently running host operating system in a guest. This will lead to severe data corruption.

Raw hard disk access, both for entire disks and individual partitions, is implemented as part of the VMDK image format support. As a result, you will need to create a special VMDK image file which defines where the data will be stored. After creating such a special VMDK image, you can use it like a regular virtual disk image. For example, you can use the Virtual Media Manager, see [The Virtual Media Manager](#), or VBoxManage to assign the image to a virtual machine.

Access to Entire Physical Hard Disk

While this variant is the simplest to set up, you must be aware that this will give a guest operating system direct and full access to an *entire physical disk*. If your *host* operating system is also booted from this disk, please take special care to not access the partition from the guest at all. On the positive side, the physical disk can be repartitioned in arbitrary ways without having to recreate the image file that gives access to the raw disk.

On a Linux host, to create an image that represents an entire physical hard disk which will not contain any actual data, as this will all be stored on the physical disk, use the following command:

```
$ VBoxManage createmedium disk --filename path-to-file.vmdk --format=VMDK
--variant RawDisk --property RawDrive=/dev/sda
```

This creates the *path-to-file.vmdk* file image that must be an absolute path. All data is read and written from */dev/sda*.

On a Windows host, instead of the above device specification, for example use `\\.\PhysicalDrive0`. On a macOS host, instead of the above device specification use for example `/dev/rdisk1`. Note that on Mac OS X you can only get access to an entire disk if no volume is mounted from it.

Creating the image requires read/write access for the given device. Read/write access is also later needed when using the image from a virtual machine. On some host platforms, such as Windows, raw disk access may be restricted and not permitted by the host OS in some situations.

Just like with regular disk images, this does not automatically attach the newly created image to a virtual machine. This can be done as follows:

```
$ VBoxManage storageattach WindowsXP --storagectl "IDE Controller" \
--port 0 --device 0 --type hdd --medium path-to-file.vmdk
```

When this is done the selected virtual machine will boot from the specified physical disk.

Access to Individual Physical Hard Disk Partitions

This *raw partition support* is quite similar to the full hard disk access described above. However, in this case, any partitioning information will be stored inside the VMDK image. This means that you can install a different boot loader in the virtual hard disk without affecting the host's partitioning information. While the guest will be able to see all partitions that exist on the physical disk, access will be filtered in that reading from partitions for which no access is allowed the partitions will only yield zeroes, and all writes to them are ignored.

To create a special image for raw partition support, which will contain a small amount of data, on a Linux host, use the command:

```
$ VBoxManage createmedium disk --filename path-to-file.vmdk --format=VMDK
--variant RawDisk --property RawDrive=/dev/sda --property Partitions=1,5
```

The command is identical to the one for full hard disk access, except for the additional `--property Partitions=1,5` parameter. This example would create the image `path-to-file.vmdk`, which must be absolute, and partitions 1 and 5 of `/dev/sda` would be made accessible to the guest.

Oracle VM VirtualBox uses the same partition numbering as your Linux host. As a result, the numbers given in the above example would refer to the first primary partition and the first logical drive in the extended partition, respectively.

On a Windows host, instead of the above device specification, use for example `\\.\PhysicalDrive0`. On a macOS host, instead of the above device specification use `/dev/rdisk1`, for example. Note that on OS X you can only use partitions which are not mounted. Unmount the respective disk first using `diskutil unmountDisk /dev/diskX`. Partition numbers are the same on Linux, Windows, and macOS hosts.

The numbers for the list of partitions can be taken from the output of the following command:

```
$ VBoxManage list hostdrives
```

The output lists available drives and their partitions with the partition types and sizes to give the user enough information to identify the partitions necessary for the guest.

Images which give access to individual partitions are specific to a particular host disk setup. You cannot transfer these images to another host. Also, whenever the host partitioning changes, the image *must be recreated*.

Creating the image requires read/write access for the given device. Read/write access is also later needed when using the image from a virtual machine. If this is not feasible, there is a special variant for raw partition access, currently only available on Linux hosts, that avoids having to give the current user access to the entire disk. To set up such an image, use:

```
$ VBoxManage createmedium disk --filename path-to-file.vmdk --format=VMDK
--variant RawDisk --property RawDrive=/dev/sda --property Partitions=1,5
--property Relative=1
```

When used from a virtual machine, the image will then refer not to the entire disk, but only to the individual partitions. In this example, `/dev/sda1` and `/dev/sda5`. As a consequence, read/write access is only required for the affected partitions, not for the entire disk. During creation however, read-only access to the entire disk is required to obtain the partitioning information.

In some configurations it may be necessary to change the MBR code of the created image. For example, to replace the Linux boot loader that is used on the host by another boot loader. This enables for example the guest to boot directly to Windows, while the host boots Linux from the "same" disk. For this purpose the `--property-file BootSector=path-to-file-with-boot-sector` parameter is provided. It specifies a file name from which to take the MBR code. The partition table is not modified at all, so a MBR file from a system with totally different partitioning can be used. An example of this is:

```
$ VBoxManage createmedium disk --filename path-to-file.vmdk --format=VMDK
--variant RawDisk --property RawDrive=/dev/sda --property Partitions=1,5
--property-file BootSector=winxp.mbr
```

The modified MBR will be stored inside the image, not on the host disk.

The created image can be attached to a storage controller in a VM configuration as usual.

!!! KVM Virtual guest virtual cores !!! [ENABLE HYPERTHREADING VIRTUAL CORES TO GUEST - KVM]

Relates specifically to Windows VMs.

It seems you can modify most cpu resources without having to worry about Windows reactivation.

Using VMM (virt-manager)...

```
1 -- Alter CPUs > Current allocation as desired.
2 -- Manually alter CPU Topology with Sockets = 1 or 2 (Windows 10 limit *) and Core = Current allocation /
   Sockets, leaving Threads = 1
This did not trigger a Windows reactivation demand for me but your experience may vary so maybe try on a backup
first.
```

Changing the CPU Model did trigger a Windows reactivation demand for me (there may be other stuff too).

```
-----
=====
```

Windows guests do not recognize all of the available cores under KVM and will usually detect only 2 cores. This happens since KVM exposes the available virtual CPU cores as physical CPUs (sockets). So, if the physical host running KVM has 2 CPUs with 4 cores each (a total of 8 cores), and the guest is configured for 8 CPUs, Windows will see 8 physical CPUs and will run only on 2, due to the hard coded limits in some of the editions of Windows Server.

To make Windows use all available cores, we need to configure the guest to expose the CPUs as cores and not as physical CPUs (sockets):

In virt-manager:

```
1-- Open the guest configuration screen
2-- Select Processor options tab
3-- Expand the "Topology" setting
4-- Set the sockets to 2
5-- Set the cores to 4 (for the guest to have a total of 8 cores) or 3 (for the guest to have a total of 6 cores).
6-- You can also expand the "Configuration" settings and click on "copy host CPU configuration" to make the guest
   fully use all of the physical host's CPU capabilities.
```

If you run the guest from command line, the KVM option for setting the CPU topology is: `-smp 8,socket=2,cores=4`

KVM/QEMU CPU Topology Optimization (Virt-Manager) - Hyperthreading in KVM/QEMU

By default, KVM passes each vCPU as a separate physical socket. To optimize performance and bypass OS limits, manually align topology with host hardware.

General Rules:

- Model: Set to host-passthrough for native instruction set support.
- Sockets: Set to 1 (prevents Windows' 2-socket limit & scheduling lag).
- Cores: Match the number of physical cores you wish to assign.
- Threads: Set to 2 (if host has Hyper-Threading) or 1 (if not).
- Pass through all 4 threads as: 1 Socket, 2 Cores, 2 Threads.

Example (4 Cores, 8 Threads):

- Pass through all 8 threads as: 1 Socket, 4 Cores, 2 Threads.
- This maintains cache locality and ensures Windows Pro/Home sees all 8 logical processors.

PCI - passthrough with KVM/libvirt qemu/ virtmanager

A PCI passthrough tutorial using libvirt and KVM on GNU/Linux

Introduction

This tutorial covers some of the nuances involved in setting up PCI passthrough with libvirt and KVM / QEMU. The documentation contained herein is primarily sourced from personal experience and research into the topic.

Since I don't have any compatible AMD®-based hardware for testing, I can only provide in-depth support for Intel®-based builds. However, the steps outlined in this publication should very accurately reflect the workflow required for AMD®-based systems.

So if you're having difficulties passing through some pcie devices to a kvm virtual machine, this guide will help you

Preamble

The goal of this tutorial is to set up a KVM hypervisor environment capable of PCI passthrough. This tutorial assumes that the would-be hypervisor's operating system and libvirt/KVM have already been installed.

Also of great importance to this tutorial is to provide the reader (you) with ample information to understand the changes that are made and *why* they need to be made.

Several quirks will present themselves during configuration and each one must be overcome in order to complete all of our stated goals:

Hardware Compatibility Criteria

Below are some things to keep in mind while planning your build. At a minimum, your hardware must meet all criteria with bold text. Other criteria are just recommendations to make the configuration process easier.

- If you're using an AMD®-based system, your motherboard and CPU **must** support [AMD® Virtualization Technology \(AMD-V™\)](#) and [AMD® I/O Virtualization Technology \(IOMMU\)](#) (also known as "AMD-Vi").
- If you're using an Intel®-based system, your motherboard and CPU **must** support [Intel® Virtualization Technology \(VT-x\)](#) and [Intel® Virtualization Technology for Directed I/O \(VT-d\)](#) .

Preparing the Hypervisor

Step #1

The first step to facilitating PCI-e passthrough of any kind is enabling virtualization acceleration and IOMMU in your motherboard's BIOS settings. Consult your motherboard manual to find options relating to AMD-V™, AMD-Vi, VT-d, VT-x or anything else with virtualization-related keywords and enable those options.

Next, we need to enable support for IOMMU in the Linux kernel. Assuming that you use GRUB as your boot loader, we need to append one of the following strings to the `GRUB_CMDLINE_LINUX_DEFAULT` variable in `/etc/default/grub`, making sure that there is a space to separate it from what's already there:

- If using an AMD®-based system, there's no need to add kernel boot parameter [if iommu enabled on bios in amd pc]
- If using an Intel®-based system, append `intel_iommu=on` to the kernel argument list.

Once you've modified the variable, it should look similar to this:

```
GRUB_CMDLINE_LINUX_DEFAULT="intel_iommu=on"
```

Note: If your device(s) don't support interrupt remapping, you might be able to quell any issues with PCI-e passthrough by adding `vfio_iommu_type1.allow_unsafe_interrupts=1` to the argument list as well. There are some security concerns that you should research first before enabling unsafe interrupts. [\[Red Hat\]](#)

Finally, run the following command to update your boot loader configuration:

```
$ update-grub
```

```
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-x.xx.x-xx-generic
Found initrd image: /boot/initrd.img-x.xx.x-xx-generic
done
```

Once your boot loader configuration is updated, reboot your system. To verify that your operating system is configured correctly for PCI-e passthrough, run the following commands and make sure that your output is similar:

```
$ apt-get install -y cpu-checker
```

```
$ kvm-ok
```

```
INFO: /dev/kvm exists
KVM acceleration can be used
```

```
$ dmesg | egrep 'DMAR|IOMMU'
```

```
[ 0.000000] ACPI: DMAR 0x00000000BD6483E0 0000B8 (v01 INTEL BDW 00000001 INTL 000
[ 0.000000] DMAR: IOMMU enabled
[ 0.000000] DMAR: Host address width 39
```

```
[ 0.000000] DMAR: DRHD base: 0x000000fed90000 flags: 0x0
[ 0.000000] DMAR: dmar0: reg_base_addr fed90000 ver 1:0 cap c0000020660462 ecap f0101a
[ 0.000000] DMAR: DRHD base: 0x000000fed91000 flags: 0x1
[ 0.000000] DMAR: dmar1: reg_base_addr fed91000 ver 1:0 cap d2008c20660462 ecap f010da
[ 0.000000] DMAR: RMRR base: 0x000000bdea5000 end: 0x000000bdeb3fff
[ 0.000000] DMAR: RMRR base: 0x000000bf000000 end: 0x000000cflffffff
[ 0.000000] DMAR-IR: IOAPIC id 8 under DRHD base 0xfed91000 IOMMU 1
[ 0.000000] DMAR-IR: HPET id 0 under DRHD base 0xfed91000
[ 0.000000] DMAR-IR: x2apic is disabled because BIOS sets x2apic opt out bit.
[ 0.000000] DMAR-IR: Use 'intremap=no_x2apic_optout' to override the BIOS setting.
[ 0.000000] DMAR-IR: Enabled IRQ remapping in xapic mode
[ 0.700747] DMAR: No ATSR found
[ 0.700776] DMAR: dmar0: Using Queued invalidation
[ 0.700782] DMAR: dmar1: Using Queued invalidation
[ 0.700828] DMAR: Setting RMRR:
[ 0.700883] DMAR: Setting identity map for device 0000:00:02.0 [0xbf000000 - 0xcflffffff
[ 0.702040] DMAR: Setting identity map for device 0000:00:14.0 [0xbdea5000 - 0xbdeb3fff
[ 0.702083] DMAR: Setting identity map for device 0000:00:1a.0 [0xbdea5000 - 0xbdeb3fff
[ 0.702128] DMAR: Setting identity map for device 0000:00:1d.0 [0xbdea5000 - 0xbdeb3fff
[ 0.702141] DMAR: Prepare 0-16MiB unity mapping for LPC
[ 0.702174] DMAR: Setting identity map for device 0000:00:1f.0 [0x0 - 0xffffffff]
[ 0.702290] DMAR: Intel(R) Virtualization Technology for Directed I/O
[ 1.146736] [drm] DMAR active, disabling use of stolen memory
```

Note: you may get more or less lines here; just look for hints that IOMMU is actually enabled.

Step #2

Load the necessary modules for pcie passthrough in kvm/libvirt qemu/virtmanager at boot up

(or manually, but at boot up/automatically is preferred, obviously)

We can achieve this in a few ways...

Method 1: editing, or adding modules name to `/etc/modules-load.d/modules.conf`

Method 2: using systemd: this is called a systemd service file /or systemd unit file /or systemd service unit file

Method 1:

1. Add components necessary to load VFIO (Virtual Function I/O). VFIO is required to pass full devices through to a virtual machine, so that Ubuntu loads everything it needs. Edit and add the following modules names to **`/etc/modules-load.d/modules.conf`** file:

```
vfio
vfio_pci
vfio_iommu_type1
kvm
kvm_intel
pci_stub
```

Method 2: using systemd; create a systemd service file to load necessary modules at boot up

create a systemd service file and make it load the necessary modules at boot up:

```
sudo modprobe vfio
sudo modprobe vfio_pci
sudo modprobe vfio_iommu_type1
sudo modprobe kvm
sudo modprobe kvm_intel
sudo modprobe pci_stub
```

If everything is configured correctly, the virtual machine should now be able to boot and use the attached PCIe device without issue. All that remains now is the installation of the appropriate device drivers if necessary.

Glossary

passthrough devices—A set of devices which are the target of PCI-e passthrough and each neighboring device within their respective IOMMU groups. This set should exclude any “PCI bridge” devices which are part of the CPU.

KVM qemu virt-manager Storage passthrough Documentation

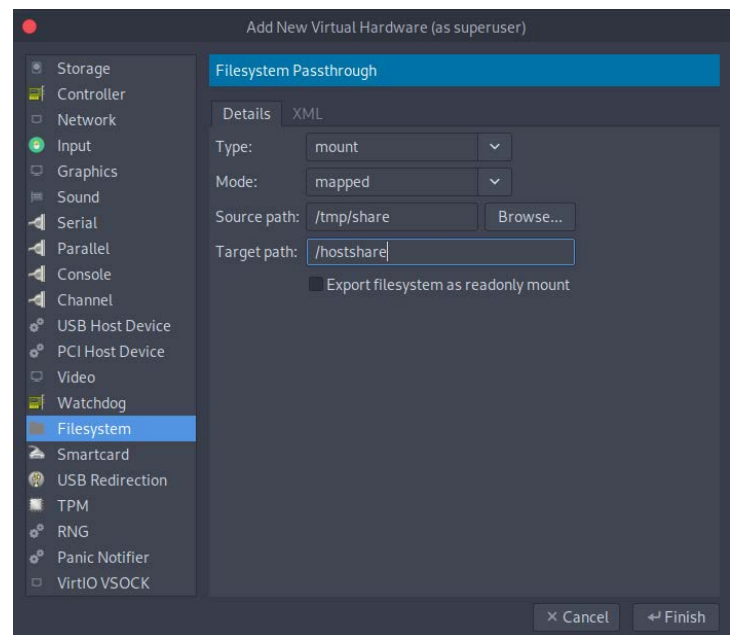
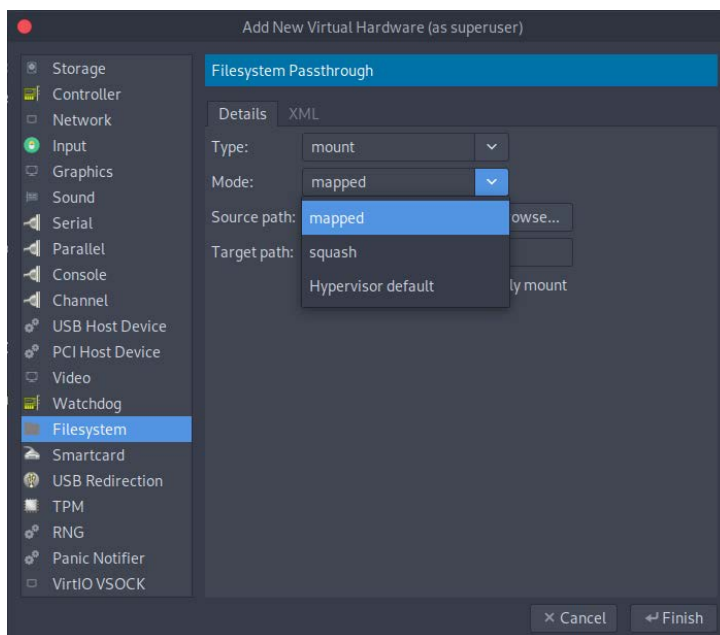
#1 Share Host Directories/Files with the Guest (Linux) filesystem passthrough

Filesystem Passthrough, Managed (Graphically), using virt-manager, for linux Guests

This assumes you've already installed a guest with virt-manager or virt-install and it's shut off. Start virt-manager, and open your VM by double clicking on it. Click the virtual hardware details.

Click add Hardware, and choose Filesystem.

Chose the mode, we chose mapped (see QEMU wiki for details on the security models), the source (we are sharing /tmp/share), and the target (we called this /hostshare).



Now, start the VM. **In the guest** we need to mount the 9p filesystem from the host using the virtio transport. The mount_tag is used to identify the host's share.

mkdir /tmp/host_files (create-directory-of-mount-point-for-shared-folder-on-guest) if not already exists

mount -t 9p -o trans=virtio (target path; shared-external-folder ex: /hostshare) (mount point-on-guest ex: /tmp/host_files)

That's it...now we can read/write files in that directory from either the host or the guest. (Note: likely to hit some issues w/ privileges since libvirt runs guests unprivileged. so execute as sudo.

With QEMU's 9pfs you can create virtual filesystem devices (virtio-9p-device) and expose them to guests, which essentially means that a certain directory on host machine is made directly accessible by a guest OS as a pass-through file system by using the 9P network protocol for communication between host and guest, if desired even accessible, shared by several guests simultaneously.

You can mount the shared folder using: (Within the Guest)

```
mount -t 9p -o trans=virtio [mount tag] [mount point]
```

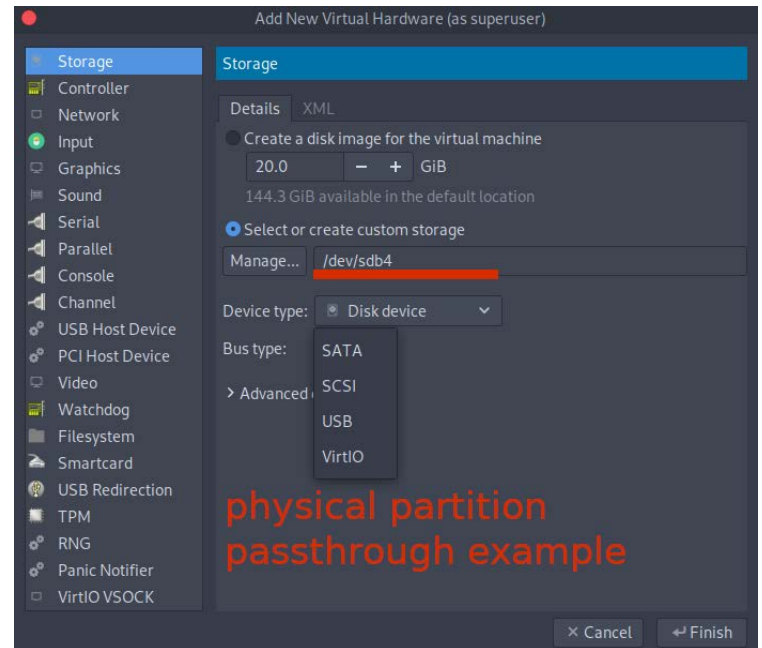
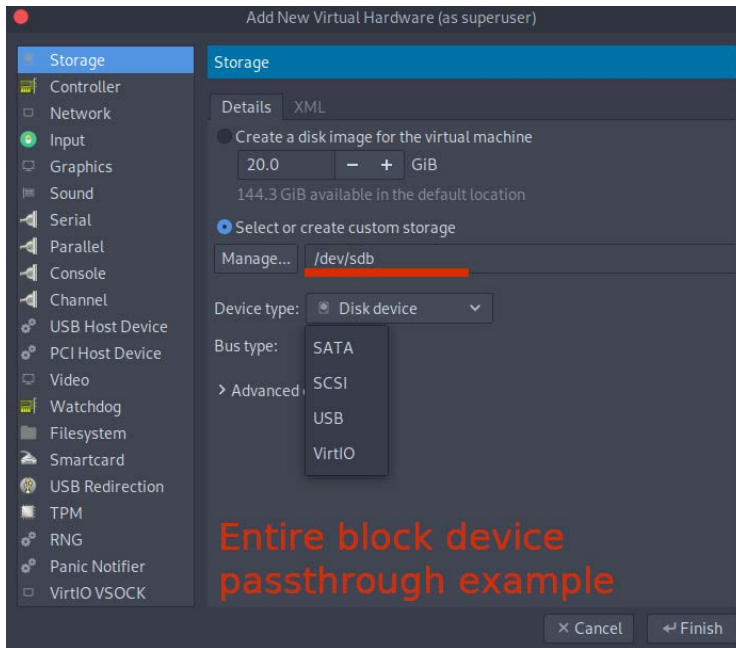
```
$ sudo mount -t 9p -o trans=virtio (target-path-on-GUEST-ex: /mount-test) (desired-mount-point-on-GUEST-ex: /mount-external)
$ sudo mount -t 9p -o trans=virtio /source-on-guest /target-on-guest
```

***** target path must be in the root / of the guest os**

#2 ADDING HARD DRIVES AND OTHER BLOCK DEVICES TO A GUEST (Passthrough Entire Physical-Partitions & Physical-Drives)

Adding Physical Block Devices to Linux Guests:

you can passthrough physical partitions or entire physical drives to a linux guest .



The devices or partitions can be assigned in different ways:

- `/dev/disk/by-uuid/<uuid of device>`
- `/dev/disk/by-partuuid/<uuid of partition>`
- `/dev/disk/by-label/<label of device>`
- `/dev/disk/by-id/<id of partition>`
- `/dev/disk/by-path/<path of partition>`

* you can also assign the devices/ partitions parameters manually by editing the xml file

Passthrough Entire block devices to a Windows Guest

- You must **install virtio drivers for windows!** :Highly recommended to download virtio-win iso from fedora
- you can only passthrough entire block devices to windows guests (no partitions, only entire devices)

*****note: it is perfectly fine to passthrough an entire device to a guest that is running on that same block device

How to mount qcow2/vdi disk image on Linux

Last updated on November 14, 2020 by Dan Nanni

Question: I have a `qcow2`-type disk image which is used by one my guest VMs on QEMU/KVM. I want to modify the content of the disk image without powering on the VM, and for this I need to mount the disk image somewhere. Is there a way to mount a `qcow2` disk image on Linux?

When you run a guest virtual machine (VM) on a hypervisor, you create one or more disk images dedicated to the VM. As a *virtual* disk volume, a disk image represents the content and structure of a storage device (e.g., a hard disk drive or flash drive) attached to the VM. If you want to modify files in a VM's disk image without powering on the VM, you could *mount* the disk image. You then would be able to modify the content of the disk image before unmounting it.

In Linux, there are ways to mount a disk image, and different types of disk images require different approaches. If you are using `qcow2`-type disk images (used by QEMU/KVM), there are at least two ways to mount them on Linux.

Method One: `qemu-nbd`

Another method to mount a `qcow2` disk image is via `qemu-nbd`, a command-line tool that exports a disk image as a network block device (`nbd`).

You can install `qemu-nbd` on Linux as follows.

On Debian-based system:

```
xterm
$ sudo apt-get install qemu-utils
```

On Red Hat-based system:

```
xterm
$ sudo yum install qemu-img
```

To mount a `qcow2` disk image, first export the image to `nbd` as follows.

```
xterm
$ sudo modprobe nbd max_part=64
$ sudo qemu-nbd --connect=/dev/nbd0 /path/to/qcow2/image
```

The first command loads the `nbd` kernel module. The `max_part=N` option specifies the maximum number of partitions we want to manage with `nbd`. The second command exports the specified disk image as a network block device (`/dev/nbd0`). As a network block device, you can use `/dev/nbd0`, `/dev/nbd1`, `/dev/nbd2`, etc. whichever is unused. As for the disk image, make sure to specify its *full* path.

For example, to export the image `xenserver.qcow2` as `/dev/nbd0`:

```
xterm
$ sudo qemu-nbd --connect=/dev/nbd0 /var/lib/libvirt/images/xenserver.qcow2
```

After this, existing disk partitions in the disk image will be mapped to `/dev/nbd0p1`, `/dev/nbd0p2`, `/dev/nbd0p3`, etc.

To check the list of `nbd`-mapped partitions, use `fdisk`:

```
xterm
$ sudo fdisk /dev/nbd0 -l
```

Finally, choose any one partition (e.g., `/dev/nbd0p1`) and mount it to a local mount point (e.g., `/mnt`).

```
xterm
$ sudo mount /dev/nbd0p1 /mnt
```

Now you will be able to access and modify the content of the disk image's mounted partition via `/mnt` mount point.

Once you are done, unmount it, and disconnect the disk image as follows.

xterm

```
$ sudo umount /mnt
```

```
$ sudo qemu-nbd --disconnect /dev/nbd0
```

```
dan@poweredge:~$
```

Export disk image to network block device

```
dan@poweredge:~$
```

```
dan@poweredge:~$ sudo qemu-nbd --connect=/dev/nbd0 /var/lib/libvirt/images/xenserver.qcow2
```

```
dan@poweredge:~$
```

```
dan@poweredge:~$ sudo fdisk /dev/nbd0 -l
```

```
Disk /dev/nbd0: 12 GiB, 12884901888 bytes, 25165824 sectors
```

```
Units: sectors of 1 * 512 = 512 bytes
```

```
Sector size (logical/physical): 512 bytes / 512 bytes
```

```
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

```
Disklabel type: gpt
```

```
Disk identifier: 518988BD-A6C5-4C66-AFAB-825D2D7F5244
```

Available devices in the disk image

Device	Start	End	Sectors	Size	Type
/dev/nbd0p1	2048	8388641	8386594	4G	Microsoft basic data
/dev/nbd0p2	8390656	16777249	8386594	4G	Microsoft basic data
/dev/nbd0p3	16779264	25165790	8386527	4G	Linux LVM

```
dan@poweredge:~$
```

```
dan@poweredge:~$ sudo mount /dev/nbd0p1 /mnt
```

Mount one device in the disk image

```
dan@poweredge:~$
```

```
dan@poweredge:~$
```

```
dan@poweredge:~$ ls /mnt
```

```
bin      dev      home     lost+found  opt          root        srv        tmp
boot     etc      lib      media      proc         sbin        sys        usr
cli-rt   EULA    lib64    mnt        Read_Me_First.html  selinux     tftpboot   var
```

```
dan@poweredge:~$
```

```
dan@poweredge:~$
```

```
dan@poweredge:~$ sudo umount /mnt
```

Unmount the device

```
dan@poweredge:~$
```

```
dan@poweredge:~$
```

Converting between virtual image formats

Converting images from one format to another is generally straightforward.

qemu-img convert: raw, qcow2, qed, vdi, vmdk, vhd

The **qemu-img convert** command can do conversion between multiple formats, including **qcow2**, **qed**, **raw**, **vdi**, **vhd**, and **vmdk**.

qemu-img format strings

Image format	Argument to qemu-img
QCOW2 (KVM, Xen)	qcow2
QED (KVM)	qed
raw (.img)	raw
VDI (VirtualBox)	vdi
VHD (Hyper-V)	vpc
VMDK (VMware)	vmdk

This example will convert a raw image file named **image.img** to a **qcow2** image file:

```
$ qemu-img convert -p -f raw -O qcow2 image.img image.qcow2
```

Run the following command to convert a **vmdk** image file to a **raw** image file:

```
$ qemu-img convert -p -f vmdk -O raw image.vmdk image.img
```

Run the following command to convert a **vmdk** image file to a **qcow2** image file:

```
$ qemu-img convert -p -f vmdk -O qcow2 image.vmdk image.qcow2
```

- **-p** indicates the image conversion progress.
- **-f** indicates the source image format.
- The part following **-O** (which must be in upper case) consists of the required format, source image file, and target image file.

Note

The **-f format** flag is optional. If omitted, **qemu-img** will try to infer the image format.

When converting an image file with Windows, ensure the virtio driver is installed. Otherwise, you will get a blue screen when launching the image due to lack of the virtio driver. Another option is to set the image properties as below when you update the image in the Image service to avoid this issue, but it will reduce virtual machine performance significantly.

```
$ openstack image set --property hw_disk_bus='ide' image_name_or_id
```

VBoxManage: VDI (VirtualBox) to raw

If you've created a VDI image using VirtualBox, you can convert it to raw format using the **VBoxManage** command-line tool that ships with VirtualBox. On Mac OS X, and Linux, VirtualBox stores images by default in the **~/VirtualBox VMs/** directory. The following example creates a raw image in the current directory from a VirtualBox VDI image.

```
$ VBoxManage clonehd ~/VirtualBox \ VMs/image.vdi image.img --format raw
```

Sparse Files

Sparse file

According to [Wikipedia](#), in computer science, a sparse file is a type of computer file that attempts to use file system space more efficiently when blocks allocated to the file are mostly empty. This is achieved by writing brief information (metadata) representing the empty blocks to disk instead of the actual "empty" space which makes up the block, using less disk space. The full block size is written to disk as the actual size only when the block contains "real" (non-empty) data.

When reading sparse files, the file system transparently converts metadata representing empty blocks into "real" blocks filled with zero bytes at runtime. The application is unaware of this conversion.

Most modern file systems support sparse files, including most Unix variants and NTFS, but notably not Apple's HFS+. Sparse files are commonly used for [disk images](#) (not to be confused with [sparse images](#)), database snapshots, log files and in scientific applications.

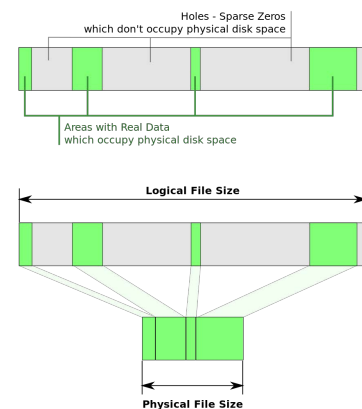
The advantage of sparse files is that storage is only allocated when actually needed: disk space is saved, and large files can be created even if there is insufficient free space on the file system.

Disadvantages are that sparse files may become fragmented; file system free space reports may be misleading; filling up file systems containing sparse files can have unexpected effects; and copying a sparse file with a program that does not explicitly support them may copy the entire file, including the empty blocks which are not on explicitly stored on the disk, which wastes the benefits of the sparse property of the file.

Contents

- 1 [Creating sparse files](#)
 - 1.1 [Making existing files sparse](#)
 - 1.2 [Formatting the file with a filesystem](#)
 - 1.3 [Mounting the file at boot](#)
- 2 [Copying the sparse file](#)
 - 2.1 [Copying with cp](#)
 - 2.2 [Archiving with tar](#)
- 3 [Resizing the sparse file](#)
 - 3.1 [Growing the file](#)
- 4 [Verify the sparse files integrity](#)

The following image illustrates the structure of a sparse file:



Creating sparse files

The `truncate` utility can create sparse files. This command creates a 1GB sparse file:

```
$ truncate -s 1G file.img
```

The `dd` utility can also be used, for example:

```
$ dd if=/dev/zero of=file.img bs=1 count=0 seek=1G
```

Sparse files have different *apparent file sizes* (the maximum size to which they may expand) and *actual file sizes* (how much space is allocated for data on disk). To check the file's *apparent size*, just run:

```
$ du -h --apparent-size file.img
1.0G    file.img
```

and, to check the *actual size* of the file on disk:

```
$ du -h file.img
0        file.img
```

As you can see, although the apparent size of the file is 512 MiB, its "actual" size is really zero—that's because due to the nature and beauty of sparse files, it will "expand" arbitrarily to minimize the space required to store its contents.

Making existing files sparse

(three methods: `fallocate -d`, `cp --sparse=always`, `dd conv=sparse`)

#make an existing file sparse using `fallocate` command (with `-d` option)

```
$ fallocate -d copy.img
```

#make an existing file sparse using the `cp` command (`--sparse=always` option)

```
$ cp <file.img> <copy.img> --sparse=always
```

#make an existing file sparse using `dd` command (`conv=sparse` option); (or image a storage/block device into a sparse image)

```
$ dd if=<input_file> of=<output_file> conv=sparse
```

Formatting the file with a filesystem:

Now that we have created the sparse file, it's time to format it with a filesystem; for example ReiserFS:

```
$ mkfs.reiserfs -f -q file.img
```

We can now check its size to see how the filesystem has affected it:

```
$ du -h --apparent-size file.img
```

```
512M    file.img
```

```
$ du -h file.img
```

```
33M     file.img
```

As you may have expected, formatting it with the filesystem has increased its actual size, but left its apparent size the same. Now we can create a directory which we will use to mount our file:

```
# mkdir mountpoint
# mount -o loop file.img mountpoint
```

you may also use **losetup** to mount a sparse file: **losetup -fP <file>**

Tada! We now have both a file and a folder into which we may store *almost* 512 MiB worth of information!

Mounting the file at boot

To mount the sparse image automatically at boot, add an entry to your **fstab**:

```
/path/to/file.img /path/to/mountpoint reiserfs loop,defaults 0 0
```

Note: Be sure to include the `loop` option, otherwise it will not mount.

Copying the sparse file

Copying with cp

Normally, **cp** is good at detecting whether a file is sparse, so it suffices to run:

```
$ cp file.img new_file.img
```

Then `new_file.img` will be sparse. However, `cp` does have a `--sparse=when` option. This is especially useful if a sparse file has somehow become non sparse (i.e. the empty blocks have been written out to disk in full). Disk space can be recovered by:

```
$ cp --sparse=always new_file.img recovered_file.img
```

Archiving with tar

One day, you may decide to back up your well-loved sparse file, and choose the **tar** utility for that very purpose; however, you soon realize you have a problem:

```
$ du -h file.img
```

```
33M     file.img
```

```
$ tar -cf file.tar file.img
```

```
$ du -h file.tar
```

```
513M    file.tar
```

Apparently, even though the current size of the sparse file is only 33 MB, archiving it with tar created an archive of the **ENTIRE SIZE OF THE FILE!** Luckily for you, though, tar has a `--sparse` (`-S`) flag, that when used in conjunction with the `--create` (`-c`) operation, tests all files for sparseness while archiving. If tar finds a file to be sparse, it uses a sparse representation of the file in the archive. This is useful when archiving files, such as dbm files, likely to contain many nulls, and dramatically decreases the amount of space needed to store such an archive.

```
$ tar -Scf file.tar file.img
```

```
$ du -h file.tar
```

```
12K     file.tar
```

Problem solved.

pipe disk image file from .tar archive to dd:

tar xOf ChromeOS-Flow.tar | dd of=/dev/sdxx bs=1M

Allow me to take you through the parameters for tar one at a time.

- **x** **extract**
- **O** **send the output through to stdout**
- **f** **parameter for the file we're working with (as opposed to a stream)**

The magic is in the O character which allows you to “|” pipe the output from tar directly to dd which then writes the stream straight to the block device (in this case /dev/sdb).

Resizing the sparse file

****You can only grow sparse files;
If you attempt to shrink a sparse file you will lose all data in the sparse file**

Growing the file

Should you ever need to grow the file, you may do the following:

```
# umount folder

# dd if=/dev/zero of=file.img bs=1 count=0 seek=1G
0+0 records in
0+0 records out
0 bytes (0 B) copied, 2.2978e-05 s, 0.0 kB/s
```

This will increase its size to 1 Gb, and leave its information intact. Next, we need to increase the size of its filesystem:

```
# resize_reiserfs file.img
resize_reiserfs 3.6.21 (2009 www.namesys.com)

ReiserFS report:
blocksize          4096
block count        262144 (131072)
free blocks        253925 (122857)
bitmap block count  8 (4)

Syncing..done

resize_reiserfs: Resizing finished successfully.
```

...and, remount it:

```
# mount -o loop file.img folder
```

Checking its size gives us:

```
# du -h --apparent-size file.img
1.0G    file.img

# du -h file.img
33M     file.img
```

...and to check for consistency:

```
# df -h folder
Filesystem      Size  Used Avail Use% Mounted on
/tmp/file.img   1.0G   33M   992M   4% /tmp/folder
```

```
# ls folder
file1 file2 file3 file4 file5

# cat folder/file1
This is a test to see if it works...
```

Verify integrity of sparse files; check sparse file for consistency against original file (md5sum).

It is common practice to calculate the checksums for files to check its integrity. (optional, but recommended for maximum data safety). Just to be careful we md5sum both the block device/or original file and the sparse file to make sure they have the same sums:

```
# check sparse files integrity against the sparse files source (md5sum source and sparse)

$ md5sum <source_file>

$ md5sum <sparse_file>
```

*this is referred to as checksumming, there are other algorithms available besides md5sum (like sha-1, sha-256, crc32) but we are using md5 here.

commands to the previously mentioned checksumming algorithms:

- sha1sum
- sha256sum
- crc32
- md5sum

**you can also checksum with GUI applications like gtkhash

How to mount and read/write raw images [LOSETUP] (.img .iso ..etc) and other image file types on Linux

How to mount and read/write raw images (.img .iso ..etc) and other image file types on Linux

Mounting Image Files in Linux:

We can easily mount and read/write image files (.img , .iso ..etc) and also images of disks with partitions. We can mount the image by using a loopback device;

a **loopback device** is software pseudo-device that acts as a middleman and allows the system to treat the file as though it were a block device such as a hard drive, etc. In doing so, we have much more flexibility when dealing with the file such as performing filesystem functions (which assume there is an underlying block device).

If you have a few image files (.img .iso ...etc) coming as disk images from devices like Hard drives, operating systems, SD cards, USB drives ..etc, and you try to just mount them with write/read capabilities you will realize that you cannot, That's because most disks have partition tables. CDs and DVDs don't typically have partition tables because it doesn't suit their use-cases. When mount starts to mount a volume, it expects to encounter a filesystem, not a partition table. That doesn't mean that it can't be done, though.

however you need to know the correct syntax that is based on understanding the information related to the partition(s) available in the image.

In order to mount the filesystem using mount, we have to tell mount where it is within the image file with a piece of information passed into the mount options. The piece of information that mount requires is the offset of the partition, that is, the number of bytes into the image file where the partition starts. There are various ways of figuring that out, but **fdisk -l <image file.img>** does the work of obtaining the needed information for you.

* **losetup -f -P < image file.img>** is by far the preferred method
or **losetup -fP <image file.img>**

losetup -D : Detach all associated loop devices /destroy/unmount loop device no longer in use

****NOTE:** loop devices are located in /dev like other block devices
example: /dev/loopX

OVERVIEW: Tools used: quick overview

- **fdisk -l <image.img>** → used here to obtain information about the images start sector and sector size
- **mount -o rw,loop,offset=<start-sector * sector size> <image-file.img> <mount-point-for-image-file>**
- or .. **mount -t vfat -o loop,rw,offset=\$((start-sector * sector size)) imgfile.img /mnt/disk**
→ using mount allows to mount only one partition at a time from a specific image file, so losetup may be preferred
- **losetup -f -P <image-file.img>** → used to automate the detection of the partitions and offsets and mountpoint of the image file (this is the preferred way)
- **losetup -d </dev/loop#>** → detach the specified loop device & the file or device associated with the specified loop device(s).
- **losetup -D** → used to Detach-all unused loopback device when the images mount point is unmounted (when loopback device was created using mount, it autoclears the loopback device on unmount, unlike with losetup, so losetup -D is needed to clear/Destroy the now unused loopback device)

**see man losetup or losetup --help for more information

*you can mount in read only mode using losetup by using -r option

Using losetup to mount an image file with read/write, quick overview:

```
# setup loopback device and autodetect partitions of image file using  
losetup
```

```
or      sudo losetup -fP /location/of/imgfile.img sudo  
        losetup -f -P /location/of/imgfile.img
```

```
#mount any other of the multiple partitions (if multiple partitions) of  
the previously created loopback device
```

```
mount /dev/loop0p1 /path/to/mount/dir
```

```
#Unmount loopback devices
```

```
umount /loop-device/mount-point
```

```
# destroy/detach/autoclear loopdevice if necessary after unmounting
```

```
        losetup -d </dev/loop#>  
or      losetup -D
```

**FOLLOWING IS AUXILIARY DOCUMENTATION THAT MAY BE USEFULL, ALTHOUGH
MOSTLY ALL OF WHAT IS NEEDED HAS ALREADY BEEN COVERED: ↓↓↓**

How To Mount ISOs And Other Image File Types In Linux?

Have you ever needed to check the contents of an image file? Maybe you forgot which version of that Linux distribution that was. There can be many reasons for double-checking an image, but how do you do that without burning it? Let's find out, no extra software required.

Every tech savvy Linux user should be familiar with the *mount* command. It's a very important part of general system administration. But, many don't know that the *mount* command is actually much more powerful than apparent at first glance.

Mounting Image files in Linux

We can quickly and easily mount, not just ISO images, but even images of disks with partitions.

Quickly, here is an example of mounting an ISO file.

```
mount -o loop disk_image.iso /path/to/mount/dir
```

Here the `-o` flag is for passing mount options, and in this case, we're telling *mount* that we want to mount the image using a **loopback device**. A loopback device is a really nice concept developed for assisting in mounting image files. A loopback device is software pseudo-device that acts as a middleman and allows the system to treat the file as though it were a block

device such as a CDROM drive, hard drive, etc. In doing so, we have much more flexibility when dealing with the file such as performing filesystem functions (*which assume there is an underlying block device*).

That's pretty cool, but who even uses ISOs anymore, right? Well, *mount* can do quite a bit more, too. If you have an image file of a hard drive (*as obtained from dd or something similar*), let's call it `hdd.img`, then you can actually mount the filesystems within it as well. And if you're suspecting that we need to use a loopback device again, you're on the right track.

But if you try this...

```
mount -o loop hdd.img /path/to/mount/dir
```

you're not going to have much luck.

That's because most disks have [partition tables](#). CDs and DVDs don't typically have partitions tables because it doesn't suit their use-cases. When *mount* starts to mount a volume, it expects to encounter a filesystem, not a partition table.

That doesn't mean that it can't be done, though. In order to mount the filesystem, we have to tell *mount* where it is within the image file with a piece of information passed into the mount options. The piece of information that *mount* requires is the offset of the partition, that is, the number of bytes into the image file where the partition **starts**. There are various ways of figuring that out, but *fdisk* does the work for you.

```
fdisk -l hdd.img
```

Suppose, *fdisk* tells us that the starting sector is 100 (*this is purely an example, it is highly improbable that this is correct in your specific case*).

What we have to do now is take that sector number and multiply it by the sector size of the HDD that the image was taken from, this will (*most likely*) be 512 bytes on older drives and possibly 4096 (*4K*) on newer ones, but you should always check this when cloning drives (*maybe put it into the file name for good measure*). The sector size is the number of bytes per sector.

We'll go with a sector size of 512 bytes for simplicity, and that would give us an offset of 51,200. This offset is the number of bytes into the image where the filesystem starts. Knowing this, we can tell *mount* exactly where to find the filesystem.

```
mount -o ro,loop,offset=51200 hdd.img /path/to/mount/dir
```

NOTE: I've added the *ro* option into this example as well to show how you can mount a filesystem as read-only. This is especially useful in forensic applications where you cannot disturb the data integrity for security, or even legal, concerns. Often, an image file will be created from the disk using a special adapter called a write-blocker that literally blocks all write commands and only allows for read commands.

That's quite a lot of work with the math and all, but there must be an easier way, right? That depends on how recent of a Linux distribution you're using, but anything from the last five years or so shouldn't have any problems.

The tool *losetup* is for setting up loopback devices. You might wonder why you want to set up a loopback device manually when *mount* takes care of all that for you. Remember how *mount* didn't want to mount the whole disk image because of that annoying partition table? Well, given that the partition table is what contains all the math about where partitions are located (*where fdisk gets all it's information*), it'd be a lot easier to let the system reference it instead. We can simply create a loopback device for the whole disk image and then tell the system to scan for partitions on the newly created loopback device.

```
losetup -f hdd.img
```

NOTE: The *-f* flag passed to *losetup* is simply telling it to create a loopback device with a name not yet used, such as */dev/loop1* if */dev/loop0* is *already in use*. Alternatively, you can pass a device name such as */dev/loop0*, if that device does not exist, *losetup* will create it, if it does, it will reassign it to the newly appointed image file.

If the command is successful, you will see the newly created loopback device under */dev/*. Now, you can use

```
partprobe /dev/loopback0
```

or

```
kpartx -u /dev/loopback0
```

if you have *partprobe* or *kpartx* installed, respectively. Though, *parted* should be installed on most modern Linux distributions.

Once this is done, you should see something like */dev/loop0p1* in */dev/*. This is your partition, which you can then easily mount.

```
mount /dev/loop0p1 /path/to/mount/dir
```

As you'll probably notice, no loopback required. That's because we've already taken care of that.

We can make this even easier, though, by adding a single flag to *losetup*.

```
losetup -f -P hdd.img
```

The *-P* flag we used actually tells *losetup* to have the kernel scan the partition table, so we can skip along to mounting the partition we want.

Using the above, you can mount a variety of disk images. It doesn't matter if they're from optical media, USB keys, MBR partitioning or GPT. The Linux kernel can handle so many different partition tables and filesystems, you'll have a harder time finding something that it doesn't support. Sure, there might be tools that make this much easier, but where's the fun in that? I've used these techniques in data recovery and general system administration and like any type of knowledge, it is extremely valuable when needed.

Tutorial: How to mount raw images (.img) images on Linux

 Author: Stefano Prenna

If you have a few .img files coming as [disk images](#) from devices like floppies, CDs, DVDs, SD cards, etc, you will realize that you cannot mount the in Linux, because they contain a file system that has to be mounted.

In linux you would need to use the mount command as for any physical device, however you need to know the correct syntax that is based on understanding the information related to the partition(s) available in the image.

First step is to read the partition Start point using fdisk:

In the terminal type:

```
sudo fdisk -l imgfile.img
```

You will see an output similar to the one below:

Device	boot	Start	End	Blocks	Id	System
imgfile.img1	*	63	266544	722233	C	W95 FAT32 (LBA)
imgfile.img2		25679	25367890	245667890+	83	Linux

As you can see there are two partitions, one that is FAT32 and the other one that it's ExtFS. This means that to mount the first partition we have to tell Linux that we need to start at the sector 63. The standard sector size is 512 bytes, however there are other possibilities like 128 or 1024. Assuming that the place from where you are downloading the image doesn't specify any sector size, we can type in the terminal:

```
sudo mount -t vfat -o loop,offset=$((63 * 512)) imgfile.img /mnt/disk
```

To mount the second partition, as you can imagine:

```
mount -t ext4 -o loop,offset=$((25679 * 512)) imgfile.img /mnt/disk1
```

It's important to copy the "Start" sector number correctly, otherwise you'll get an error message like:



mount : wrong fs type, bad option, band superblock on /dev/loop,
missing codepage or helper program, or other error
In some cases useful info is found in syslog – try
dmesg | tail or so

One last thing, the standard sector size for CDs and DVDs is 2352 instead of 512. If you are opening such image, you'll have to use this value instead of 512.

Incoming search terms:

- <https://stefanoprenna.com/blog/2014/09/22/tutorial-how-to-mount-raw-images-img-images-on-linux/>

Kernel Boot Parameters / Kernel Parameters

Kernel Boot Parameters

About Kernel Boot Parameters:

Kernel Parameters/kernel boot parameters, sometimes also known as kernel arguments, are used to customize the behavior of Linux and some Linux features at boot time only. Kernel boot parameters can be modified in the `/etc/default/grub` configuration file

This document provides simple instructions for adding a Linux kernel boot parameter to an Ubuntu system using **GRUB 2**.

When advised to try using a particular kernel boot parameter, it is usually a good idea to first add the parameter *temporarily* for testing. Once you have determined that adding the parameter has the desired effect, you can then add it *permanently*. Directions for both methods are below.

Kernel boot parameters are text strings which are interpreted by the system to change specific behaviors and enable or disable certain features. Kernel boot parameters:

- are case-sensitive
- are sometimes just a simple word ("splash" or "noapic")
- sometimes have an = sign with various possible values ("acpi_backlight=vendor")
- sometimes include punctuation ("i8042.noloop")

Always enter kernel boot parameters exactly as advised. Errors in spelling or formatting or invalid values will not result in any error message, they will simply have no effect.

The list of valid parameters may change between different kernel versions, refer to these Ubuntu [man pages](#):

- [bootparam\(7\)](#)
- [kernel-command-line\(7\)](#)

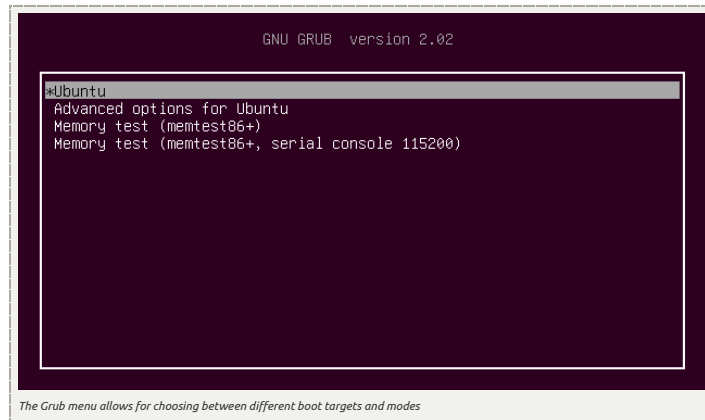
A complete list of parameters supported by the *latest* mainline kernel is available at [The kernel's command-line parameters](#).

Contents

1. [About Kernel Boot Parameters](#)
2. [Temporarily Add a Kernel Boot Parameter for Testing](#)
3. [Permanently Add a Kernel Boot Parameter](#)

Temporarily Add a Kernel Boot Parameter for Testing

1. Restart your system, or shut it down and then power it on again.
2. As the system starts to boot repeated tap the **Esc** key in order to force the "GNU GRUB" menu to appear. For BIOS-mode (not UEFI-mode) booting a **SHIFT** key can be held down continuously until the menu appears. The "GNU GRUB" menu will look something like this:



- If the system hangs at the initial vendor logo screen when you hold down **SHIFT**, just release the **SHIFT** key briefly then hold it down again until the "GNU GRUB" menu appears.

Once the "GNU GRUB" menu appears highlight the desired entry and press the **e** key to enter edit mode. The edit mode screen will look something like this:



Use the down arrow key to move the cursor to the line beginning with the word "linux", then press the **END** key to move the cursor to the end of that line. Normally this will be just after the words "quiet splash".

- Press **SPACE** to add a blank space (after "splash") then carefully type in the kernel boot parameter that you need to add. (If you need to add multiple parameters separate them with **SPACE** but do not add spaces before or after any = signs or punctuation in the parameters themselves).
- Press **Ctrl+X** to boot the system using the parameters you've added.

Note: The parameter(s) you've added to the linux line are only temporary: They will affect this boot session, but once you restart the system they will no longer be in effect.

Permanently Add a Kernel Boot Parameter

1. Log in to the system and start a terminal window (Applications->Accessories->Terminal).
2. In the terminal window at the **\$** prompt, enter the command: **sudo gedit /etc/default/grub**
 - Enter your password when prompted by [sudo].
 - If the file /etc/default/grub appears to be empty or does not exist, see the instructions for earlier releases above).
3. In the editor window, use the arrow keys to move the cursor to the line beginning with "GRUB_CMDLINE_LINUX_DEFAULT" then edit that line, adding your parameter(s) to the text *inside* the double-quotes after the words "quiet splash". (Be sure to add a **SPACE** after "splash" before adding your new parameter.) Click the **Save** button, then close the editor window.
4. In the terminal window at the **\$** prompt, enter the command: **sudo update-grub**
5. Restart the system.

Note: The parameter(s) you've added to the GRUB_CMDLINE_LINUX_DEFAULT line are persistent, and will be in effect for every subsequent boot session (unless you remove them by repeating the procedure above).

modify kernel parameters with commands only/custom .cfg file (helpful for automation)
one can add configuration files to /etc/default/grub.d in order to influence the generation of:
/etc/default/grub

Scripts: /etc/default/grub.d/customgrubscript.cfg

The scripts in this directory are read during execution of the update-grub command and their instructions are incorporated into /boot/grub/grub.cfg. you can automate the editing of kernel parameters by using this.

-ultimately /boot/grub2/grub.cfg is where the generated configuration is saved for GRUB to use, but it is not meant to be edited by users, so grub configurations are made in /etc/default/grub and in
/etc/default/grub.d/script.cfg

:::its syntax is as follows:::

GRUB_CMDLINE_LINUX_DEFAULT="\$GRUB_CMDLINE_LINUX_DEFAULT <desired kernel parameters to add>"

or

GRUB_CMDLINE_LINUX_DEFAULT="<desired kernel parameters to add>"

-- example

GRUB_CMDLINE_LINUX_DEFAULT="\$GRUB_CMDLINE_LINUX_DEFAULT mitigations=off"

or

GRUB_CMDLINE_LINUX_DEFAULT="mitigations=off"

sudo update-grub

::::to automate this in a bash file::::

echo 'GRUB_CMDLINE_LINUX_DEFAULT="\$GRUB_CMDLINE_LINUX_DEFAULT <desired-kernel-parameters>"' \
| sudo tee /etc/default/grub.d/<name-of-custom-script>.cfg

or

echo 'GRUB_CMDLINE_LINUX_DEFAULT="<desired-kernel-parameters>"' \
| sudo tee /etc/default/grub.d/<name-of-custom-script>.cfg

OR TO APPEND TO THE FILE:

echo 'GRUB_CMDLINE_LINUX_DEFAULT="<desired-kernel-parameters>"' \
| sudo tee -a /etc/default/grub.d/<name-of-custom-script>.cfg

sudo update-grub

** confirm that changes were applied correctly: cat /proc/cmdline

Main page
Table of contents
Getting involved
Wiki news
Random page

Interaction

Help
Contributing
Recent changes
Recent talks
New pages
Statistics
Requests

Tools
What links here
Related changes
Special pages
Printable version
Permanent link
Page information

In other languages

Español
日本語
Português
中文（简体）

Page Discussion

Read

View source

View history

Search ArchWiki



Kernel parameters

There are three ways to pass options to the kernel and thus control its behaviour:

1. When building the kernel—in the kernel's `config` file. See [Kernel#Compilation](#) for details.
2. When starting the kernel—using command line parameters (usually through a boot loader).
3. At runtime—through the files in `/proc/sys/` (see [sysctl](#)) and `/sys/`.

Between the three methods, the configurable options differ in availability, their name and the method in which they are specified. This page only explains the second method (kernel command line parameters) and shows a list of the most used kernel parameters in Arch Linux.

Most parameters are associated with subsystems and work only if the kernel is configured with those subsystems built in. They also depend on the presence of the hardware they are associated with.

Kernel command line parameters either have the format `parameter` or `parameter=value`.

Note: All kernel parameters are case-sensitive.

Tip: The options of loadable modules can be set via `.conf` files in `/etc/modprobe.d/`. See [Kernel module#Using files in /etc/modprobe.d/](#).

Contents [hide]

- 1 [Configuration](#)
 - 1.1 [Syslinux](#)
 - 1.2 [systemd-boot](#)
 - 1.3 [GRUB](#)
 - 1.4 [GRUB Legacy](#)
 - 1.5 [LILO](#)
 - 1.6 [rEFInd](#)
 - 1.7 [EFISTUB](#)
 - 1.8 [dracut](#)
 - 1.9 [Hijacking cmdline](#)
- 2 [Parameter list](#)
- 3 [See also](#)

Configuration

Note:

- You can check the parameters your system was booted up with by running `cat /proc/cmdline` and see if it includes your changes.
- The Arch Linux [installation medium](#) uses [Syslinux](#) for [BIOS](#) systems, and [systemd-boot](#) for [UEFI](#) systems.

Kernel parameters can be set either temporarily by editing the boot entry in the boot loader's boot selection menu, or permanently by modifying the boot loader's configuration file.

The following examples add the `quiet` and `splash` parameters to [Syslinux](#), [systemd-boot](#), [GRUB](#), [GRUB Legacy](#), [LILO](#), and [rEFInd](#).

Syslinux

- Press `Tab` when the menu shows up and add them at the end of the string:

```
linux /boot/vmlinuz-linux root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw initrd=/boot/initramfs-linux.img quiet splash
```

Press `Enter` to boot with these parameters.

- To make the change persistent after reboot, edit `/boot/syslinux/syslinux.cfg` and add them to the `APPEND` line:

```
APPEND root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
```

For more information on configuring Syslinux, see the [Syslinux](#) article.

systemd-boot

- Press `e` when the menu appears and add the parameters to the end of the string:

```
initrd=\initramfs-linux.img root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
```

Press `Enter` to boot with these parameters.

Note:

- If you have not set a value for menu timeout, you will need to hold `Space` while booting for the systemd-boot menu to appear.
- If you cannot edit the parameters from the boot menu, you may need to edit `/boot/loader/loader.conf` and add `editor 1` to enable editing.

- To make the change persistent after reboot, edit `/boot/loader/entries/arch.conf` (assuming you set up your [EFI system partition](#)) and add them to the `options` line:

```
options root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
```

For more information on configuring systemd-boot, see the [systemd-boot](#) article.

GRUB

- Press `e` when the menu shows up and add them on the `linux` line:

```
linux /boot/vmlinuz-linux root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
```

Press `Ctrl+x` to boot with these parameters.

- To make the change persistent after reboot, you *could* manually edit `/boot/grub/grub.cfg` with the exact line from above, but the best practice is to: Edit `/etc/default/grub` and append your kernel options between the quotes in the `GRUB_CMDLINE_LINUX_DEFAULT` line:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
```

And then automatically re-generate the `grub.cfg` file with:

```
# grub-mkconfig -o /boot/grub/grub.cfg
```

For more information on configuring GRUB, see the [GRUB](#) article.

GRUB Legacy

- Press `e` when the menu shows up and add them on the `kernel` line:

```
kernel /boot/vmlinuz-linux root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
```

Press `b` to boot with these parameters.

- To make the change persistent after reboot, edit `/boot/grub/menu.lst` and add them to the `kernel` line, exactly like above.

For more information on configuring GRUB Legacy, see the [GRUB Legacy](#) article.

LILO

- Add them to `/etc/lilo.conf`:

```
image=/boot/vmlinuz-linux
...
quiet splash
```

For more information on configuring LILO, see the [LILO](#) article.

rEFInd

- Press `+`, `F2`, or `Insert` on the desired menu entry and press it again on the submenu entry. Add kernel parameters at the end of the string:

```
root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw initrd=\boot\initramfs-linux.img quiet splash
```

Press `Enter` to boot with these parameters.

- To make the change persistent after reboot, edit `/boot/refind_linux.conf` and append them between the quotes in all required lines, for example

```
"Boot using default options" "root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash"
```

- If you have disabled auto-detection of OSes in rEFInd and are defining OS stanzas instead in `esp/EFI/refind/refind.conf` to load your OSes, you can edit it like:

```
menuentry "Arch Linux" {
...
options "root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash"
...
}
```

For more information on configuring rEFInd, see the [rEFInd](#) article.

EFISTUB

See [EFISTUB#Using UEFI directly](#).

dracut

[dracut](#) is capable of embedding the kernel parameters in the initramfs, thus allowing to omit them from the boot loader configuration. See [dracut#Kernel command line options](#).

Hijacking cmdline

Even without access to your bootloader it is possible to change your kernel parameters to enable debugging (if you have root access). This can be accomplished by overwriting `/proc/cmdline` which stores the kernel parameters. However `/proc/cmdline` is not writable even as root, so this hack is accomplished by using a bind mount to mask the path.

First create a file containing the desired kernel parameters

```
/root/cmdline

root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 ro console=tty1 logo.nologo debug
```

Then use a bind mount to overwrite the parameters

```
# mount -n --bind -o ro /root/cmdline /proc/cmdline
```

The `-n` option skips adding the mount to `/etc/mtab`, so it will work even if root is mounted read-only. You can `cat /proc/cmdline` to confirm that your change was successful.

Parameter list

This list is not comprehensive. For a complete list of all options, please see the [kernel documentation](#).

parameter	Description
init	Run specified binary instead of <code>/sbin/init</code> as init process. The <code>systemd-sysvcompat</code> package symlinks <code>/sbin/init</code> to <code>/usr/lib/systemd/systemd</code> to use systemd . Set it to <code>/bin/sh</code> to boot to the shell.
initrd	Specify the location of the initial ramdisk. For UEFI boot managers and EFISTUB , the path must be specified using backslashes (<code>\</code>) as path separators.
debug	Enable kernel debugging (events log level).
lsm	Set the initialisation order of the Linux security modules, used to install AppArmor .
maxcpus	Maximum number of processors that an SMP kernel will bring up during bootup.
mem	Force usage of a specific amount of memory to be used.
netdev	Network devices parameters.
nomodeset	Disable Kernel mode setting .
nvidia-drm.modeset	Enables the proprietary NVIDIA Direct Rendering Manager .
panic	Time before automatic reboot on kernel panic.
resume	Specify a swap device to use when waking from hibernation .
ro	Mount root device read-only on boot (default ¹).
root	Root filesystem. See init/do_mounts.c for kernel's supported device name formats. Note that an initramfs with udev supports more name formats .
rootflags	Root filesystem mount options. Useful for setting options that cannot be applied by remounting (i.e. by <code>systemd-remount-fs.service(8)</code>). For example, the <code>discard</code> option of an XFS root volume.
rw	Mount root device read-write on boot.
systemd.unit	Boot to a specified target .
video	Override framebuffer video defaults.
zswap.enabled	Enable Zswap .

¹ **mkinitcpio** uses `ro` as default value when neither `rw` or `ro` is set by the **boot loader**. Boot loaders may set the value to use, for example GRUB uses `rw` by default (see **FS#36275** as a reference).

See also

- [“The kernel’s command-line parameters” documentation](#)
- [Power saving#Kernel parameters](#)
- [List of kernel parameters with further explanation and grouped by similar options](#)

Category: [Kernel](#)

Kernel module

🌐 8 languages ▾

PageDiscussionReadView sourceView history

Categories: [Kernel](#) | [Hardware detection and troubleshooting](#) | [Boot process](#)

Kernel modules are pieces of code that can be loaded and unloaded into the kernel upon demand. They extend the functionality of the kernel without the need to reboot the system.

To create a kernel module, you can read [The Linux Kernel Module Programming Guide](#). A module can be configured as built-in or loadable. To dynamically load or remove a module, it has to be configured as a loadable module in the kernel configuration (the line related to the module will therefore display the letter `M`).

To rebuild a kernel module automatically when a new kernel is installed, see [Dynamic Kernel Module Support](#) (DKMS).

- Related articles
- [Boot debugging](#)
- [Kernel](#)
- [Kernel parameters](#)
- [Compile kernel module](#)

1 Obtaining information

Modules are stored in `/usr/lib/modules/kernel_release`. You can use the command `uname -r` to get your current kernel release version.

Note: Module names often use underscores (`_`) or dashes (`-`); however, those symbols are interchangeable when using the `modprobe` command and in configuration files in `/etc/modprobe.d/`.

To show what kernel modules are currently loaded:

```
$ lsmod
```

To show information about a module:

```
$ modinfo module_name
```

To list the options that are set for a loaded module use `systool (1)` from `sysfsutils`:

```
$ systool -v -m module_name
```

To display the comprehensive configuration of all the modules:

```
$ modprobe -c | less
```

To display the configuration of a particular module:

```
$ modprobe -c | grep module_name
```

List the dependencies of a module (or alias), including the module itself:

```
$ modprobe --show-depends module_name
```

2 Automatic module loading

Today, all necessary modules loading is handled automatically by `udev`, so if you do not need to use any out-of-tree kernel modules, there is no need to put modules that should be loaded at boot in any configuration file. However, there are cases where you might want to load an extra module during the boot process, or blacklist another one for your computer to function properly.

2.1 systemd

Kernel modules can be explicitly listed in files under `/etc/modules-load.d/` for `systemd` to load them during boot. Each configuration file is named in the style of `/etc/modules-load.d/program.conf`. Configuration files simply contain a list of kernel modules names to load, separated by newlines. Empty lines and lines whose first non-whitespace character is `#` or `;` are ignored.

```
/etc/modules-load.d/virtio-net.conf
-----
# Load virtio_net.ko at boot
virtio_net
```

See `modules-load.d(5)` for more details.

2.2 Early module loading

The `inittarafs` image may not contain the kernel modules asked for in `/etc/modules-load.d/`, it also may lack the files that have been set in that folder. Early module loading depend on the inittarafs generator used:

- `mkinitpio`: see `Mkinitpio#MODULES`
- `dracut`: see `Dracut#Early kernel module loading`
- `boosted`: see `Booster#Early module loading`

3 Manual module handling

Kernel modules are handled by tools provided by `kmod` package. You can use these tools manually.

Note: If you have upgraded your kernel but have not yet rebooted, `modprobe` will fail with no error message and exit with code 1, because the path `/usr/lib/modules/$ (uname -r) /` no longer exists. Check manually if this path exists when `modprobe` failed to determine if this is the case.

To load a module:

```
# modprobe module_name
```

To load a module by filename (i.e. one that is not installed in `/usr/lib/modules/$ (uname -r) /`):

```
# insmod filename [args]
```

To unload a module:

```
# modprobe -r module_name
```

Or, alternatively:

```
# rmmod module_name
```

4 Setting module options

To pass a parameter to a kernel module, you can pass them manually with `modprobe` or assure certain parameters are always applied using a `modprobe` configuration file or by using the kernel command line. If the module is built into the kernel, the kernel command line must be used and other methods will not work.

4.3 Manually at load time using modprobe

The basic way to pass parameters to a module is using the `modprobe` command. Parameters are specified on command line using simple `key=value` assignments:

```
# modprobe module_name parameter_name=parameter_value
```

4.4 Using files in /etc/modprobe.d/

Files in `/etc/modprobe.d/` directory can be used to pass module settings to `udev`, which will use `modprobe` to manage the loading of the modules during system boot. Configuration files in this directory can have any name, given that they end with the `.conf` extension. The syntax is:

```
/etc/modprobe.d/myfilename.conf
-----
options module_name parameter_name=parameter_value
```

Multiple module parameters are separated by spaces, in turn a parameter can receive a list of values which is separated by commas:

```
/etc/modprobe.d/myfilename.conf
-----
options module_name param1=value1 param2=value2a,value2b ...
```

For example:

```
/etc/modprobe.d/thinkfan.conf
-----
# On ThinkPads, this lets the 'thinkfan' daemon control fan speed
options thinkpad_acpi fan_control=1
```

Note: If any of the affected modules is loaded from the `initramfs`, then you will need to add the appropriate `.conf` file to `FILES` in `mkinitcpio.conf` or use the `modconf` `hook`, then **regenerate the `initramfs`** to include the `.conf` file. To see the contents of the default `initramfs` use `lsinitcpio`.

4.5 Using kernel command line

You can also pass options to the module using the kernel command line. This is the only working option for modules built into the kernel. For all common boot loaders, the following syntax is correct:

```
module_name.parameter_name=parameter_value
```

For example:

```
thinkpad_acpi.fan_control=1
```

Simply add this to your bootloader's kernel-line, as described in [Kernel Parameters](#).

5 Aliasing

Aliases are alternate names for a module. For example: `alias my-mod really_long_modulename` means you can use `modprobe my-mod` instead of `modprobe really_long_modulename`. You can also use shell-style wildcards, so `alias my-mod* really_long_modulename` means that `modprobe my-mod-something` has the same effect. Create an alias:

```
/etc/modprobe.d/myalias.conf
-----
alias mymod really_long_module_name
```

Some modules have aliases which are used to automatically load them when they are needed by an application. Disabling these aliases can prevent automatic loading but will still allow the modules to be manually loaded.

```
/etc/modprobe.d/modprobe.conf
-----
# Prevent Bluetooth autoload
alias net-pf-3l off
```

6 Blacklisting

Blacklisting, in the context of kernel modules, is a mechanism to prevent the kernel module from loading. This could be useful if, for example, the associated hardware is not needed, or if loading that module causes problems: for instance there may be two kernel modules that try to control the same piece of hardware, and loading them together would result in a conflict.

Some modules are loaded as part of the **initramfs**. `mkinitcpio -M` will print out all automatically detected modules: to prevent the `initramfs` from loading some of those modules, blacklist them in a `.conf` file under `/etc/modprobe.d` and it shall be added in by the `modconf` hook during image generation. Running `mkinitcpio -v` will list all modules pulled in by the various hooks (e.g. `filesystems` hook, `block` hook, etc.). Remember to add that `.conf` file to the `FILES` array in `/etc/mkinitcpio.conf` if you do not have the `modconf` hook in your `HOOKS` array (e.g. you have deviated from the default configuration), and once you have blacklisted the modules **regenerate the `initramfs`**, and reboot afterwards.

6.6 Using files in /etc/modprobe.d/

Create a `.conf` file inside `/etc/modprobe.d/` and append a line for each module you want to blacklist, using the `blacklist` keyword. If for example you want to prevent the `pcspkr` module from loading to avoid sounds through the **PC speaker**:

```
/etc/modprobe.d/nobeep.conf
-----
# Do not load the 'pcspkr' module on boot.
blacklist pcspkr
```

Note: The `blacklist` command will blacklist a module so that it will not be loaded automatically, but the module may be loaded if another non-blacklisted module depends on it or if it is loaded manually.

However, there is a workaround for this behaviour; the `install` command instructs `modprobe` to run a custom command instead of inserting the module in the kernel as normal, so you can force the module to always fail loading with:

```
/etc/modprobe.d/blacklist.conf
-----
...
install module_name /bin/true
...
```

This will effectively blacklist that module and any other that depends on it.

6.7 Using kernel command line

Tip: This can be very useful if a broken module makes it impossible to boot your system.

You can also blacklist modules from the bootloader.

Simply add `module_blacklist=modname1,modname2,modname3` to your bootloader's kernel line, as described in [Kernel parameters](#).

Note: When you are blacklisting more than one module, note that they are separated by commas only. Spaces or anything else might presumably break the syntax.

7 Troubleshooting

7.8 Modules do not load

In case a specific module does not load and the boot log (accessible by running `journalctl -b` as root) says that the module is blacklisted, but the directory `/etc/modprobe.d/` does not show a corresponding entry, check another `modprobe` source directory at `/usr/lib/modprobe.d/` for blacklisting entries.

A module will not be loaded if the "vermagic" string contained within the kernel module does not match the value of the currently running kernel. If it is known that the module is compatible with the current running kernel the "vermagic" check can be ignored with `modprobe --force-vermagic`.

Warning: Ignoring the version checks for a kernel module can cause a kernel to crash or a system to exhibit undefined behavior due to incompatibility. Use `--force-vermagic` only with the utmost caution.

8 See also

- [Disable PC speaker beep](#)
- [Writing a WMI driver](#) [↗](#) - an LWM introduction

dd command linux and securely wipe disks

dd

dd is a **core utility** whose primary purpose is to convert and copy a file.

Similarly to *cp*, by default *dd* makes a bit-to-bit copy of the file, but with lower-level I/O flow control features.

For more information see [dd \(1\)](#) or the [full documentation](#).

Tip: By default, *dd* outputs nothing until the task has finished. To monitor the progress of the operation, add the `status=progress` option to the command.

Warning: One should be extremely cautious using *dd*, as with any command of this kind it can destroy data irreversibly.

Contents [\[hide\]](#)

- 1 [Installation](#)
- 2 [Disk cloning and restore](#)
 - 2.1 [Cloning a partition](#)
 - 2.2 [Cloning an entire hard disk](#)
 - 2.3 [Backing up the partition table](#)
 - 2.4 [Create disk image](#)
 - 2.5 [Restore system](#)
- 3 [Binary file patching](#)
- 4 [Backup and restore MBR](#)
 - 4.1 [Remove bootloader](#)
- 5 [Troubleshooting](#)
 - 5.1 [Partial read](#)

Installation

dd is part of the GNU **coreutils**. For other utilities in the package, please refer to [Core utilities](#).

Disk cloning and restore

The *dd* command is a simple, yet versatile and powerful tool. It can be used to copy from source to destination, block-by-block, regardless of their filesystem types or operating systems. A convenient method is to use *dd* from a live environment, as in a Live CD.

Warning: As with any command of this type, you should be very cautious when using it; it can destroy data. Remember the order of input file (`if=`) and output file (`of=`) and do not reverse them! Always ensure that the destination drive or partition (`of=`) is of equal or greater size than the source (`if=`).

Cloning a partition

From physical disk `/dev/sda`, partition 1, to physical disk `/dev/sdb`, partition 1:

```
# dd if=/dev/sda1 of=/dev/sdb1 bs=64K conv=noerror,sync status=progress
```

Note: Be careful that if the output partition `of=` (`sdb1` in the example) does not exist, `dd` will create a file with this name and will start filling up your root file system.

Cloning an entire hard disk

From physical disk `/dev/sda` to physical disk `/dev/sdb`:

```
# dd if=/dev/sda of=/dev/sdb bs=64K conv=noerror,sync status=progress
```

This will clone the entire drive, including the MBR (and therefore bootloader), all partitions, UUIDs, and data.

- `bs=` sets the block size. Defaults to 512 bytes, which is the "classic" block size for hard drives since the early 1980s, but is not the most convenient. Use a bigger value, 64K or 128K. Also, please read the warning below, because there is more to this than just "block sizes" -it also influences how read errors propagate. See [\[1\]](#) and [\[2\]](#) for details and to figure out the best `bs` value for your use case.
- `noerror` instructs `dd` to continue operation, ignoring all read errors. Default behavior for `dd` is to halt at any error.
- `sync` fills input blocks with zeroes if there were any read errors, so data offsets stay in sync.
- `status=progress` shows periodic transfer statistics which can be used to estimate when the operation may be complete.

Note: The block size you specify influences how read errors are handled. Read below. For data recovery, use [ddrescue](#).

The `dd` utility technically has an "input block size" (IBS) and an "output block size" (OBS). When you set `bs`, you effectively set both IBS and OBS. Normally, if your block size is, say, 1 MiB, `dd` will read 1024×1024 bytes and write as many bytes. But if a read error occurs, things will go wrong. Many people seem to think that `dd` will "fill up read errors with zeroes" if you use the `noerror,sync` options, but this is not what happens. `dd` will, according to documentation, fill up the OBS to IBS size *after completing its read*, which means adding zeroes at the *end* of the block. This means, for a disk, that effectively the whole 1 MiB would become messed up because of a single 512 byte read error in the beginning of the read: 12ERROR89 would become 128900000 instead of 120000089.

If you are positive that your disk does not contain any errors, you could proceed using a larger block size, which will increase the speed of your copying several fold. For example, changing `bs` from 512 to 64K changed copying speed from 35 MB/s to 120 MB/s on a simple Celeron 2.7 GHz system. But keep in mind that read errors on the source disk will end up as *block errors* on the destination disk, i.e. a single 512-byte read error will mess up the whole 64 KiB output block.

Tip: If you would like to view `dd` progressing, use the `status=progress` option. See [dd\(1\)](#) for details.

Note:

- To regain unique UUIDs of an `ext2/3/4` filesystem, use `tune2fs /dev/sdXY -U random` on every partition. For swap partitions, use `mkswap /dev/sdXY` instead.
- Partition table changes from `dd` are not registered by the kernel. To notify of changes without rebooting, use a utility like `partprobe` (part of [GNU Parted](#)).

Backing up the partition table

See [fdisk#Backup and restore partition table](#) or [gdisk#Backup and restore partition table](#) .

Create disk image

Boot from a live medium and make sure no partitions are mounted from the source hard drive.

Then mount the external hard drive and backup the drive:

```
# dd if=/dev/sda conv=sync,noerror bs=64K | gzip -c > /path/to/backup.img.gz
```

If necessary (e.g. when the resulting files will be stored on a [FAT32](#) file system) split the disk image into multiple parts (see also [split\(1\)](#)):

```
# dd if=/dev/sda conv=sync,noerror bs=64K | gzip -c | split -a3 -b2G - /path/to/backup.
img.gz
```

If there is not enough disk space locally, you may send the image through *ssh*:

```
# dd if=/dev/sda conv=sync,noerror bs=64K | gzip -c | ssh user@local dd of=backup.img.g
z
```

Finally, save extra information about the drive geometry necessary in order to interpret the partition table stored within the image. The most important of which is the cylinder size.

```
# fdisk -l /dev/sda > /path/to/list_fdisk.info
```

Note: You may wish to use a block size (`bs=`) that is equal to the amount of cache on the HD you are backing up. For example, `bs=8192K` works for an 8 MiB cache. The 64 KiB mentioned in this article is better than the default `bs=512` bytes, but it will run faster with a larger `bs=` .

Tip: *gzip* is only able to compress data using a single CPU core, which leads to a data throughput considerably lower than the write speeds on modern storage. In order to leverage multicore compression and create a disk image more quickly, one could for instance install the [pigz](#) package, and simply replace the `gzip -c` command above with `pigz -c` . For large disks, this can potentially save hours.

Restore system

To restore your system:

```
# gunzip -c /path/to/backup.img.gz | dd of=/dev/sda
```

When the image has been split, use the following instead:

```
# cat /path/to/backup.img.gz* | gunzip -c | dd of=/dev/sda
```

Binary file patching

If one wants to replace offset `0x123AB` of a file with the `FF C0 14` hexadecimal sequence, this can be done with the command line:

```
# printf '\xff\xc0\x14' | dd seek=$((0x123AB)) conv=notrunc bs=1 of=/path/to/file
```

Backup and restore MBR

Before making changes to a disk, you may want to backup the partition table and partition scheme of the drive. You can also use a backup to copy the same partition layout to numerous drives.

The MBR is stored in the the first 512 bytes of the disk. It consists of 4 parts:

1. The first 440 bytes contain the bootstrap code (boot loader).
2. The next 6 bytes contain the disk signature.
3. The next 64 bytes contain the partition table (4 entries of 16 bytes each, one entry for each primary partition).
4. The last 2 bytes contain a boot signature.

To save the MBR as `mbr_file.img`:

```
# dd if=/dev/sdX of=/path/to/mbr_file.img bs=512 count=1
```

You can also extract the MBR from a full dd disk image:

```
# dd if=/path/to/disk.img of=/path/to/mbr_file.img bs=512 count=1
```

To restore (be careful, this destroys the existing partition table and with it access to all data on the disk):

```
# dd if=/path/to/mbr_file.img of=/dev/sdX bs=512 count=1
```

Warning: Restoring the MBR with a mismatching partition table will make your data unreadable and nearly impossible to recover. If you simply need to reinstall the bootloader see their respective pages as they also employ the **DOS compatibility region** : **GRUB** or **Syslinux**.

If you only want to restore the boot loader, but not the primary partition table entries, just restore the first 440 bytes of the MBR:

```
# dd if=/path/to/mbr_file.img of=/dev/sdX bs=440 count=1
```

To restore only the partition table, one must use:

```
# dd if=/path/to/mbr_file.img of=/dev/sdX bs=1 skip=446 count=64
```

Remove bootloader

To erase the MBR bootstrap code (may be useful if you have to do a full reinstall of another operating system) only the first 440 bytes need to be zeroed:

```
# dd if=/dev/zero of=/dev/sdX bs=440 count=1
```

Troubleshooting

Partial read

Files created with *dd* can end up with a smaller size than requested if a full input block is not available and the `read(2)` system call returns early. This can happen when reading from a [pipe\(7\)](#) or when reading from `/dev/random` and there is not enough entropy [\[3\]](#), or from `/dev/urandom` when reading more than 32 MiB [\[4\]](#).

It is possible, but not guaranteed, that *dd* will warn you about the issue:

```
dd: warning: partial read (X bytes); suggest iflag=fullblock
```

The solution is to do as the warning says and add `iflag=fullblock` to the *dd* command. For example:

```
$ dd if=/dev/random of=bigsecret.img bs=1K count=1 iflag=fullblock
$ dd if=/dev/urandom of=bigsecret.img bs=40M count=1 iflag=fullblock
```

Note: It is strongly recommended to always add the `iflag=fullblock` option to the *dd* command when the input file is `/dev/random` or `/dev/urandom`.

An alternative for `/dev/urandom` is to specify a block size smaller than 32 MiB, but a larger copy count. For example:

```
$ dd if=/dev/urandom of=bigsecret.img bs=1M count=40
```

When reading from a pipe, an alternative to `iflag=fullblock` is to limit `bs` to the `PIPE_BUF` constant value, defined in `/usr/include/linux/limits.h` [\[5\]](#). For example:

```
$ cat input.img | dd of=output.img bs=4k count=100
```

dd block sizes (from `dd --help`): `bs=`

```
N and BYTES may be followed by the following multiplicative suffixes:
c=1, w=2, b=512, kB=1000, K=1024, MB=1000*1000, M=1024*1024, xM=M,
GB=1000*1000*1000, G=1024*1024*1024, and so on for T, P, E, Z, Y.
Binary prefixes can be used, too: KiB=K, MiB=M, and so on.
```

Securely Wipe Disks

Securely wipe disk

Wiping a disk is done by writing new data over every single bit.

Tip: References to "disks" in this article also apply to loopback devices.

Contents [\[hide\]](#)

- 1 [Common use cases](#)
 - 1.1 [Wipe all data left on the device](#)
 - 1.2 [Preparations for block device encryption](#)
- 2 [Data remanence](#)
 - 2.1 [Operating system, programs and filesystem](#)
 - 2.2 [Hardware-specific issues](#)
 - 2.2.1 [Flash memory](#)
 - 2.2.2 [Marked Bad Sectors](#)
 - 2.2.3 [Residual magnetism](#)
- 3 [Select a target](#)
- 4 [Select a data source](#)
 - 4.1 [Zeros](#)
 - 4.2 [Random data](#)
- 5 [Select a block size](#)
 - 5.1 [Calculate blocks to wipe manually](#)
- 6 [Overwrite the target](#)
 - 6.1 [By redirecting output](#)
 - 6.2 [dd](#)
 - 6.3 [wipe](#)
 - 6.4 [shred](#)
 - 6.5 [Badblocks](#)
 - 6.6 [hdparm](#)
- 7 [See also](#)

TO ZERO FILL

**** alternativeley you can use: ****
sfill - part of secure-delete package
-wipe free space/unallocated non zero
(use sfill -fllzv <device_mnt_point>)

to zero fill directory
without overwriting whole device
along with its partition tables

Common use cases

Wipe all data left on the device

The most common usecase for completely and irrevocably wiping a device is when the device is going to be given away or sold. There may be (unencrypted) data left on the device and you want to protect against simple forensic investigation that is mere child's play with for example [File recovery](#) software.

If you want to quickly wipe everything from the disk, `/dev/zero` or simple patterns allow maximum performance while adequate randomness can be advantageous in some cases that should be covered up in [#Data remanence](#).

Every overwritten bit means to provide a level of data erasure not allowing recovery with normal system functions (like standard ATA/SCSI commands) and hardware interfaces. Any file recovery software mentioned above then would need to be specialized on proprietary storage-hardware features.

In case of a HDD, data recreation will not be possible without at least undocumented drive commands or tinkering with the device's controller or firmware to make them read out for example reallocated sectors (bad blocks that [S.M.A.R.T.](#) retired from use).

There are different wiping issues with different physical storage technologies. Most notably, all Flash memory based devices and older magnetic storage (old HDDs, floppy disks, tape).

Preparations for block device encryption

To prepare a drive for [block device encryption](#) inside the wiped area afterwards, it is recommended to use [#Random data](#) generated by a cryptographically strong random number generator (referred to as RNG in this article from now on).

See also [Wikipedia:Random number generation](#) .

Warning: If block device encryption is mapped on a partition that contains non-random or unencrypted data, the encryption is weakened and becomes comparable to filesystem-level encryption: disclosure of usage patterns on the encrypted drive becomes possible. Therefore, do not fill space with zeros, simple patterns (like badblocks) or other non-random data before setting up block device encryption if you are serious about it.

Data remanence

See also [Wikipedia:Data remanence](#) . The representation of data may remain even after attempts have been made to remove or erase the data.

Operating system, programs and filesystem

The operating system, executed programs or [journaling file systems](#)  may copy your unencrypted data throughout the block device. When writing to plain disks, this should only be relevant in conjunction with one of the above.

If the data can be exactly located on the disk and was never copied anywhere else, wiping with random data can be thoroughgoing and impressively quick as long there is enough entropy in the pool.

A good example is cryptsetup using `/dev/urandom` for [wiping the LUKS keyslots](#) .

Hardware-specific issues

Flash memory

[Write amplification](#)  and other characteristics make Flash memory, including SSDs, a stubborn target for reliable wiping. As there is a lot of transparent abstraction in between data as seen by a device's controller chip and the operating system, sight data is never overwritten in place and wiping particular blocks or files is not reliable.

Other "features" like transparent compression (all SandForce SSDs) can compress your zeros or repetitive patterns, so if wiping is fast beyond belief this might be the cause.

Disassembling Flash memory devices, unsoldering the chips and analyzing data content without the controller in between is feasible without difficulty using [simple hardware](#) . Data recovery companies do it for cheap money.

For more information see:

- [Solid state drive/Memory cell clearing](#)
- [Reliably Erasing Data From Flash-Based Solid State Drives](#) .
- [#Select a target](#)

Marked Bad Sectors

If a hard drive marks a sector as bad, it cordons it off, and the section becomes impossible to write to via software. Thus a full overwrite would not reach it. However because of block sizes, these sections would only amount to a few theoretically recoverable KiB.

Residual magnetism

A single, full overwrite with zeros or random data does not lead to any recoverable data on a modern high-density storage device. Note that repeating the operation should not be necessary nowadays. [\[1\]](#)  Indications otherwise refer to single residual bits; reconstruction of byte patterns is generally not feasible. [\[2\]](#)  See also [\[3\]](#) , [\[4\]](#)  and [\[5\]](#) .

Select a target

Use **fdisk** to locate all read/write devices the user has read access to.

Check the output for lines that start with devices such as `/dev/sd"X"`.

This is an example for a HDD formatted to boot a linux system:

```
# fdisk -l

Disk /dev/sda: 250.1 GB, 250059350016 bytes, 488397168 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00ff784a

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1    *        2048     206847       102400   83   Linux
/dev/sda2                206848  488397167    244095160   83   Linux
```

Or another example with the Arch Linux image written to a 4GB USB thumb drive:

```
# fdisk -l

Disk /dev/sdb: 4075 MB, 4075290624 bytes, 7959552 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x526e236e

   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1    *           0      802815       401408   17   Hidden HPFS/NTFS
```

If you are worried about unintentional damage of important data on the primary computer, consider using an isolated environment such as a virtual environment (VirtualBox, VMWare, QEMU, etc...) with direct connected disk drives to it or a single computer only with a storage disk(s) that need to be wiped booted from a [Live Media](#) (USB, CD, PXE, etc...) or use a script to [prevent wiping mounted partitions by typo](#).

Select a data source

To wipe sensitive data, one can use any data pattern matching the needs.

Zeros

Overwriting with `/dev/zero` or simple patterns is considered secure in most situations. With today's HDDs, it is deemed appropriate and fast for disk wiping.

However, a drive that is abnormally fast in writing patterns or zeroing could be doing transparent compression. It is obviously presumable not all blocks get wiped this way. Some **#Flash memory** devices do "feature" that.

To setup block device encryption afterwards, one should wipe the area with random data (see next section) to avoid weakening the encryption.

Warning: Subject to compression and to be used carefully with flash memory and SSDs, to be avoided for block encryption preparation as stated above.

Random data

True random data source using `/dev/random` is impractical for wiping large capacities as it will take too long to wait for the entropy generation. `/dev/urandom` can be used as a reasonable source of pseudorandom data. For differences between random and pseudorandom data as source, please see [Random number generation](#).

Another alternative for pseudorandom data generation is to use an encrypted datastream. For example, if one wants to prepare a device for block encryption and will use AES for the encrypted partition, it is appropriate to wipe it with a similar cipher prior to creating the filesystem to make the empty space not distinguishable from the used space.

Select a block size

See also [Wikipedia:Dd \(Unix\)#Block size](#), [blocksize io-limits](#).

If you have an **Advanced Format** hard drive it is recommended that you specify a block size larger than the default 512 bytes. To speed up the overwriting process choose a block size matching your drive's physical geometry by appending the block size option to the `dd` command (i.e. `bs=4096` for 4 KiB).

`fdisk` prints physical and logical sector size for every disk. Alternatively `sysfs` does expose information:

```
/sys/block/sdX/size
/sys/block/sdX/queue/physical_block_size
/sys/block/sdX/queue/logical_block_size
/sys/block/sdX/sdXY/alignment_offset
/sys/block/sdX/sdXY/start
/sys/block/sdX/sdXY/size
```

Warning: These methods show the block size the drive reports to the kernel. However, many Advanced Format drives incorrectly understate the physical block size as 512.

Tip: The script `genwipe.sh`^{AUR} helps to calculate parameters to wipe a device/partition with `dd`, e.g.
`genwipe.sh /dev/sd"XY"`.

Calculate blocks to wipe manually

A block storage devices contains sectors and a size of a single sector that can be used to calculate the whole size of device in bytes. You can do it by multiplying sectors with size of the sector.

As an example we use the parameters with the `dd` command to wipe a partition:

```
# dd if=data_source of=/dev/sdX bs=sector_size count=sector_number seek=partitions_start_sector status=progress
```

Here, to illustrate with a practical example, we will show the output of the *fdisk* command on the partition `/dev/sdX`:

```
# fdisk -l /dev/sdX

Disk /dev/sdX: 1.8 TiB, 2000398934016 bytes, 3907029168 sectors
Disk model: ST3500413AS
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
...
Device      Boot      Start          End      Sectors      Size Id Type
/dev/sdX1                2048      3839711231   3839709184    1,8T 83 Linux
/dev/sdX2            3839711232   3907029167   67317936      32,1G  5 Extended
```

- The first line of the *fdisk* output shows the disk size in bytes and in logical sectors.
- The size in bytes of the storage device or of the partition can also be obtained with the command `blockdev --getsize64 /dev/sdXY`.
- The *Units* line of the *fdisk* output shows the size of single logical sector; the logical sector size can also be derived from the number of bytes divided by the number of logical sectors, here use:
`echo $((2000398934016 / 3907029168))`.
- To know the physical sector size in bytes (that will make it work faster), we can use the next line.
- To get the disk size in physical sectors, one can divide the disk size in bytes by the size of a single physical sector, here
`echo $((2000398934016 / 4096))`,

Note:

- In the examples below we will use the logical sector size.
- You can even wipe unallocated disk space with a `dd` command by calculating the difference between the end of one and start of the next partition.

To wipe partition `/dev/sdX1`, the example parameters with logical sectors would be used like follows.

- By using the starting address of the partition on the device using the `seek=` parameter:

```
# dd if=data_source of=/dev/sdX bs=${BytesInSector} count=$((End - Start)) seek=${Start} status=progress
```

with `Start=2048`, `End=3839711231` and `BytesInSector=512`.

- Or by using the partitions size in logical sectors:

```
# dd if=data_source of=/dev/sdX1 bs=${BytesInSector} count=${LogicalSectors} status=progress
```

with `LogicalSectors=3839709184`.

Or, to wipe the whole disk by using physical sectors:

```
# dd if=data_source of=/dev/sdX bs=${PhysicalSectorSizeBytes} count=${AllDiskPhysicalSectors} seek=0 status=progress
```

with `AllDiskPhysicalSectors=488378646` and `PhysicalSectorSizeBytes=4096`.

Note: The `count=` option is not necessary when wiping all the physical area e.g. `sdXY` or `sdX` from the start to the end but it shows an error about out of free space when it tries to write outside the limits.

Overwrite the target

You can choose from several utilities to overwrite a drive. If you only want to wipe a single file, [Securely wipe disk/Tips and tricks#Wipe a single file](#) has considerations in addition to the utilities mentioned below.

By redirecting output

The redirected output can be used to create files, rewrite free space on the partition, and to wipe the whole device or a single partition on it.

The following examples show how to rewrite the partition or a block device by redirecting [stdout](#) from other utilities:

```
$ cat /dev/urandom > /dev/sd"XY"

cat: write error: No space left on device
```

```
$ xz -z0 /dev/urandom -c > /dev/sd"XY"

xz: (stdout): Write error: No space left on device
```

```
$ dd if=/dev/urandom > /dev/sd"XY"

dd: writing to 'standard output': No space left on device
20481+0 records in
20480+0 records out
10485760 bytes (10 MB, 10 MiB) copied, 2.29914 s, 4.6 MB/s
```

The file copy command `cp` can also be used to rewrite the device, because it ignores the type of the destination:

```
$ cp /dev/urandom /dev/sd"XY"

cp: error writing '/dev/sd"XY"': No space left on device
cp: failed to extend '/dev/sd"XY"': No space left on device
```

To show speed and time you can use `pv`:

```
# pv --timer --rate --stop-at-size -s "$(blockdev --getsize64 /dev/sd"XY" )" /dev/zero
> /dev/sd"XY"
```

dd

See also [dd](#) and [Securely wipe disk/Tips and tricks#Wipe a single file](#) .

Warning: There is no confirmation regarding the sanity of this command so **double-check** that the correct drive or partition has been targeted. Make certain that the `of=...` option points to the target drive and not to a system disk.

Zero-fill the disk by writing a zero byte to every addressable location on the disk using the [/dev/zero](#) stream.

```
# dd if=/dev/zero of=/dev/sdX bs=4096 status=progress
```

Or the [/dev/urandom](#) stream:

```
# dd if=/dev/urandom of=/dev/sdX bs=4096 status=progress
```

The process is finished when dd reports `No space left on device` and returns control back:

```
dd: writing to '/dev/sdX': No space left on device
7959553+0 records in
7959552+0 records out
4075290624 bytes (4.1 GB, 3.8 GiB) copied, 1247.7 s, 3.3 MB/s
```

To speed up wiping a large drive, see also:

- [Securely wipe disk/Tips and tricks#dd - advanced example](#) which uses OpenSSL,
- [Securely wipe disk/Tips and tricks#Using a template file](#) which wipes with non-random preset data (e.g. overwrite a whole disk with a single file) but is very fast
- [Dm-crypt/Drive preparation#dm-crypt specific methods](#) which uses dm-crypt.

wipe

A program specialized on wiping files. It is available as part of the [wipe](#) package. To make a quick wipe of a destination, you can use something like:

```
$ wipe -r /path/to/wipe
```

See also [wipe\(1\)](#) . The tool was last updated in 2009. Its [SourceForge page](#) suggests that it is **currently unmaintained**.

shred

[shred](#) (from the [coreutils](#) package) is a Unix command that can be used to securely delete individual files or full devices so that they can be recovered only with great difficulty with specialised hardware, if at all. By default *shred* uses three passes, writing [pseudo-random data](#) to the device during each pass. This can be reduced or increased.

The following command invokes shred with its default settings and displays the progress.

```
# shred -v /dev/sdX
```

Shred can also be used on a single partition, e.g. to wipe the first partition use `shred -v /dev/sdX1` .

Alternatively, shred can be instructed to do only one pass, with entropy from e.g. `/dev/urandom`.

```
# shred --verbose --random-source=/dev/urandom -n1 /dev/sdX
```

Badblocks

The tool **badblocks** from **e2fsprogs** is able to perform destructive read-write test, effectively wiping the device. By default, it performs four passes and can take very long.

```
# badblocks -wsv /dev/device
```

hdparm

Warning: Do not attempt to issue a Secure Erase ATA command on a device connected through USB; see https://ata.wiki.kernel.org/index.php/ATA_Secure_Erase and [this answer](#) for details.

hdparm supports **ATA Secure Erase**, which is functionally equivalent to zero-filling a disk. It is however handled by the hard drive firmware itself, and includes "hidden data areas". As such, it can be seen as a modern-day "low-level format" command. **SSD** drives reportedly achieve factory performance after issuing this command, but may not be sufficiently wiped (see **#Flash memory**).

Some drives support **Enhanced Secure Erase**, which uses distinct patterns defined by the manufacturer. If the output of `hdparm -I` for the device indicates a manifold time advantage for the **Enhanced** erasure, the device probably has a hardware encryption feature and the wipe will be performed to the encryption keys only.

For detailed instructions on using ATA Secure Erase, see [Solid state drive/Memory cell clearing](#) and the [Linux ATA wiki](#).

See also

- [Wipe free space in Linux](#)

Securely Wipe data with dd Short Summary:

Write with zeros/ fill with 0's (ZERO-FILL):

```
$ dd if=/dev/zero of=/dev/sdX bs=4096 status=progress
```

**** alternativeley you can use: ****
sfill - part of secure-delete package
-wipe free space/unallocated non zero
(use **sfill -fllzv <device mnt point>**)

Write with randomness:

```
$ dd if=/dev/random of=/dev/sdX bs=4096 status=progress
```

to zero fill directory
without overwriting whole device
along with its partition tables

Write with pseudo-randomness:

```
$ dd if=/dev/urandom of=/dev/sdX bs=4096 status=progress
```

*****NOTE 4k (4096 bytes) seems to be the default block size on modern hard disks (rather than 512 bytes (drives-pre 2012) by default with no bs=x option)**
And it results in doubling of the copy speed. so it is VERY VERY good practice to use bs=4096

Persistent block device naming

(/dev/disk/: by-id, by-label, by-partlabel, by-partuuid, by-path, by-uuid)

Persistent block device naming

This article describes how to use persistent names for your **block devices**. This has been made possible by the introduction of **udev** and has some advantages over bus-based naming. If your machine has more than one SATA, SCSI or IDE disk controller, the order in which their corresponding device nodes are added is arbitrary. This may result in device names like `/dev/sda` and `/dev/sdb` switching around on each boot, culminating in an unbootable system, kernel panic, or a block device disappearing. Persistent naming solves these issues.

Note:

- Persistent naming has limits that are out-of-scope in this article. For example, while **mkinitcpio** may support a method, **systemd** may impose its own limits (e.g. [FS#42884](#)) on naming it can process during boot.
- This article is not relevant for **LVM** logical volumes as the `/dev/VolumeGroupName/LogicalVolumeName` device paths are persistent.

note: Remember:
#to list all block devices
\$ lsblk

- note:
- by-partuuid is only available to gpt disks
 - by-uuid is available to both mbr and gpt disks (so its preferred)

Contents [hide]

1 Persistent naming methods

1.1 by-label

1.2 by-uuid

1.3 by-id and by-path

1.4 by-partlabel

1.5 by-partuuid

1.6 Static device names with udev

2 Using persistent naming

2.1 fstab

2.2 Kernel parameters

```
$ls /dev/disk
by-id by-label by-partlabel by-partuuid by-path by-uuid
```

#[SUMMARY] : In order to get a devices persistent block device naming information [FOR UUID SPECIFICALLY, PREFERRED]:

```
$ lsblk -f
$ lsblk -f /dev/<sdXY device>
$ lsblk -f | grep sdX
$ ls -l /dev/disk/by-uuid/
$ ls -l /dev/disk/by-uuid/ | grep <sdXY device>

$ ls -l /dev/disk/by-<bydirectory>
$ ls -l /dev/disk/by-<bydirectory> | grep sdX
```

Persistent naming methods

There are four different schemes for persistent naming: **by-label**, **by-uuid**, **by-id** and **by-path**. For those using disks with **GUID Partition Table (GPT)**, two additional schemes can be used **by-partlabel** and **by-partuuid**. You can also use **static device names by using Udev**.

The directories in `/dev/disk/` are created and destroyed dynamically, depending on whether there are devices in them.

Note: Beware that **Disk cloning** creates two different disks with the same name.

The following sections describe what the different persistent naming methods are and how they are used.

The **lsblk** command can be used for viewing graphically the first persistent schemes:

```
$ lsblk -f
```

NAME	FSTYPE	LABEL	UUID	MOUNTPOINT
sda				
└─sda1	vfat		CBB6-24F2	/boot
└─sda2	ext4	Arch Linux	0a3407de-014b-458b-b5c1-848e92a327a3	/
└─sda3	ext4	Data	b411dc99-f0a0-4c87-9e05-184977be8539	/home
└─sda4	swap		f9fe0b69-a280-415d-a03a-a32752370dee	[SWAP]
mmcblk0				
└─mmcblk0p1	vfat		F4CA-5D75	

For those using **GPT**, use the **blkid** command instead. The latter is more convenient for scripts, but more difficult to read.

```
# blkid

/dev/sda1: UUID="CBB6-24F2" TYPE="vfat" PARTLABEL="EFI system partition" PARTUUID="d0d0d110-0a71-4ed6-936a-304969ea36af"
/dev/sda2: LABEL="Arch Linux" UUID="0a3407de-014b-458b-b5c1-848e92a327a3" TYPE="ext4" PARTLABEL="GNU/Linux" PARTUUID="98a81274-10f7-40db-872a-03df048df366"
/dev/sda3: LABEL="Data" UUID="b411dc99-f0a0-4c87-9e05-184977be8539" TYPE="ext4" PARTLABEL="Home" PARTUUID="7280201c-fc5d-40f2-a9b2-466611d3d49e"
/dev/sda4: UUID="f9fe0b69-a280-415d-a03a-a32752370dee" TYPE="swap" PARTLABEL="Swap" PARTUUID="039b6c1c-7553-4455-9537-1befbc9fbc5b"
/dev/mmcblk0: PTUUID="0003e1e5" PTTYPE="dos"
/dev/mmcblk0p1: UUID="F4CA-5D75" TYPE="vfat" PARTUUID="0003e1e5-01"
```

by-label

Almost every **file system type** can have a label. All your volumes that have one are listed in the `/dev/disk/by-label` directory.

```
$ ls -l /dev/disk/by-label

total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 Data -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 Arch\x20Linux -> ../../sda2
```

Most file systems support setting the label upon file system creation, see the **man page** of the relevant `mkfs.*` utility. For some file systems it is also possible to change the labels. Following are some methods for changing labels on common file systems:

swap

```
swapon -L "new label" /dev/XXX using util-linux
```

ext2/3/4

```
e2label /dev/XXX "new label" using e2fsprogs
```

btrfs

```
btrfs filesystem label /dev/XXX "new label" using btrfs-progs
```

reiserfs

```
reiserfstune -l "new label" /dev/XXX using reiserfsprogs
```

jfs

```
jfs_tune -L "new label" /dev/XXX using jfsutils
```

xfs

```
xfs_admin -L "new label" /dev/XXX using xfsprogs
```

fat/vfat

```
fatlabel /dev/XXX "new label" using dosfstools
mlabel -i /dev/XXX ::"new label" using mtools
```

exfat

```
tune.exfat -L "new label" /dev/XXX using exfatprogs
exfatlabel /dev/XXX "new label" using exfatprogs or exfat-utils
```

ntfs

```
ntfslabel /dev/XXX "new label" using ntfs-3g
```

udf

```
udflabel /dev/XXX "new label" using udftools
```

crypto_LUKS (LUKS2 only)

```
cryptsetup config --label="new label" /dev/XXX using cryptsetup
```

The label of a device can be obtained with *lsblk*:

```
$ lsblk -dno LABEL /dev/sda2

Arch Linux
```

Or with *blkid*:

```
# blkid -s LABEL -o value /dev/sda2

Arch Linux
```

Note:

- The file system must not be mounted to change its label. For the root file system this can be accomplished by booting from another volume.
- Labels have to be unambiguous to prevent any possible conflicts.
- Labels can be up to 16 characters long.
- Since the label is a property of the filesystem, it is not suitable for addressing a single RAID device persistently.
- When using encrypted containers with **dm-crypt**, the labels of filesystems inside of containers are not available while the container is locked/encrypted.

by-uuid

UUID [🔗] is a mechanism to give each **filesystem** a unique identifier. These identifiers are generated by filesystem utilities (e.g. `mkfs.*`) when the device gets formatted and are designed so that collisions are unlikely. All GNU/Linux filesystems (including swap and LUKS headers of raw encrypted devices) support UUID. FAT, exFAT and NTFS filesystems do not support UUID, but are still listed in `/dev/disk/by-uuid/` with a shorter UID (unique identifier):

```
$ ls -l /dev/disk/by-uuid/

total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 0a3407de-014b-458b-b5c1-848e92a327a3 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 b411dc99-f0a0-4c87-9e05-184977be8539 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 CBB6-24F2 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 f9fe0b69-a280-415d-a03a-a32752370dee -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 F4CA-5D75 -> ../../mmcblk0p1
```

The UUID of a device can be obtained with *lsblk*:

```
$ lsblk -dno UUID /dev/sda1

CBB6-24F2
```

Or with *blkid*:

```
# blkid -s UUID -o value /dev/sda1

CBB6-24F2
```

The advantage of using the UUID method is that it is much less likely that name collisions occur than with labels. Further, it is generated automatically on creation of the filesystem. It will, for example, stay unique even if the device is plugged into another system (which may perhaps have a device with the same label).

The disadvantage is that UUIDs make long code lines hard to read and break formatting in many configuration files (e.g. **fstab** or **crypttab**). Also every time a volume is reformatted a new UUID is generated and configuration files have to get manually adjusted.

Tip: In case your swap does not have an UUID assigned, you will need to reset it using the **mkswap** utility.

by-id and by-path

by-id creates a unique name depending on the hardware serial number, **by-path** depending on the shortest physical path (according to sysfs). Both contain strings to indicate which subsystem they belong to (i.e. `pci-` for **by-path**, and `ata-` for **by-id**), so they are linked to the hardware controlling the device. This implies different levels of persistence: the **by-path** will already change when the device is plugged into a different port of the controller, the **by-id** will change when the device is plugged into a port of a hardware controller subject to another subsystem. ^[1][🔗] Thus, both are not suitable to achieve persistent naming tolerant to hardware changes.

However, both provide important information to find a particular device in a large hardware infrastructure. For example, if you do not manually assign persistent labels (`by-label` or `by-partlabel`) and keep a directory with hardware port usage, **by-id** and **by-path** can be used to find a particular device. ^[2][🔗] ^[3][🔗]

by-id also creates **World Wide Name** [🔗] links of storage devices that support it. Unlike other **by-id** links, WWNs are fully persistent and will not change depending on the used subsystem.



This article or section needs expansion.

Reason: Explain and provide examples with `/dev/disk/by-id/nvme-eui.*` links^[4][🔗]. Will the WWID of NVME devices always start with `eui.`? (Discuss in [Talk:Persistent block device naming#](#))



Note: **by-id** and **by-path** links can only be considered persistent for disks, not partitions. Partitions will be referenced by their number in the partition table and that can change if the partitions are reordered.

```
$ ls -l /dev/disk/by-id/

total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470 -> ../../sda
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part1 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part2 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part3 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part4 -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 mmc-SD32G_0x0040006d -> ../../mmcblk0
lrwxrwxrwx 1 root root 10 May 27 23:31 mmc-SD32G_0x0040006d-part1 -> ../../mmcblk0p1
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f -> ../../sda
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part1 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part2 -> ../../sda2
```

```
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part3 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part4 -> ../../sda4
```

```
$ ls -l /dev/disk/by-path/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1 -> ../../sda
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part1 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part2 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part3 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part4 -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:07:00.0-platform-rtsx_pci_sdmmc.0 -> ../../mmcblk0
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:07:00.0-platform-rtsx_pci_sdmmc.0-part1 -> ../../mmcblk0p1
```

by-partlabel

Note: This method only concerns disks with [GUID Partition Table \(GPT\)](#) .

GPT partition labels can be defined in the header of the [partition entry](#) on GPT disks.

This method is very similar to the [filesystem labels](#), except the partition labels do not get affected if the file system on the partition is changed.

All partitions that have partition labels are listed in the `/dev/disk/by-partlabel` directory.

```
ls -l /dev/disk/by-partlabel/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 EFI\x20system\x20partition -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 GNU\x2fLinux -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 Home -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 Swap -> ../../sda4
```

The partition label of a device can be obtained with *lsblk*:

```
$ lsblk -dno PARTLABEL /dev/sda1
```

```
EFI system partition
```

Or with *blkid*:

```
# blkid -s PARTLABEL -o value /dev/sda1
```

```
EFI system partition
```

Note:

- GPT partition labels also have to be different to avoid conflicts. To change your partition label, you can use [gdisk](#) or the ncurses-based version [cgdisk](#). Both are available from the [gptfdisk](#) package. See [Partitioning#Partitioning tools](#) .
- According to the specification, GPT partition labels can be up to 72 characters long.

by-partuuid

Like [GPT partition labels](#) , GPT partition UUIDs are defined in the [partition entry](#) on GPT disks.

MBR does not support partition UUIDs, but Linux [\[5\]](#) and software using libblkid [\[6\]](#) (e.g. udev [\[7\]](#)) are capable of generating pseudo PARTUUIDs for MBR partitions. The format is `SSSSSSSS-PP`, where `SSSSSSSS` is a zero-filled 32-bit [MBR disk signature](#), and `PP` is a zero-filled partition number in hexadecimal form. Unlike a regular PARTUUID of a GPT partition, MBR's pseudo PARTUUID can change if the partition number changes.

The dynamic directory is similar to other methods and, like [filesystem UUIDs](#), using UUIDs is preferred over labels.

```
ls -l /dev/disk/by-partuuid/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 0003e1e5-01 -> ../../mmcblk0p1
lrwxrwxrwx 1 root root 10 May 27 23:31 039b6c1c-7553-4455-9537-1befbc9fbc5b -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 7280201c-fc5d-40f2-a9b2-466611d3d49e -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 98a81274-10f7-40db-872a-03df048df366 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 d0d0d110-0a71-4ed6-936a-304969ea36af -> ../../sda1
```

The partition UUID of a device can be obtained with *lsblk*:

```
$ lsblk -dno PARTUUID /dev/sda1
```

```
d0d0d110-0a71-4ed6-936a-304969ea36af
```

Or with *blkid*:

```
# blkid -s PARTUUID -o value /dev/sda1
```

```
d0d0d110-0a71-4ed6-936a-304969ea36af
```

Static device names with udev

See [udev#Setting static device names](#) .

Using persistent naming

There are various applications that can be configured using persistent naming. Following are some examples of how to configure them.

fstab

See the main article: [fstab#Identifying filesystems](#) .

Kernel parameters

To use persistent names in [kernel parameters](#), the following prerequisites must be met. On a standard installation following the installation guide both prerequisites are met:

- You are using an [initramfs](#) image that has [udev](#) in it.
 - For [mkinitcpio](#), enable either the udev or systemd hook in `/etc/mkinitcpio.conf`

The location of the root filesystem is given by the parameter `root` on the kernel command line. The kernel command line is configured from the [boot loader](#), see [Kernel parameters#Configuration](#). To change to persistent device naming, only change the parameters which specify block devices, e.g. `root` and `resume`, while leaving other parameters as is. Various naming schemes are supported:

Persistent device naming [using label](#) and the `LABEL=` format, in this example `Arch Linux` is the LABEL of the root file system.

```
root="LABEL=Arch Linux"
```

Persistent device naming [using UUID](#) and the `UUID=` format, in this example `0a3407de-014b-458b-b5c1-848e92a327a3` is the UUID of the root file system.

```
root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3
```

Persistent device naming [using disk id](#) and the `/dev` path format, in this example `wwn-0x60015ee0000b237f-part2` is the id of the root partition.

```
root=/dev/disk/by-id/wwn-0x60015ee0000b237f-part2
```

Persistent device naming [using GPT partition UUID](#) and the `PARTUUID=` format, in this example `98a81274-10f7-40db-872a-03df048df366` is the PARTUUID of the root partition.

```
root=PARTUUID=98a81274-10f7-40db-872a-03df048df366
```

Persistent device naming [using GPT partition label](#) and the `PARTLABEL=` format, in this example `GNU/Linux` is the PARTLABEL of the root partition.

```
root="PARTLABEL=GNU/Linux"
```

#[SUMMARY] : In order to get a devices persistent block device naming information [FOR UUID SPECIFICALLY, PREFERRED]:

\$ lsblk -f

\$ lsblk -f /dev/<sdXY device>

\$ ls -l /dev/disk/by-uuid/

\$ ls -l /dev/disk/by-uuid/ | grep <sdXY device >

\$ lsblk -f | grep sdX

\$ ls -l /dev/disk/by-<bydirectory>

\$ ls -l /dev/disk/by-<bydirectory> | grep sdX

ex; \$ ls -l /dev/disk/by-partuuid

ex2; \$ ls -l /dev/disk/by-partuuid | grep sda



RED HAT PERSISTENT NAMING

(/dev/disk/ : by-id, by-label, by-partlabel, by-partuuid, by-path, by-uuid)

PERSISTENT NAMING

Red Hat Enterprise Linux provides a number of ways to identify storage devices. It is important to use the correct option to identify each device when used in order to avoid inadvertently accessing the wrong device, particularly when installing to or reformatting drives.

Major and Minor Numbers of Storage Devices (sd<letter(s)>[number(s)])

Storage devices managed by the sd driver are identified internally by a collection of major device numbers and their associated minor numbers. The major device numbers used for this purpose are not in a contiguous range. Each storage device is represented by a major number and a range of minor numbers, which are used to identify either the entire device or a partition within the device. There is a direct association between the major and minor numbers allocated to a device and numbers in the form of sd<Letter(s)>[number(s)] . Whenever the sd driver detects a new device, an available major number and minor number range is allocated. Whenever a device is removed from the operating system, the major number and minor number range is freed for later reuse.

The major and minor number range and associated sd names are allocated for each device when it is detected. This means that the association between the major and minor number range and associated sd names can change if the order of device detection changes. Although this is unusual with some hardware configurations (for example, with an internal SCSI controller and disks that have their SCSI target ID assigned by their physical location within a chassis), it can nevertheless occur. Examples of situations where this can happen are as follows:

- A disk may fail to power up or respond to the SCSI controller. This will result in it not being detected by the normal device probe. The disk will not be accessible to the system and subsequent devices will have their major and minor number range, including the associated sd names shifted down. For example, if a disk normally referred to as sdb is not detected, a disk that is normally referred to as sdc would instead appear as sdb.
- A SCSI controller (host bus adapter, or HBA) may fail to initialize, causing all disks connected to that HBA to not be detected. Any disks connected to subsequently probed HBAs would be assigned different major and minor number ranges, and different associated sd names.
- The order of driver initialization could change if different types of HBAs are present in the system. This would cause the disks connected to those HBAs to be detected in a different order. This can also occur if HBAs are moved to different PCI slots on the system.
- Disks connected to the system with Fibre Channel, iSCSI, or FCoE adapters might be inaccessible at the time the storage devices are probed, due to a storage array or intervening switch being powered off, for example. This could occur when a system reboots after a power failure, if the storage array takes longer to come online than the system take to boot. Although some Fibre Channel drivers support a mechanism to specify a persistent SCSI target ID to WWPN mapping, this will not cause the major and minor number ranges, and the associated sd names to be reserved, it will only provide consistent SCSI target ID numbers.

These reasons make it undesirable to use the major and minor number range or the associated sd names when referring to devices, such as in the /etc/fstab file. There is the possibility that the wrong device will be mounted and data corruption could result.

Occasionally, however, it is still necessary to refer to the sd names even when another mechanism is used (such as when errors are reported by a device). This is because the Linux kernel uses sd names (and also SCSI host/channel/target/LUN tuples) in kernel messages regarding the device.

25.8.2. World Wide Identifier (WWID)

The *World Wide Identifier* (WWID) can be used in reliably identifying devices. It is a persistent, system-independent ID that the SCSI Standard requires from all SCSI devices. The WWID identifier is guaranteed to be unique for every storage device, and independent of the path that is used to access the device.

This identifier can be obtained by issuing a SCSI Inquiry to retrieve the *Device Identification Vital Product Data* (page `0x83`) or *Unit Serial Number* (page `0x80`). The mappings from these WWIDs to the current `/dev/sd` names can be seen in the symlinks maintained in the `/dev/disk/by-id/` directory.

Example 25.4. WWID

For example, a device with a page `0x83` identifier would have:

```
scsi-3600508b400105e210000900000490000 -> ../../sda
```

Or, a device with a page `0x80` identifier would have:

```
scsi-SSEAGATE_ST373453LW_3HW1RHM6 -> ../../sda
```

Red Hat Enterprise Linux automatically maintains the proper mapping from the WWID-based device name to a current `/dev/sd` name on that system. Applications can use the `/dev/disk/by-id/` name to reference the data on the disk, even if the path to the device changes, and even when accessing the device from different systems.

If there are multiple paths from a system to a device, **DM Multipath** uses the WWID to detect this. **DM Multipath** then presents a single "pseudo-device" in the `/dev/mapper/wwid` directory, such as `/dev/mapper/3600508b400105df70000e00000ac0000`.

The command `multipath -l` shows the mapping to the non-persistent identifiers: `Host:Channel:Target:LUN`, `/dev/sd` name, and the `major:minor` number.

```
3600508b400105df70000e00000ac0000 dm-2 vendor,product
[size=20G][features=1 queue_if_no_path][hwhandler=0][rw]
\ round-robin 0 [prio=0][active]
  \ 5:0:1:1 sdc 8:32 [active][undef]
  \ 6:0:1:1 sdg 8:96 [active][undef]
\ round-robin 0 [prio=0][enabled]
  \ 5:0:0:1 sdb 8:16 [active][undef]
  \ 6:0:0:1 sdf 8:80 [active][undef]
```

DM Multipath automatically maintains the proper mapping of each WWID-based device name to its corresponding `/dev/sd` name on the system. These names are persistent across path changes, and they are consistent when accessing the device from different systems.

When the `user_friendly_names` feature (of **DM Multipath**) is used, the WWID is mapped to a name of the form `/dev/mapper/mpathn`. By default, this mapping is maintained in the file `/etc/multipath/bindings`. These `mpathn` names are persistent as long as that file is maintained.



Important

If you use `user_friendly_names`, then additional steps are required to obtain consistent names in a cluster. Refer to the [Consistent Multipath Device Names in a Cluster](#) section in the **DM Multipath** book.

In addition to these persistent names provided by the system, you can also use `udev` rules to implement persistent names of your own, mapped to the WWID of the storage.

25.8.3. Device Names Managed by the `udev` Mechanism in

`/dev/disk/by-*`

The `udev` mechanism consists of three major components:

The kernel

Generates events that are sent to user space when devices are added, removed, or changed.

The `udev`d service

Receives the events.

The `udev` rules

Specifies the action to take when the `udev` service receives the kernel events.

This mechanism is used for all types of devices in Linux, not just for storage devices. In the case of storage devices, Red Hat Enterprise Linux contains `udev` rules that create symbolic links in the `/dev/disk/` directory allowing storage devices to be referred to by their contents, a unique identifier, their serial number, or the hardware path used to access the device.

`/dev/disk/by-label/`

Entries in this directory provide a symbolic name that refers to the storage device by a label in the contents (that is, the data) stored on the device. The **blkid** utility is used to read data from the device and determine a name (that is, a label) for the device. For example:

```
/dev/disk/by-label/Boot
```



Note

The information is obtained from the contents (that is, the data) on the device so if the contents are copied to another device, the label will remain the same.

The label can also be used to refer to the device in `/etc/fstab` using the following syntax:

```
LABEL=Boot
```

`/dev/disk/by-uuid/`

Entries in this directory provide a symbolic name that refers to the storage device by a unique identifier in the contents (that is, the data) stored on the device. The **blkid** utility is used to read data from the device and obtain a unique identifier (that is, the UUID) for the device. For example:

```
UUID=3e6be9de-8139-11d1-9106-a43f08d823a6
```

`/dev/disk/by-id/`

Entries in this directory provide a symbolic name that refers to the storage device by a unique identifier (different from all other storage devices). The identifier is a property of the device but is not stored in the contents (that is, the data) on the devices. For example:

```
/dev/disk/by-id/scsi-3600508e000000000ce506dc50ab0ad05
```

```
/dev/disk/by-id/wmn-0x600508e0000000000ce506dc50ab0ad05
```

The id is obtained from the world-wide ID of the device, or the device serial number. The `/dev/disk/by-id/` entries may also include a partition number. For example:

```
/dev/disk/by-id/scsi-3600508e000000000ce506dc50ab0ad05-part1
```

```
/dev/disk/by-id/wmn-0x600508e0000000000ce506dc50ab0ad05-part1
```

`/dev/disk/by-path/`

Entries in this directory provide a symbolic name that refers to the storage device by the hardware path used to access the device, beginning with a reference to the storage controller in the PCI hierarchy, and including the SCSI host, channel, target, and LUN numbers and, optionally, the partition number. Although these names are preferable to using major and minor numbers or `sd` names, caution must be used to ensure that the target numbers do not change in a Fibre Channel SAN environment (for example, through the use of persistent binding) and that the use of the names is updated if a host adapter is moved to a different PCI slot. In addition, there is the possibility that the SCSI host numbers could change if a HBA fails to probe, if drivers are loaded in a different order, or if a new HBA is installed on the system. An example of by-path listing is:

```
/dev/disk/by-path/pci-0000:03:00.0-scsi-0:1:0:0
```

The `/dev/disk/by-path/` entries may also include a partition number, such as:

```
/dev/disk/by-path/pci-0000:03:00.0-scsi-0:1:0:0-part1
```

25.8.3.1. Limitations of the `udev` Device Naming Convention

The following are some limitations of the `udev` naming convention.

- It is possible that the device may not be accessible at the time the query is performed because the `udev` mechanism may rely on the ability to query the storage device when the `udev` rules are processed for a `udev` event. This is more likely to occur with Fibre Channel, iSCSI or FCoE storage devices when the device is not located in the server chassis.

- The kernel may also send `udev` events at any time, causing the rules to be processed and possibly causing the `/dev/disk/by-*/` links to be removed if the device is not accessible.
- There can be a delay between when the `udev` event is generated and when it is processed, such as when a large number of devices are detected and the user-space `udev` service takes some amount of time to process the rules for each one). This could cause a delay between when the kernel detects the device and when the `/dev/disk/by-*/` names are available.
- External programs such as **blkid** invoked by the rules may open the device for a brief period of time, making the device inaccessible for other uses.

25.8.3.2. Modifying Persistent Naming Attributes

Although `udev` naming attributes are persistent, in that they do not change on their own across system reboots, some are also configurable. You can set custom values for the following persistent naming attributes:

- `UUID` : file system UUID
- `LABEL` : file system label

Because the `UUID` and `LABEL` attributes are related to the file system, the tool you need to use depends on the file system on that partition.

- To change the `UUID` or `LABEL` attributes of an XFS file system, unmount the file system and then use the **xfs_admin** utility to change the attribute:

```
# umount /dev/device
# xfs_admin [-U new_uuid] [-L new_label] /dev/device
# udevadm settle
```

- To change the `UUID` or `LABEL` attributes of an ext4, ext3, or ext2 file system, use the **tune2fs** utility:

```
# tune2fs [-U new_uuid] [-L new_label] /dev/device
# udevadm settle
```

Replace `new_uuid` with the UUID you want to set; for example, `1cdfbc07-1c90-4984-b5ec-f61943f5ea50` . Replace `new_label` with a label; for example, `backup_data` .



Note

Changing `udev` attributes happens in the background and might take a long time. The `udevadm settle` command waits until the change is fully registered, which ensures that your next command will be able to utilize the new attribute correctly.

You should also use the command after creating new devices; for example, after using the **parted** tool to create a partition with a custom `PARTUUID` or `PARTLABEL` attribute, or after creating a new file system.

TIPS & TRICKS

[automatically add users to groups]

This can be achieved using the **/etc/adduser.conf** file in the following way. Edit the file:

sudo nano /etc/adduser.conf

change these lines:

```
# Set this if you want the --add_extra_groups option to adduser to add
# new users to other groups.
# This is the list of groups that new non-system users will be added to
# Default:
#EXTRA_GROUPS="dialout cdrom floppy audio video plugdev users"
# If ADD_EXTRA_GROUPS is set to something non-zero, the EXTRA_GROUPS
# option above will be default behavior for adding new, non-system users
#ADD_EXTRA_GROUPS=1
```

To:

```
# Set this if you want the --add_extra_groups option to adduser to add
# new users to other groups.
# This is the list of groups that new non-system users will be added to
# Default:
EXTRA_GROUPS="audio"
# If ADD_EXTRA_GROUPS is set to something non-zero, the EXTRA_GROUPS
# option above will be default behavior for adding new, non-system users
ADD_EXTRA_GROUPS=1
```

Debconf Module Description

Debian based systems use the debconf database to record configuration choices the user made during the installation of a package. The system uses the stored answers and therefore does not need to query the user again when the package is upgraded or reinstalled at a later time.

The debconf type allows preseeding the database with given answers to allow an unattended package installation or modification of package defaults. The type can optionally also manage the seen flag for an item.

The standard package type uses the responsefile parameter to provide a file with preseeded answers during installation. The debconf type is a more general solution as it allows to update settings without the need to install the package at the same time.

Setup Requirements

This module uses programs provided by the Debian/Ubuntu **debconf-utils** / **debconf** package. Debian assigns the required priority to this package so it should already be installed everywhere.

Usage

```
debconf-show <package> : to find out about valid debconf entries for already installed package.
debconf-get-selections : output all contents of the current debconf database
debconf-get-selections | grep <package name>: output in the correct formatting to use for debconf-set-selections
debconf-set-selections : insert new values into the debconf database from a file or stdin
```

File locations for debconf config file(s):

```
/var/cache/debconf/config.dat
/etc/debconf.conf
```

How to read and insert new values into the debconf database & get valid/available config entries for packages: Debconf Usage

Obtain valid debconf config values in the correct debconf formatting for a given (already previously installed) package : **!Must be from an already installed package with desired dpkg/ncurses/debconf configuration previously selected, to obtain the desired configuration that will be used in future unattended package installations :::**

You will need `debconf-get-selections` on a source system on which you want to read values from the debconf database. It is provided by `debconf-utils` package

```
debconf-get-selections | grep <package-name> : will output in the correct formatting to use for debconf-set-selections
```

Insert new values into the debconf database:

- `echo 'output of debconf-get-selections | grep <package-name>' | sudo debconf-set-selections`
----- OR -----
- `cat << EOF | sudo debconf-set-selections`
iptables-persistent iptables-persistent/autosave_v6 boolean true
iptables-persistent iptables-persistent/autosave_v4 boolean false
EOF

Linux tar Command – How to Archive & Compress Files in Linux

TAR

```
tar -cvf <destinationFileName.tar> <Source>
tar -cvfz <destinationFileName.tgz> <Source>
```

```
tar -cSvf <destinationFileName.tar> <Sparse File Source>
for sparse files use: -S or --sparse options
```



File compression is an essential utility across all platforms. It helps you reduce file size and share files efficiently. And compressed files are also easier to copy to remote servers.

You can also compress older and rarely used files and save them for future use which helps you conserve disk space.

In this post, we'll look at how to compress files with the `tar` command in Linux, along with some examples of `tar` in action.

What is the tar command?

We use the `tar` command to compress and expand files from the command line. The syntax is shown below:

```
tar [flags] destinationFileName Source
tar -cvf <destinationFileName.tar> <Source>
tar -cvfz <destinationFileName.tgz> <Source>
tar -cSvf <destinationFileName.tar> <Sparse File Source>
for sparse files use: -S or --sparse options
```

The `tar` command uses the following flags to customize the command input:

FLAG	EXPLANATION	USAGE
-c	Create a new archive.	We use this flag whenever we need to create a new archive.
-z	Use gzip compression.	When we specify this flag, it means that archive will be created using gzip compression.
-v	Provide verbose output.	Providing the -v flag shows details of the files compressed.
-f	Archive file name.	Archive file names are mapped using the -f flag.
-x	Extract from a compressed file.	We use this flag when files need to be extracted from an archive.

Here, the `-czvf` flags are working as demonstrated before, but the original files are also removed. Once we list the files, we only see the archive.

How to view the contents of an archive

You might need to view the contents of an archive without actually extracting it. You can do this with the `-t` flag.

```
tar -tvf logs_archive.tar.gz
```

In this command, `-t` flag specifies that we need to only view the contents of the archive. `-f` specifies the filename and `-v` displays the detailed contents.

```
zaira@DESKTOP-H7KNNP2:~/zipTutorial$ tar -tvf logs_archive.tar.gz
-rw-r--r-- zaira/zaira 5242880 2021-10-02 11:32 audit.20210110.00.log
-rw-r--r-- zaira/zaira 5242880 2021-10-02 11:32 audit.20210110.01.log
-rw-r--r-- zaira/zaira 5242880 2021-10-02 11:32 audit.20210110.03.log
-rw-r--r-- zaira/zaira 5242880 2021-10-02 11:32 audit.20210110.04.log
-rw-r--r-- zaira/zaira 5242880 2021-10-02 11:32 audit.20210110.05.log
-rw-r--r-- zaira/zaira 5242880 2021-10-02 11:32 audit.log
```

Viewing contents of an archive.

How to extract an archive

To extract files from an archive, you use the `-x` flag like this:

```
tar -xvf logs_archive.tar
tar -xzvf logs_archive.tar.gz
```

Let's break down this command and look into each flag.

- `x` is extracting and archive.
- `z` specifies that the archive is gzip.
- `v` is providing details of the files that have been archived. (verbose)
- `f` is extracting from an archive/file

```
zaira@DESKTOP-H7KNNP2:~/zipTutorial$ tar -xzvf logs_archive.tar.gz
audit.20210110.00.log
audit.20210110.01.log
audit.20210110.03.log
audit.20210110.04.log
audit.20210110.05.log
audit.log
zaira@DESKTOP-H7KNNP2:~/zipTutorial$ ls -lrt
total 30752
-rw-r--r-- 1 zaira zaira 5242880 Oct 2 11:32 audit.log
-rw-r--r-- 1 zaira zaira 5242880 Oct 2 11:32 audit.20210110.00.log
-rw-r--r-- 1 zaira zaira 5242880 Oct 2 11:32 audit.20210110.01.log
-rw-r--r-- 1 zaira zaira 5242880 Oct 2 11:32 audit.20210110.03.log
-rw-r--r-- 1 zaira zaira 5242880 Oct 2 11:32 audit.20210110.04.log
-rw-r--r-- 1 zaira zaira 5242880 Oct 2 11:32 audit.20210110.05.log
-rw-r--r-- 1 zaira zaira 31030 Oct 2 11:49 logs_archive.tar.gz
```

Verbose output due to flag -v.

Extracted files list.

Original archive

Extracting an archive.

Here's a useful tip: commands that take a long time to execute can continue in the background with `&`.

Adding files to an archive and extracting an archive can take a while. To keep the commands running in the background while you keep working, pair the command with `&` like this:

```
tar -xvzf logs_archive.tar.gz &
```

How to search in compressed log files

You might still need to access certain files once they're archived. Luckily, there is a method you can use to search and view compressed log files without decompressing them and compromising disk space.

The command you can use to search in compressed files is `zgrep` :

```
zaira@DESKTOP-H7KNNP2:~/zipTutorial$ more audit.log
#include <syslog.h>

setlogmask (LOG_UPTO (LOG_NOTICE));

openlog ("exampleprog", LOG_CONS | LOG_PID | LOG_NDELAY, LOG_LOCAL1);

syslog (LOG_NOTICE, "Program started by User %d", getuid ());
syslog (LOG_INFO, "A tree falls in a forest");

closelog ();
```

Contents of a file 'audit.log' in an archive.

We can search for a string in an archive using the below command:

```
zgrep -Hna 'string-to-search' compressedFile.tar.gz
```

Let's briefly look at the flags.

- H lists the file name that contains the match.
- n displays the line number that contains the matched string.
- a treats all files as text files.

This is the result:

```
zaira@DESKTOP-H7KNNP2:~/zipTutorial$ zgrep -Hna 'LOG_UPTO' archive.tar.gz
archive.tar.gz:3:setlogmask (LOG_UPTO (LOG_NOTICE));
```

↓
Archive name that contains the match.

↓
Line number 3, as -n flag prints the matching line #.

Wrapping Up

File compression helps us save time and resources when sharing files. Servers are almost always rotating and archiving huge log files.

You can also schedule file compression via `cron` jobs to automate disk cleaning. I highly recommend that you take advantage of this utility.

7z [7zip] (File Archiver and Compressor) Command Examples in Linux

What is 7zip?

7zip is a free and open-source file archiver, similar to WinZip or WinRAR on Windows. It was developed by Igor Pavlov and is available for Windows, Linux, and macOS. One of main advantages of 7zip is its ability to compress files to a high degree, which can save a lot of disk space. It also supports a wide range of file formats, including its own 7z format, as well as ZIP, TAR, and others.

Installing 7zip on Linux

Before we dive into examples, you need to make sure that 7zip is installed on your Linux system. Most Linux distributions include 7zip in their default repositories, so you can use your package manager to install it. For example, on Ubuntu, you can use following command –

```
sudo apt-get install p7zip-full
```

On CentOS or Fedora, you can use –

```
sudo yum install p7zip
```

If 7zip is not available in your distribution's repositories, you can download it from official website and compile it from source.

7zip Command Examples in Linux

Now that you have 7zip installed on your Linux system, let's look at some command examples that you can use to compress and extract files.

Creating a 7z Archive

To create a 7z archive, you can use "7z" command followed by "a" option, which stands for "add." Here's an example –

```
7z a archive.7z file1.txt file2.txt file3.txt
```

This will create a new file called "archive.7z" that contains files "file1.txt", "file2.txt", and "file3.txt". Note that you can add as many files as you want to archive.

Extracting a 7z Archive

To extract a 7z archive, you can use "7z" command followed by "x" option, which stands for "extract." Here's an example –

```
7z x archive.7z
```

This will extract contents of "archive.7z" file to current directory. If you want to extract files to a specific directory, you can use "-o" option followed by path to directory. For example –

```
7z x archive.7z -o /home/user/documents
```

This will extract files to "/home/user/documents" directory.

Compressing with Different Compression Levels

By default, 7zip compresses files with "normal" compression level, which is a good balance between compression ratio and speed. However, if you want to compress files more aggressively or less aggressively, you can use "-m" option followed by compression level. Here are some examples –

```
7z a -m0=lzma2 archive.7z file1.txt file2.txt file3.txt
```

This will use "lzma2" compression method with "fastest" compression level, which will compress files quickly but with a lower compression ratio.

```
7z a -m9=lzma2 archive.7z file1.txt file2
```

This will use "lzma2" compression method with "ultra" compression level, which will compress files more aggressively but with a slower speed. You can experiment with different compression levels to find one that suits your needs.

Compressing Directories

In addition to compressing individual files, you can also compress entire directories using 7zip. To do this, simply specify path to directory instead of file names. For example –

```
7z a -m0=lzma2 archive.7z /home/user/documents/
```

This will compress entire "documents" directory using "lzma2" compression method with "fastest" compression level.

Using Encryption

If you need to encrypt your archives for security reasons, 7zip provides several encryption options. most secure option is AES-256 encryption, which is considered unbreakable by current standards. To use AES-256 encryption, you can use "-p" option followed by a password, and "-mhe=on" option to enable header encryption. For example –

```
7z a -pMyPassword -mhe=on archive.7z file1.txt file2.txt file3.txt
```

This will create a new file called "archive.7z" that is encrypted with AES-256 and protected by password "MyPassword". Note that you should choose a strong password and keep it safe, as it cannot be recovered if you forget it.

Splitting Archives

If you need to split your archives into smaller parts for easier transfer or storage, you can use "-v" option followed by size of each part. For example –

```
7z a -v1m archive.7z file1.txt file2.txt file3.txt
```

This will create a new file called "archive.7z" that is split into 1 MB parts. To extract files from a split archive, you only need to extract first part (e.g. "archive.7z.001"), and 7zip will automatically detect other parts and extract files.

Adding Files to an Existing Archive

If you already have an existing archive and want to add more files to it, you can use "7z" command followed by "u" option, which stands for "update." Here's an example –

```
7z u archive.7z file4.txt file5.txt
```

This will add files "file4.txt" and "file5.txt" to existing "archive.7z" file.

Creating a Compressed Tar Archive

7zip can also be used to create compressed tar archives, which are commonly used in Linux. To create a compressed tar archive, you can use "7z" command followed by "a" option and "-ttar" option to specify archive format. For example –

```
7z a -ttar archive.tar.7z file1.txt file2.txt file3.txt
```

This will create a compressed tar archive called "archive.tar.7z" that contains files "file1.txt", "file2.txt", and "file3.txt".

Extracting a Specific File From an Archive

If you only need to extract a specific file from an archive, you can use "7z" command followed by "-o" option and path to file. For example –

```
7z x archive.7z -o /home/user/documents file2.txt
```

This will extract only file "file2.txt" from "archive.7z" file and save it to "/home/user/documents" directory.

Creating a Password-protected Archive

In addition to encryption, 7zip also allows you to create password-protected archives, which can be useful when you want to prevent unauthorized access to your files. To create a password-protected archive, you can use "-p" option followed by password. For example –

```
7z a -pMyPassword archive.7z file1.txt file2.txt file3.txt
```

This will create a new file called "archive.7z" that is password-protected with password "MyPassword".

Listing Contents of an Archive

If you want to see contents of an archive without extracting them, you can use "7z" command followed by "l" option, which stands for "list." For example –

```
7z l archive.7z
```

This will list contents of "archive.7z" file, including file names, sizes, and compression ratios.

Extracting an Archive with Progress Bar

If you want to see progress of an extraction operation, you can use "-bsp1" option to enable progress bar. For example –

```
7z x -bsp1 archive.7z
```

This will extract contents of "archive.7z" file with a progress bar that shows percentage of extraction progress.

Creating a Compressed Archive with Exclusion

If you want to create a compressed archive that excludes certain files or directories, you can use "-x" option followed by file or directory names. For example –

```
7z a archive.7z * -x!*.log -x!temp/
```

This will create a compressed archive called "archive.7z" that contains all files in current directory except for those with extension ".log" and directory "temp/".

Extracting an Archive with Exclusion

If you want to extract an archive while excluding certain files or directories, you can use "-x" option followed by file or directory names. For example –

```
7z x archive.7z -x!*.log -x!temp/
```

This will extract contents of "archive.7z" file while excluding files with extension ".log" and directory "temp/".

Creating a self-extracting Archive

If you want to create a self-extracting archive that can be run on any system without 7zip installed, you can use "-sfx" option followed by name of executable file to create. For example –

```
7z a -sfx archive.exe file1.txt file2.txt file3.txt
```

This will create a self-extracting archive called "archive.exe" that contains files "file1.txt", "file2.txt", and "file3.txt".

Extracting a self-extracting Archive

To extract a self-extracting archive, you simply need to run executable file. For example –

```
./archive.exe
```

This will extract contents of self-extracting archive to current directory.

These are just a few examples of many ways you can use 7zip commands in Linux. With its versatile features and powerful compression capabilities, 7zip is an essential tool for managing files and archives on your Linux system.

Conclusion

7zip is a powerful and versatile tool that can help you compress and extract files on your Linux system. With examples in this article, you should be able to use 7zip to manage your archives efficiently and securely. Whether you're sending files to someone or backing up your data, 7zip is a reliable and easy-to-use tool that can save you time and disk space.

7z - Linux man page

Name

7z - A file archiver with highest compression ratio

Synopsis

**7z [adeltux] [-] [SWITCH] <ARCHIVE_NAME>
<ARGUMENTS>...**

Description

7-Zip is a file archiver with the highest compression ratio. The program supports 7z (that implements LZMA compression algorithm), ZIP, CAB, ARJ, GZIP, BZIP2, TAR, CPIO, RPM and DEB formats. Compression ratio in the new 7z format is 30-50% better than ratio in ZIP format.

7z uses plugins to handle archives.

Function Letters

a	Add
d	Delete
e	Extract
l	List
t	Test
u	Update
x	eXtract with full paths

Switches

-ai[r[-|0]]{@listfile|!wildcard}
Include archives
-ax[r[-|0]]{@listfile|!wildcard}
eXclude archives
-bd
Disable percentage indicator
-i[r[-|0]]{@listfile|!wildcard}

Include filenames

-l
don't store symlinks; store the files/directories they point to
(CAUTION : the scanning stage can never end because of recursive symlinks like 'ln -s .. ldir')

-m{Parameters}

-mhe=on|off
7z format only : enables or disables archive header encryption
(Default : off)

-mmt[N]
(e.g. -mmt=2)
set number of CPU threads

-o{Directory}
Set Output directory

-p{Password}
Set Password

-r[-|0]
Recurse subdirectories (CAUTION: this flag does not do what you think, avoid using it)

-sfx[{name}]
Create SFX archive

-si
Read data from StdIn (eg: tar cf - directory | 7z a -si directory.tar.7z)

-so
Write data to StdOut (eg: % echo foo | 7z a dummy -tgzip -si -so > /dev/null)

-slt
Sets technical mode for l (list) command

-t{Type}
Type of archive (7z, zip, gzip, bzip2 or tar. 7z format is default)

-v{Size}[b|k|m|g]
Create volumes

-u[-][p#][q#][r#][x#][y#][z#][!newArchiveName]
Update options

-w[path]
Set Working directory

-x[r[-|0]]{@listfile|!wildcard}
Exclude filenames

-y
Assume Yes on all queries

Diagnostics

7-Zip returns the following exit codes:

- 0
Normal (no errors or warnings detected)
- 1
Warning (Non fatal **error**(s)). For example, some files

cannot be read during compressing. So they were not compressed

2

Fatal error

7

Bad command line parameters

8

Not enough memory for operation

255

User stopped the process with control-C (or similar)

Backup and limitations

DO NOT USE the 7-zip format for backup purpose on Linux/Unix because : - 7-zip does not store the owner/group of the file.

On Linux/Unix, in order to backup directories you must use tar : - to backup a directory : `tar cf - directory | 7za a -si directory.tar.7z`
- to restore your backup : `7za x -so directory.tar.7z | tar xf -`

If you want to send files and directories (not the owner of file) to others Unix/MacOS/Windows users, you can use the 7-zip format.

example : `7za a directory.7z directory`

Do not use "-r" because this flag does not do what you think.

Do not use `directory/*` because of ".*" files (example : `"directory/*"` does not match `"directory/.profile"`)

Example 1

7z a -t7z -m0=lzma -mx=9 -mfb=64 -md=32m -ms=on archive.7z dir1

adds all files from directory "dir1" to archive archive.7z using "ultra settings"

-t7z

7z archive

-m0=lzma

lzma method

-mx=9

level of compression = 9 (Ultra)

-mfb=64

number of fast bytes for LZMA = 64

-md=32m

dictionary size = 32 megabytes

-ms=on

solid archive = on

Example 2

```
7z a -sfx archive.exe dir1
```

add all files from directory "dir1" to SFX archive archive.exe
(Remark : SFX archive MUST end with ".exe")

Example 3

```
7z a -mhe=on -pmy_password archive.7z a_directory
```

add all files from directory "a_directory" to the archive
"archive.7z" (with data and header archive encryption on)

See Also

[**7za**\(1\)](#), [**7zr**\(1\)](#), [**bzip2**\(1\)](#), [**gzip**\(1\)](#), [**zip**\(1\)](#)

HTML Documentation

Author

Written for Debian by Mohammed Adnene Trojette.

QUICK RECIPE: create password protected encrypted 7z
archive file, with archive header encryption on.

example:

```
7z a -pMyPassword -mhe=on archive.7z file1.txt file2.txt file3.txt
```

This will create a new file called "archive.7z" that is encrypted with AES-256 and protected by password "MyPassword". With the archive headers encrypted also (-mhe=on option) *Note that you should choose a strong password and keep it safe, as it cannot be recovered if you forget it.

HOW TO REORDER LINUX DRIVE PARTITION NUMBERS (using fdisk)

So you've been messing around with your hard drive partitions and your partition numbers are listed in the wrong order and now you want to know how to fix them so they run sequentially from lowest to highest.

Here's a quick textual example of ordered storage device partition naming to help you understand:

sda1 | sda2 | sda3 | sda4

sda1 | sda5 | sda6 | sda7

And here's an example of unordered storage device partition naming:

sda1 | sda7 | sda5 | sda6

It is very easy to fix partition name ordering with that most feared and deadly disk management tool called fdisk.

Warning: fixing, moving or altering disk partitions can result in data loss. Fdisk and other disk management tools have completely bugged many storage devices when used to correct drive faults. Although you are about to learn something that is usually safe, I must warn you to be cautious and back up irreplaceable data first. You should consider using testdisk if a mistake is made and data is lost.

USING FDISK TO REORDER DRIVE PARTITION LABELS

Firstly, we need to learn the logical name of the disk that houses the unordered partitions name. If you don't already know which drive's partition naming is an unordered mess then you probably don't have anything to fix. But for informational purposes...

#List the partition tables for all devices or the specified devices

```
$ sudo fdisk -l          or          $ fdisk -l <device>
```

That's a lowercase "l".

This will list storage devices in alphanumeric order. Look through the data reported and note the name of any devices that are listed out of numerical order.

TO REORDER LINUX DRIVE PARTITION NUMBERS FOR A SPECIFIC STORAGE DEVICE:

- #use fdisk on a specific storage device

```
$ sudo fdisk <device>
```

- **#then**

- press "x" to enter Expert Mode
- press "f" to fix the drive order
- press "r" to return to main menu
- press "w" to write changes to the disk
- press "q" to quit fdisk

It's that simple.

Replace /dev/sdXY with the name of the device you wish to reorganize partition numbers on.

Now check the drive partition naming order by typing **sudo fdisk -l** into a terminal.

Storage devices can be listed in the file system in many ways..

But here we are using device files under /dev so they are referred to as, for example:

- /dev/sda
- /dev/sdb
- /dev/sdc
- etc....

There are a few ways to discover the logical name of a storage device:

- graphical
- textual

The graphical method is easiest for most people. If you want to try it, install a program such as GParted. There are many textual methods we can use to learn information about storage devices attached to a computer.

- **sudo fdisk -l**
- **lsblk**
- **lsblk -f**

Ubuntu/Linux internet sharing From one interface into another and LAN over Ethernet between two+ machines

Ubuntu/Linux internet sharing From one interface into another and LAN over Ethernet between two+ machines

Its easy to setup a LAN between two Linux machines connected over ethernet & share internet from one into another. If one of those machines, which we will call the **server**, is also connected to the internet via another device (like a wireless card) it will automagically share its internet connection as well. Begin by connecting the **client** and **server** machines via ethernet.

On the **server** machine, click the network icon on the top right and select "Edit Connections > Wired connection 1 > Edit > IPv4 Settings" and change "Method" to "Shared to other computers". Then open the network icon menu again and click "Wired connection 1" to ensure that the connection has been established. Running `ifconfig` in the terminal should show that the wired interface has an ip address.

On the **client** machine, click the network icon on the top right and select "Wired connection 1". All done. Run `ifconfig` on this machine as well to see the ip address you've been assigned.

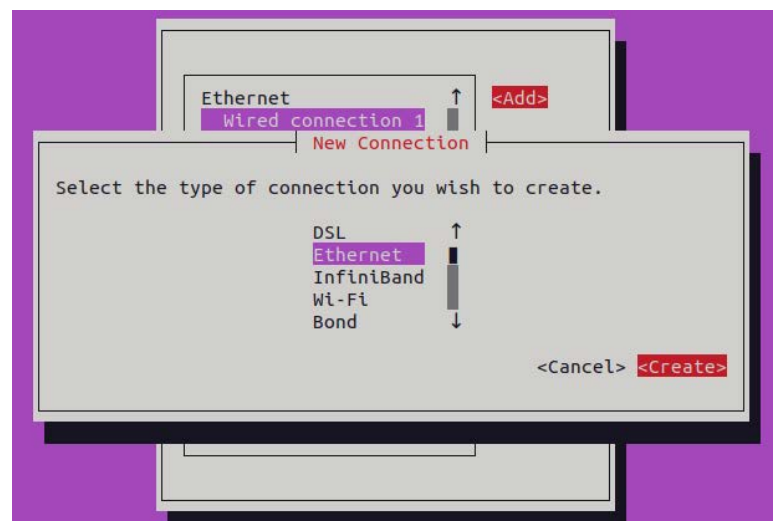
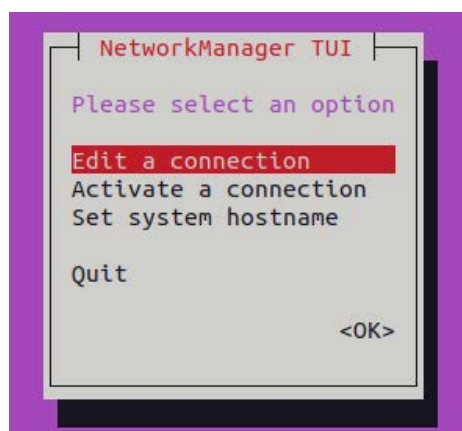
CONNECT TWO LINUX COMPUTERS WITH ETHERNET CABLE & SHARE FILES (VIA SAMBA)

Connect two Linux computers with ethernet cable & share files

If you have a graphical desktop environment installed, you can use graphical tools to do this setup, but they vary according (GNOME and KDE Plasma have them integrated in the settings, while others need an external tool). Using a terminal app is the common ground, so I will proceed with them, but feel free to try out if you can do it with a GUI app if you have such.

in a terminal in both machines you can run the **nmtui** program, (nmtui stands for **N**etwork **M**anager **T**erminal **U**ser **I**nterface). In there you can manage with ease network interfaces and profiles. You move around with the arrows, jump around fields with TAB, and toggle checkmarks with spacebar.

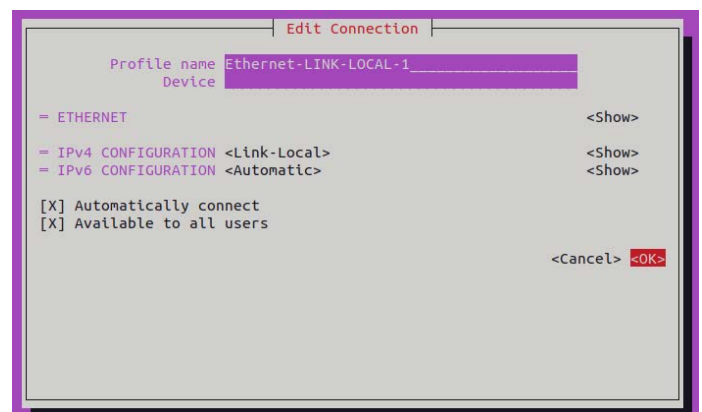
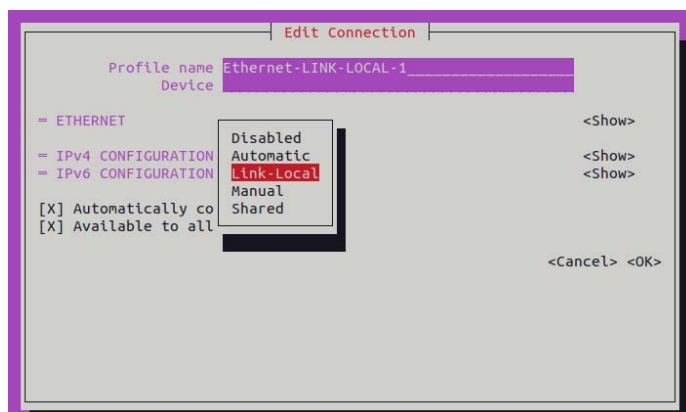
- Select "edit a connection", then "<add>". Select "Ethernet" as the type of connection



- Now you can name your new network profile or leave it as is, it does not matter.

In the "device" field you can either put the MAC address of your network card or leave it blank so it automatically applies to all your ethernet cards (but I assume you only have one so it does not matter that much)

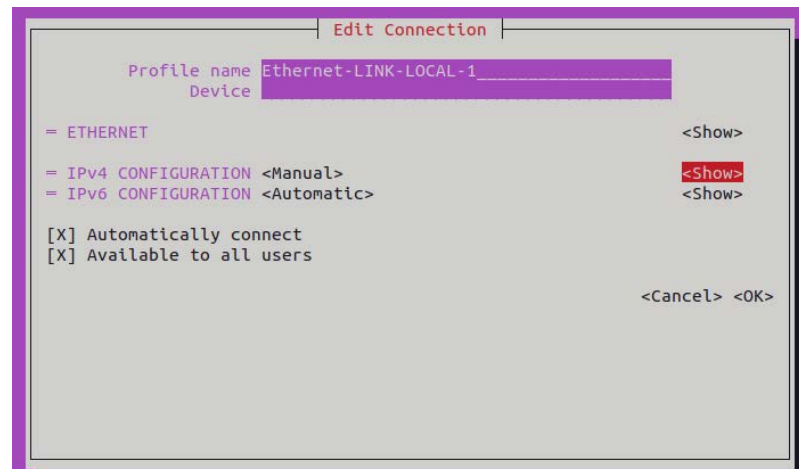
- you can skip the "ETHERNET" and "802.1X SECURITY" sections and go straight to "IPv4". Change the "<automatic>" to either "Link-Local" or "Manual"



The **"LINK LOCAL"** will automatically setup a point-to-point connection between your devices, but you will have little to no control over what IP address each device gets. Use it if you feel more comfortable with it, but you will need to find the IP address of your device each time.
(unless if using SAMBA to share files, since samba automatically brings up samba shared directories in the network)

The **"MANUAL"** option is for manually giving it an IP, and I will explain in more detail:

- After selecting "<manual>" in the IPv4 configuration, go and select "<show>" to see the options.



- In "Addresses" add the IP you want for each computer there, I think you can come up with one that respects the rules of IP addresses. **IPv4 range: 192.168.0.0 -> 192.168.255.255**

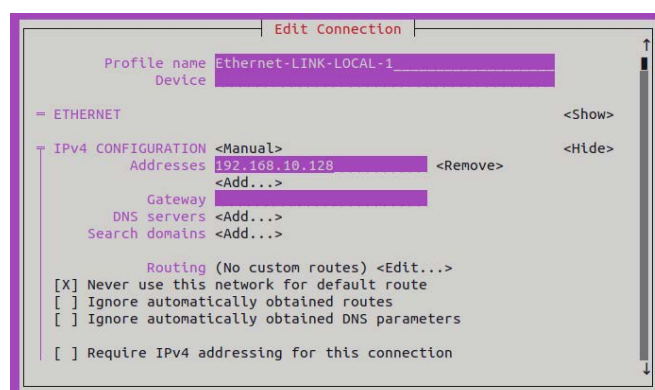
I may suggest using 192.168.10.X/24, with X being any number between 1 and 254.
192.168.10.[1-254]

- You can leave out "gateway", "DNS Servers" and "Search domains" because they are meant for using that connection for internet.

It is not required, but you can check the "Never use this network for default route" option. That is to avoid the computer trying to use that connection for internet.

Also, I like to go and disable the IPv6 configuration. IPv6 is an improvement on IPv4 to handle bigger networks, but we don't need the pros it has right now. Again, it is not mandatory, I just like to do it.

Select "<OK>" and in case the profile hasn't been activated automatically, go to the "Activate a connection" option and select the profile you created.

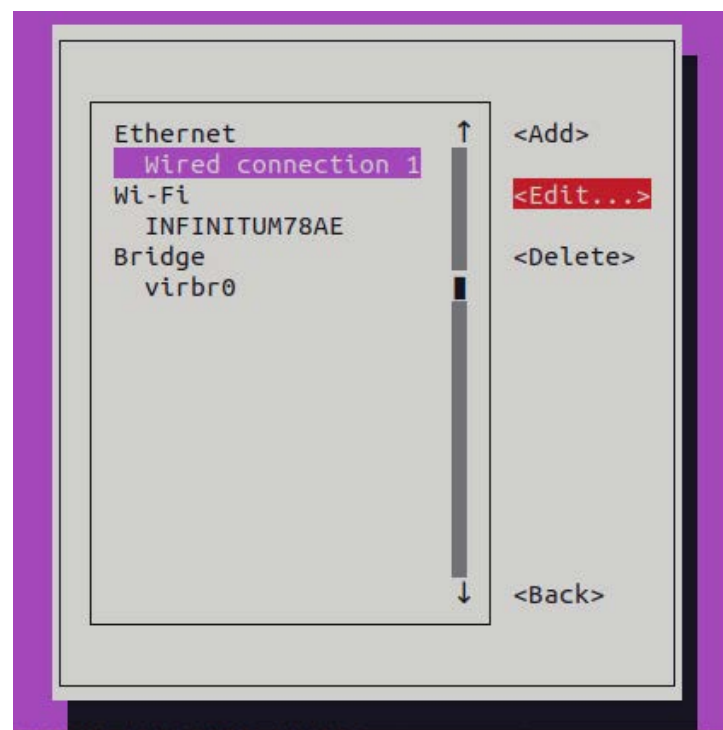
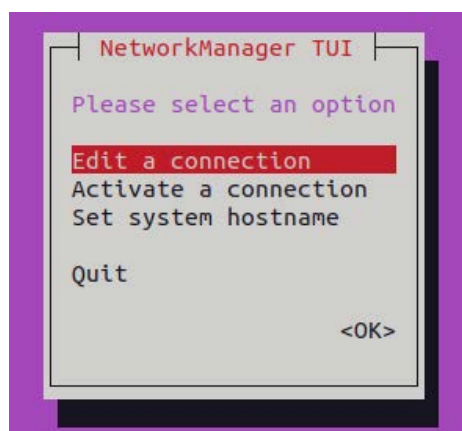


METHOD 2: CONNECT TWO LINUX COMPUTERS WITH ETHERNET CABLE : IPV4 >> SHARED / SHARED TO OTHER COMPUTERS

If you have a graphical desktop environment installed, you can use graphical tools to do this setup, but they vary according (GNOME and KDE Plasma have them integrated in the settings, while others need an external tool). Using a terminal app is the common ground, so I will proceed with them, but feel free to try out if you can do it with a GUI app if you have such.

in a terminal in both machines you can run the **nmtui** program, (nmtui stands for **N**etwork **M**anager **T**erminal **U**ser **I**nterface). In there you can manage with ease network interfaces and profiles. You move around with the arrows, jump around fields with TAB, and toggle checkmarks with spacebar.

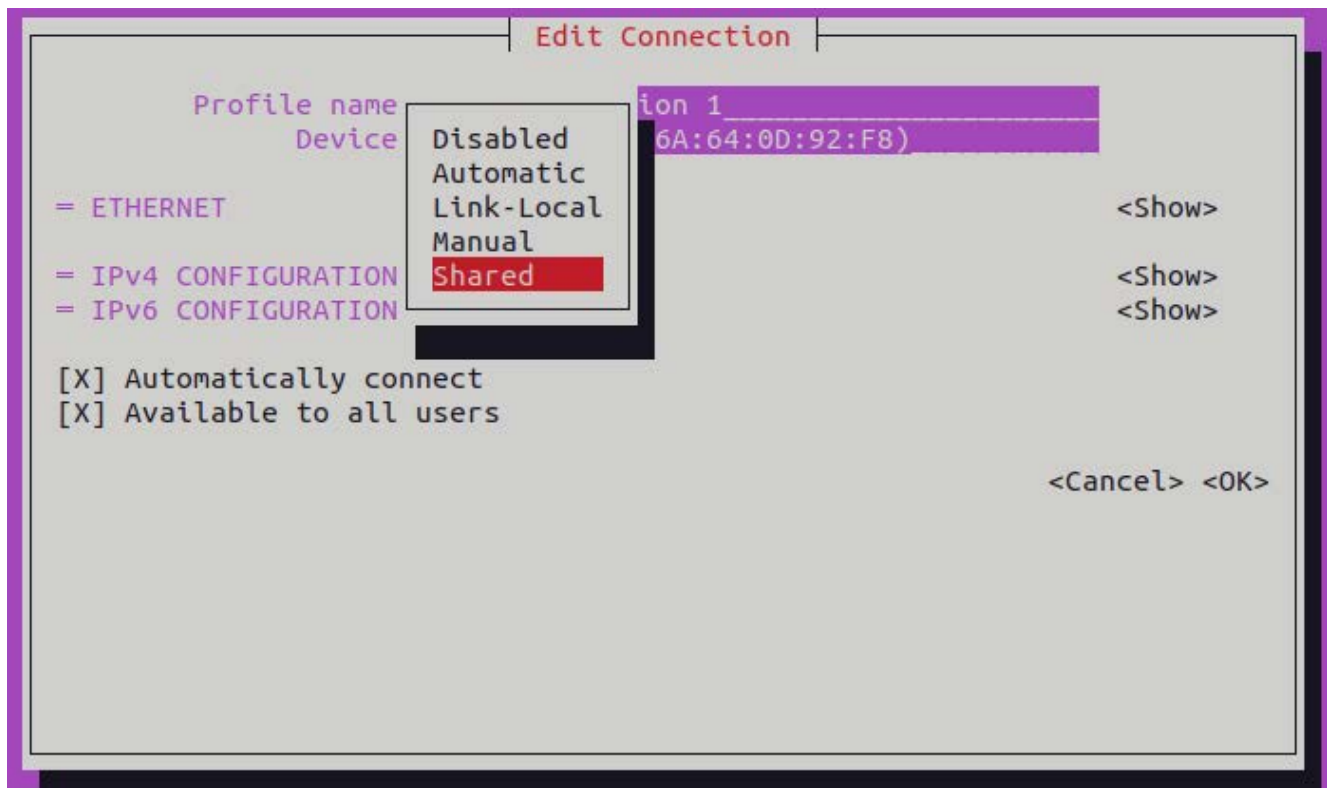
- Select "edit a connection"; choose the ETHERNET connection to share with other computers, then select "<edit>". Select "Ethernet" as the type of connection



- Now you can name your new network profile or leave it as is, it does not matter.

In the "device" field you can either put the MAC address of your network card or leave it blank so it automatically applies to all your ethernet cards (but I assume you only have one so it does not matter that much)

- you can skip the "ETHERNET" and "802.1X SECURITY" sections and go straight to "IPv4". Change the "<automatic>" to "**Shared**"



The **"SHARED"** mode will automatically share internet connectivity to attached devices as well as automatically provide them with an ip address, but you will have little to no control over what IP address each device gets.

you can now share files, internet connectivity & remotely access/control devices under this shared interface with nothing more than an ethernet cable.

How To Change Speed & Duplex of Ethernet Card in Linux with Ethtool Command

NOV 24, 2024

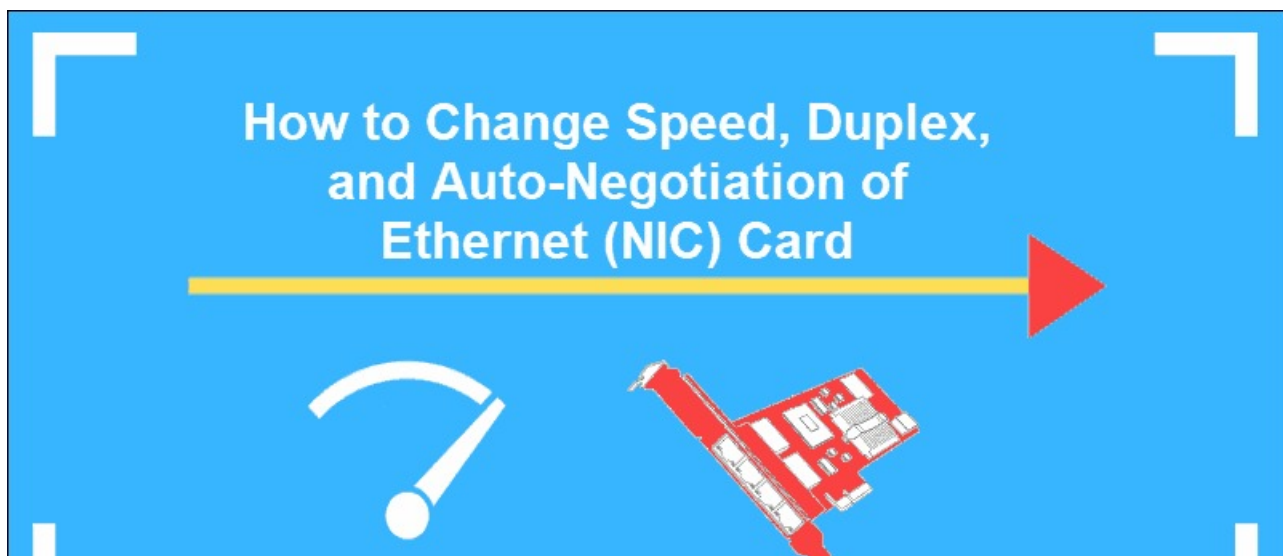
How To Change Speed & Duplex of Ethernet Card in Linux with Ethtool Command

Introduction

The configuration of your Ethernet Card defines how effectively your servers communicate.

It is necessary to understand how **Auto-Negotiation, Speed, and Duplex** settings affect the transfer of data to maintain network connectivity with minimal effort.

This article will show you how to change Speed, Duplex, and Auto-Negotiation settings in Linux (CentOS) with ethtool commands.



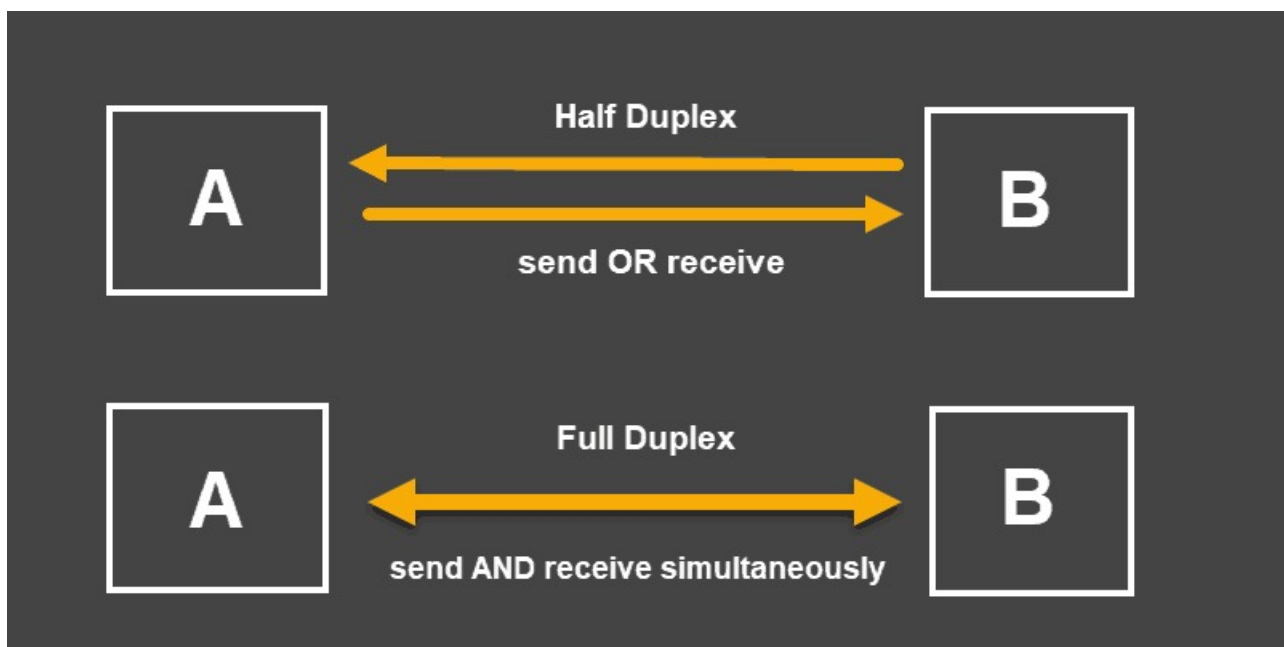
Prerequisites

- Command-line/terminal window
- A user account with root or **sudo** privileges
- The **Ethtool** configuration tool installed

Half Duplex, Full Duplex, and Auto-Negotiation

Half-duplex mode allows a device to either send or receive packets in turn. A device set to this mode cannot perform both actions at the same time.

When a device's mode is at **full-duplex**, it can also send and receive packets simultaneously.



Auto-Negotiation is a mechanism by which a device automatically chooses the best performing transmission mode based on its counterparts' characteristics. It is recommended to keep Auto-Negotiation enabled as it allows devices to choose the most efficient means for the transfer of data.

What is a Duplex Mismatch?

When a device, with enabled auto-negotiation, connects to a device that is not using this signaling method, the process does not work. The end of the connection with an active

auto-negotiation is still able to detect the speed of the other end, but cannot correctly detect the duplex mode. As a rule, the auto-negotiating end of the connection is going to use half-duplex while the other end might be at full-duplex. This situation is considered a **duplex mismatch**.

A duplex mismatch does not stop communication completely. Single packets and small amounts of data do not cause immediate issues. However, when a large amount of data is sent from either end, the speed drops significantly. The connection is working, but the performance is reduced as the [data transfer rate](#) is asymmetrical and might lead to [packet loss](#).

How to Use Ethtool Command to Configure NIC Settings

Ethtool is a Network Interface Card configuration command that allows you to retrieve information and change your [NIC](#) settings. These settings include **Speed, Duplex, Auto-Negotiation**, and many other parameters.

To proceed, you'll need to know the name of your network interface card.

To find the name of your network interface card, run the following command from the command terminal:

```
ifconfig
```

The output provides the name of the device interface card. To learn more about this command, read our guide [How to Install and Use ifconfig](#).

```
[phoenixnap@localhost ~]$ ifconfig -a
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::dcea:a867:893a:9ca0 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:45:31:34 txqueuelen 1000 (Ethernet)
    RX packets 10515 bytes 14502986 (13.8 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2644 bytes 163130 (159.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 20 bytes 1720 (1.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 20 bytes 1720 (1.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

In the above example, the name of the device is **enp0s3**.

Now that you have determined the name of the device, check the current Speed, Auto-Negotiation, and Duplex mode settings with the command: `ethtool devicename`.

In our specific example the command is:

```
ethtool enp0s3
```

The output shows that the current speed is `1000Mb/s`, that the Duplex is at `Full`, and that Auto-Negotiation is turned `on`.

Ethtool Command to Change Ethernet Adapter Settings



Note: The `ethernet` adapter settings cannot be changed for virtualized environments (such as a virtual machine) and on unsupported network drivers.

The `ethtool -s` command can be used to change the current settings by defining the values for `speed`, `duplex`, and `autoneg` in the following format:

```
sudo ethtool -s [device_name] autoneg [on/off] speed [10/100/1000]  
duplex [half/full]
```

For example, to set the speed at **1000Mb/s**, the duplex mode to **full** and the auto-negotiation to **on** the command would be:

```
sudo ethtool -s enp0s3 autoneg on speed 1000 duplex full
```

The **ethtool [device_name]** command is necessary to confirm that the changes have been applied.

Ethtool_opt Variable to Permanently Set Ethtool Command Settings

Changes made with **Ethtool** are by default reverted after a system is re-booted.

To apply custom settings each time a system boots edit the file for the device interface:

```
sudo vi /etc/sysconfig/network-scripts/ifcfg-enp0s3
```

Add the desired values as a line at the end of the file using the following syntax:

```
ETHTOOL_OPTS="speed [100|1000|10000] duplex [half|full] autoneg [on|off]"
```

For example:

```
ETHTOOL_OPTS="speed 1000 duplex full autoneg on"
```

Save the changes and exit the file.

Now the changes are applied after every reboot and are permanent unless the file is altered again.

Conclusion

By following this tutorial, you have successfully changed the settings on your Network Interface Card with **ethtool commands**. You have also gained a better understanding of how Auto-Negotiation and Duplex modes affect server performance.

CHANGE NETWORK INTERFACE NAME

CHANGE NETWORK INTERFACE NAME:

- List available Network Interfaces:

```
$ ip a
```

- Stop Network Manager Service via systemctl:

```
$ systemctl stop NetworkManager
```

- Put interface down:

```
$ ip link set <interface> down
```

- Change interface Name:

```
$ ip link set <interface> name <new interface name>
```

- Bring interface back up:

```
$ ip link set <interface> up
```

- start/restart NetworkManager service

```
$ systemctl start NetworkManager
```

example:

```
sudo systemctl stop NetworkManager
```

```
sudo ip link set wlan0 down
```

```
sudo ip link set wlan0 name wlan00newname
```

```
sudo ip link set wlan00newname up
```

```
sudo systemctl start NetworkManager
```

Add/Del Virtual Network Interface(s) to a Physical NETIFACE (VLAN)

usage examples: Simultaneous use of a Wi-Fi adapter in different modes

- Use one Wi-Fi Card as a Client & Access Point
- Use one Wi-Fi Card as a Client & Monitor Mode
- etc

to create a virtual Network interface(s) in managed mode with different/specific MAC addresses (optional):

```
1  sudo systemctl stop NetworkManager
2  sudo iw dev wlps0 interface add wlan0_sta type managed addr 00:c0:ca:00:00:02
3  sudo systemctl start NetworkManager
```

```
1  sudo systemctl stop NetworkManager
2  sudo iw dev <device> interface add <New Virtdevice Name> type managed
3  sudo systemctl start NetworkManager
```

DELETE a virtual Network interface(s) :

```
1  sudo systemctl stop NetworkManager
2  sudo iw dev <device> del
3  sudo systemctl start NetworkManager
```

```
1  sudo systemctl stop NetworkManager
2  sudo iw <device> del
3  sudo systemctl start NetworkManager
```

Commands To View & Edit Network Information

- iw -

[list wireless devices]

iw dev

[add multiple wlan interfaces on a wireless card, monitor/managed etc]

iw <device> interface add <new dev name> type < monitor, managed etc>

iw dev <device> interface add < new dev name> type <device mode>

iw phy <phy#> interface add <new dev name> type <device mode>

[delete interfaces with iw]

iw <device> del

iw dev <device> del

iw phy <phy#> del

- ip -

[show ip address, MAC & other information of all interfaces]

ip a

ip addr

[show default gateway / route]

ip route

[show information for specific device]

ip a show <device>

ip a show wlan0

[assign specific ip address to interface]*delete use instead of <add>

ip a add <ip_addr/mask> dev <interface>

ex: **ip a add 192.168.1.200/24 dev eth0**

ex: **ip a add 192.168.1.200/255.255.255.0 dev eth0**

[and much more]

- ip link -

[change state of device UP or DOWN]

ip link set dev <device> <up | down>

ip link set <device> <up | down>

[change name of interface]

ip link set <device> down

ip link set <device> name <new name>

ip link set <newly named dev> up

[change MAC address of interface]

ip link set <device> down

ip link set <device> address <XX:YY:ZZ:AA:BB:CC>

ip link set <device> up

Virtual Wi-Fi interfaces for simultaneous use of a Wi-Fi adapter in different modes

Table of contents

1. What is dev and phy for iw command
2. Simultaneous use of a Wi-Fi adapter in different modes
3. Adding an interface in monitor mode
4. How to remove a virtual interface
5. Types of virtual interfaces
6. How to add an AP interface with iw
7. How to use one Wi-Fi card as a client and an Access Point
8. Commands for displaying the names of network interfaces
9. Additional sources

What is dev and phy for iw command

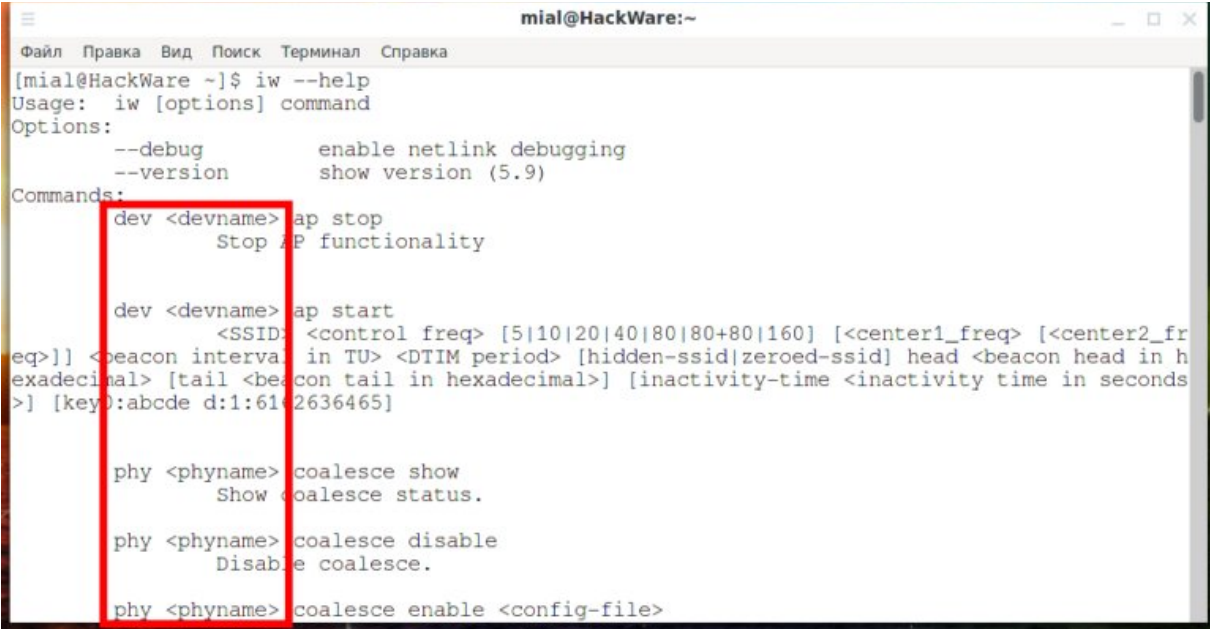
The [Airmo-n-g](#) program for wireless adapters shows information such as **PHY** and **Interface**.

```
mial@HackWare:~  
Файл Правка Вид Поиск Терминал Справка  
[mial@HackWare ~]$ sudo airmo-n-g  


| PHY  | Interface   | Driver    | Chipset                                                      |
|------|-------------|-----------|--------------------------------------------------------------|
| phy0 | wlo1        | iwlwifi   | Intel Corporation Wireless-AC 9560 [Jefferson Peak] (rev 10) |
| phy8 | wlp0s20f0u1 | ath9k_htc | Qualcomm Atheros Communications AR9271 802.11n               |

  
[mial@HackWare ~]$
```

The **iw** program allows you to control the network adapter by referring to it either as a **phy** device or as a **dev** interface. In this case, the set of commands is different.



```
mial@HackWare:~  
Файл Правка Вид Поиск Терминал Справка  
[mial@HackWare ~]$ iw --help  
Usage: iw [options] command  
Options:  
  --debug          enable netlink debugging  
  --version        show version (5.9)  
Commands:  
  dev <devname> ap stop  
                  Stop AP functionality  
  
  dev <devname> ap start  
                  <SSID> <control freq> [5|10|20|40|80|80+80|160] [<center1_freq> [<center2_fr  
eq>]] <beacon interval in TU> <DTIM period> [hidden-ssid|zeroed-ssid] head <beacon head in h  
exadecimal> [tail <beacon tail in hexadecimal>] [inactivity-time <inactivity time in seconds  
>] [key]:abcde d:1:61:2636465]  
  
  phy <phname> coalesce show  
                  Show coalesce status.  
  
  phy <phname> coalesce disable  
                  Disable coalesce.  
  
  phy <phname> coalesce enable <config-file>
```

A **PHY**, an abbreviation for "physical layer", is an electronic circuit, usually implemented as an integrated circuit, required to implement physical layer functions of the OSI model in a network interface controller.

A PHY connects a link layer device (often called MAC as an acronym for medium access control) to a physical medium such as an optical fiber or copper cable. A PHY device typically includes both Physical Coding Sublayer (PCS) and Physical Medium Dependent (PMD) layer functionality.

That is, in Linux, PHY is the number assigned to the device by the PHY driver/framework for accessing it and configuring it at the data link layer.

As for **dev** (device name, interface name), this is the conventional name assigned to the device by the system. There are different naming schemes, so different Linux distributions have different names for network interfaces. If desired, the name of the network interface can be changed.

Simultaneous use of a Wi-Fi adapter in different modes

It would seem, so what? Through **phy**, you can manage settings, the meaning of which is not clear without reading special technical documentation. Using the **dev** interface name, you can control more familiar settings (switch to monitor mode, switch to a specific channel, and so on).

The point is that for wireless adapters, a network interface is not just a name that identifies a device. There are different types of interfaces and interfaces must be added to access some of the capabilities of the wireless adapters. Moreover, one Wi-Fi adapter can have several network interfaces at once that

perform different functions, for example, a combination of interfaces in managed mode and in monitor mode, or a combination of interfaces in managed mode and Access Point mode, while the wireless adapter will simultaneously perform two functions at once! Moreover, different interfaces of one Wi-Fi adapter can be assigned different MAC addresses.

See also:

- [How to find the MAC address and How to find the manufacturer by MAC address](#)
- [How to change MAC address in Linux, how to enable and disable automatic MAC change \(spoofing\) in Linux](#)

Run the following command for a complete listing of the capabilities of all your wireless interfaces:

```
1 | iw list
```

Look for the following lines:

- **software interface modes (can always be added)**
- **valid interface combinations** – that is, possible combinations of interfaces

Example for the first Wi-Fi adapter:

```
1 | software interface modes (can always be added):
2 |     * AP/VLAN
3 |     * monitor
4 | valid interface combinations:
5 |     * #{ managed, P2P-client } <= 2, #{ AP, mesh point, P2P-GO } <= 2,
6 |     total <= 2, #channels <= 1
```

That is, instead of setting the interface from managed mode to monitor mode or launching an Access Point on it, you can add a virtual interface in monitor mode or in AP mode to the existing one.

Also, this wireless adapter can have up to two network interfaces in managed, P2P-client modes, up to two AP, mesh point, P2P-GO interfaces, but in any case, the total number of interfaces cannot be more than two, while all of them have there must be one channel (#channels <= 1).

Next example:

```
1 | software interface modes (can always be added):
2 |     * AP/VLAN
3 |     * monitor
4 | valid interface combinations:
5 |     * #{ managed } <= 1, #{ AP, P2P-client, P2P-GO } <= 1, #{ P2P-device } <= 1,
6 |     total <= 3, #channels <= 2
```

Again, you can add an interface in monitor mode, there can be only one

interface in controlled mode, there can be up to three interfaces in total and they can use up to two different channels (`#channels <= 2`).

Another example:

```
1 software interface modes (can always be added):
2     * AP/VLAN
3     * monitor
4 valid interface combinations:
5     * #{ AP, mesh point } <= 4,
6       total <= 4, #channels <= 1
```

Up to four AP interfaces can be added.

And another example:

```
1 software interface modes (can always be added):
2     * AP/VLAN
3     * monitor
4 valid interface combinations:
5     * #{ AP, mesh point } <= 8,
6       total <= 8, #channels <= 1
```

Up to 8 AP interfaces can be made, but all of them can use one channel.

Adding an interface in monitor mode

My preferred way to get monitor mode is to move the existing wireless interface from managed mode to monitor mode:

```
1 sudo ip link set <INTERFACE> down
2 sudo iw <INTERFACE> set monitor control
3 sudo ip link set <INTERFACE> up
```

As a result, the Wi-Fi card, while it is in monitor mode, loses its ability to connect to the Access Points, that is, there is no Internet connection.

See also: [Linux Wi-Fi Cheat Sheet: Tips and Troubleshooting](#)

This problem can be solved by adding a new interface.

Adding an interface in monitor mode is done as follows:

```
1 sudo iw INTERFACE interface add NEW_INTERFACE type monitor
```

For `INTERFACE`, specify the name of an existing interface, and for `NEW_INTERFACE`, you can specify an arbitrary name.

Optionally, you can assign an arbitrary MAC address to the new interface:

```
1 sudo iw INTERFACE interface add NEW_INTERFACE type monitor addr MAC_ADDRESS
```

Let's see what wireless interfaces are available in the system:

```
1 iw dev
```

We see that the wlp0s20f0u1 Wi-Fi card is connected to a network named Jenechka.

```
mial@HackWare:~  
Файл Правка Вид Поиск Терминал Справка  
[mial@HackWare ~]$ iw dev  
phy#11  
    Interface wlp0s20f0u1  
        ifindex 18  
        wdev 0xb00000001  
        addr 00:c0:ca:96:cf:cb  
        ssid Jenechka  
        type managed  
        channel 8 (2447 MHz), width: 20 MHz, center1: 2447 MHz  
        txpower 36.00 dBm  
  
phy#0  
    Unnamed/non-netdev interface  
        wdev 0x5  
        addr c0:b6:f9:da:af:3e  
        type P2P-device  
        txpower 0.00 dBm  
    Interface wlol1  
        ifindex 3  
        wdev 0x1  
        addr 1e:cb:60:1e:10:e3  
        type managed  
        txpower 22.00 dBm  
        multicast TXQ:  
            qsz-byt  qsz-pkt  flows  drops  marks  overlmt  hashcoltx-bytes tx-packets  
            0        0        0      0      0      0        0        00  
[mial@HackWare ~]$
```

Let's add an interface named wlanmon in monitor mode to wlp0s20f0u1 and set the MAC address on it:

```
1 | sudo iw wlp0s20f0u1 interface add wlanmon type monitor addr 'ca:fe:de:ad:be:ef'
```

Let's take a look at our interfaces again:

```
1 | iw dev
```

```
mial@HackWare:~  
Файл Правка Вид Поиск Терминал Справка  
[mial@HackWare ~]$ sudo iw wlp0s20f0u1 interface add wlanmon type monitor addr 'ca:fe:de:ad:be:ef'  
[mial@HackWare ~]$ iw dev  
phy#11  
Interface wlanmon  
    ifindex 19  
    wdev 0xb00000002  
    addr ca:fe:de:ad:be:ef  
    type monitor  
    txpower 36.00 dBm  
Interface wlp0s20f0u1  
    ifindex 18  
    wdev 0xb00000001  
    addr 00:c0:ca:96:cf:cb  
    ssid Jenechka  
    type managed  
    channel 8 (2447 MHz), width: 20 MHz, center1: 2447 MHz  
    txpower 36.00 dBm  
phy#0  
Unnamed/non-netdev interface  
    wdev 0x5  
    addr c0:b6:f9:da:af:3e  
    type P2P-device  
    txpower 0.00 dBm  
Interface wlol  
    ifindex 3  
    wdev 0x1  
    addr ce:84:7c:fc:41:a5  
    type managed  
    txpower 22.00 dBm  
    multicast TXQ:
```

```

[miial@HackWare ~]$

```

qszt-byt	qszt-pkt	flows	drops	marks	overlmt	nasncoi	tx-bytes	tx-packets
0	0	0	0	0	0	0	0	0

Please note that there is a new **wlanmon** interface.

Let's run **airodump-ng** using the newly created virtual interface:

```

1 | sudo airodump-ng wlanmon

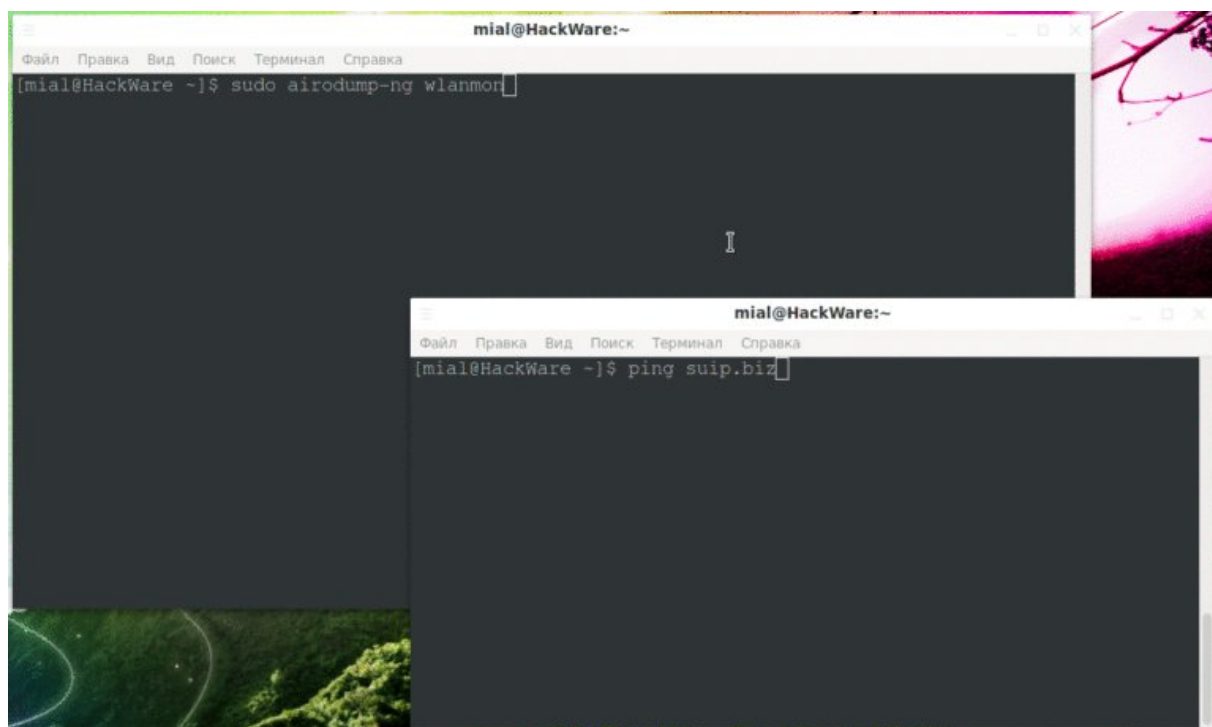
```

And in another window, let's start a ping to make sure that the network connection is still available:

```

1 | ping suip.biz

```



Please note that in spite of the fact that the Wi-Fi adapter needs to be on the same channel to communicate with the AP, nevertheless, the interface in the monitor mode switches over the channels.

Moreover, after stopping the screen recording, **airodump-ng** even managed to catch a handshake.

```

miial@HackWare:~
Файл  Правка  Вид  Поиск  Терминал  Справка
CH  7  ][ Elapsed: 42 s ][ 2021-05-07 09:06 ][ WPA handshake: EC:43:F6:DC:7D:68

```

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSID
F8:1A:67:49:44:62	-60	378	1 0	7	270	WPA2	CCMP	PSK	Old'est_WEB
E4:18:6B:1A:BE:4C	-65	221	0 0	10	270	WPA2	CCMP	PSK	114
EC:43:F6:DC:7D:68	-63	401	833 2	8	130	WPA2	CCMP	PSK	Jenechka
50:FF:20:07:32:58	-84	11	0 0	9	270	WPA2	CCMP	PSK	Keenetic
1C:74:0D:8F:8F:5C	-89	176	0 0	8	270	WPA2	CCMP	PSK	Keenetic-5769
52:FF:20:42:55:E8	-86	14	0 0	8	270	WPA2	CCMP	PSK	<length: 0>
52:FF:20:44:0C:5C	-87	11	0 0	8	130	WPA2	CCMP	PSK	<length: 0>
E4:18:6B:28:A7:7C	-86	153	0 0	8	270	WPA2	CCMP	PSK	Keenetic-2597
50:FF:20:32:55:E8	-86	33	0 0	8	270	WPA2	CCMP	PSK	MIKE WIFI

50:FF:20:34:0C:5C	-87	6	0	0	8	130	WPA2 CCMP	PSK	Guardiwe
C0:4A:00:32:BA:74	-81	3	0	0	9	270	WPA2 CCMP	PSK	polosatik
BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes		
(not associated)	FA:C1:F2:05:42:99	-31	0 - 1	0	4				
(not associated)	B0:B5:C3:E3:F2:8F	-79	0 - 1	0	5				
(not associated)	74:DF:BF:07:A9:43	-85	0 - 1	0	3		pb		
(not associated)	AC:92:32:AD:D6:1C	-90	0 - 1	0	1				

Quitting...

```
[mial@HackWare ~]$
```

I have not noticed that the virtual interface in monitor mode, if used to capture data, interferes with the Internet connection. But if you try to send something from it (for example, to deauthenticate wireless clients), then the Internet connection is terminated. Most likely, this is due to a change in the channel used.

How to remove a virtual interface

To remove the virtual interface, use the command:

```
1 | sudo iw INTERFACE del
```

For example:

```
1 | sudo iw wlanmon del
```

Types of virtual interfaces

Above were shown examples of the command in which we specified the name of the network interface:

```
1 | sudo iw dev DEV-NAME interface add NAME type NAME [mesh_id ] [4addr on|off] [flags *] [addr ]
```

You can also add a virtual interface named PHY:

```
1 | sudo iw phy PHY-NAME interface add NAME type TYPE [mesh_id ] [4addr on|off] [flags *] [addr ]
```

This can probably be useful if you have removed all existing interfaces (if possible).

Available interface types:

- **managed**
- **ibss**
- **monitor**
- **mesh**
- **wds**

Flags are used only for monitor interfaces, valid flags are:

```
1 none:      no special flags
2 fcsfail:   show frames with FCS errors
3 control:   show control frames
4 otherbss:  show frames from other BSSes
5 cook:      use cooked mode
6 active:    use active mode (ACK incoming unicast packets)
7 mumimo-groupid <GROUP_ID>: use MUMIMO according to a group id
8 mumimo-follow-mac <MAC_ADDRESS>: use MUMIMO according to a MAC address
```

The mesh_id is used only for mesh mode.

How to add an AP interface with iw

You may have noticed that there is no **AP** in the above list of valid interface types. This AP type interface will be created automatically when the Access Point is created.

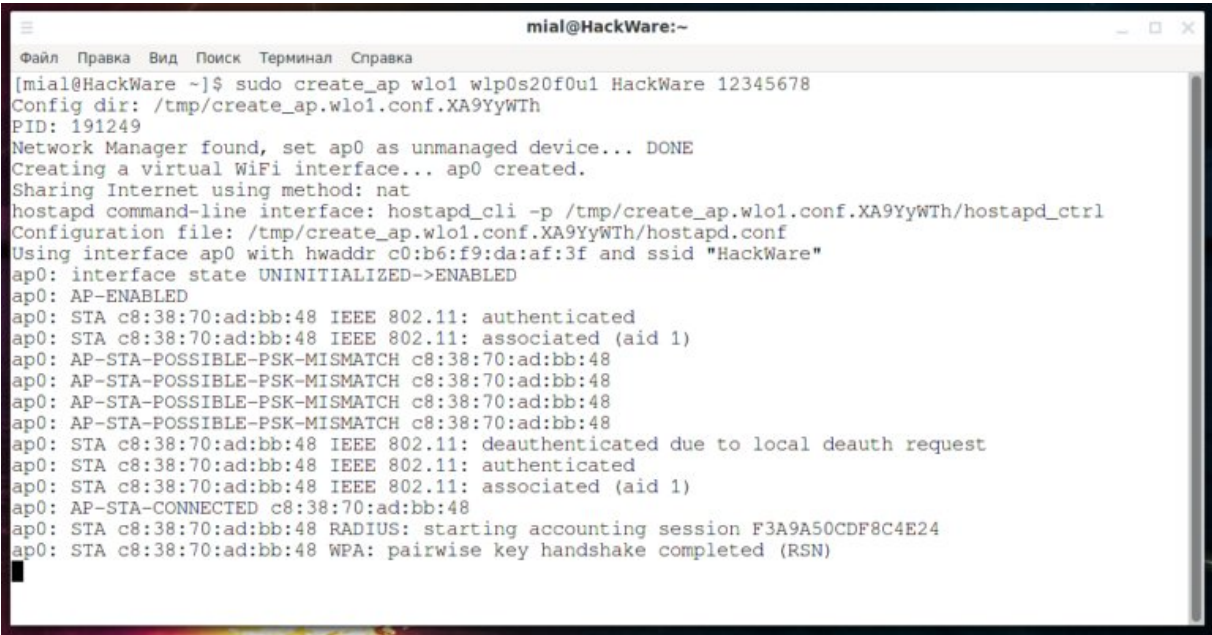
For simplicity, we will use the [create_ap](#) program (the link shows how to install it).

Usage:

```
1 | sudo create_ap [options] AP-INTERFACE INTERFACE-WITH-INTERNET [AP-NAME [PASSWORD]]
```

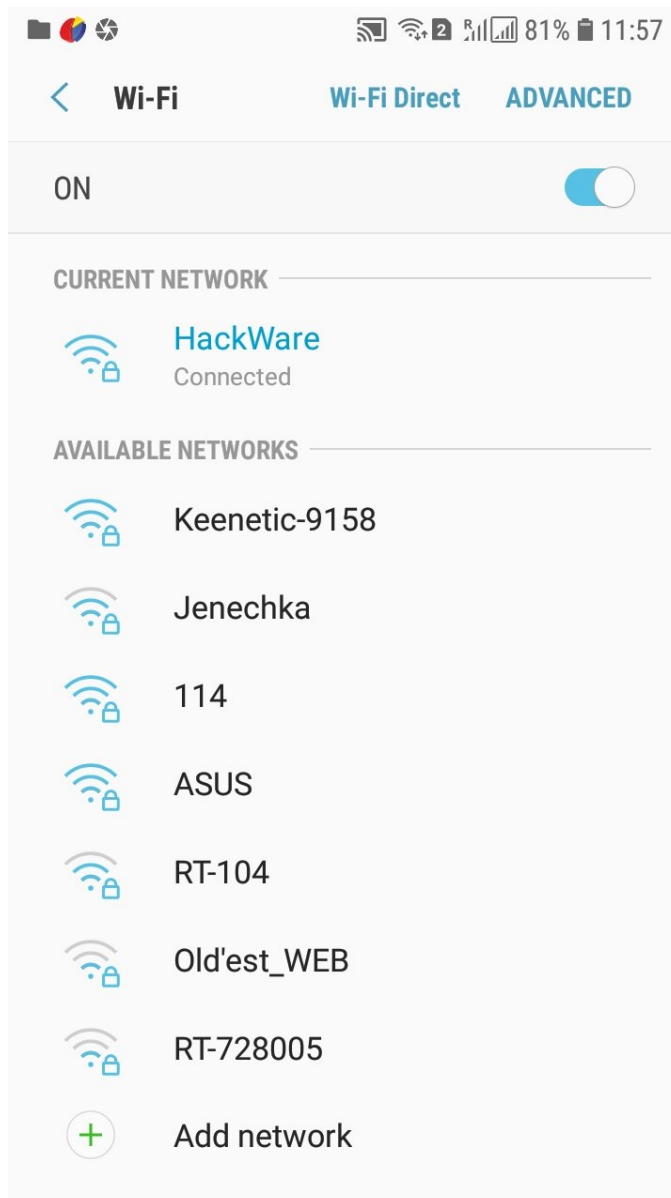
For example, on the **wlo1** interface, I want to create an AP with the name **HackWare** and the password **12345678**, the **wlp0s20f0u1** interface should be used as the Internet source, then the command is as follows:

```
1 | sudo create_ap wlo1 wlp0s20f0u1 HackWare 12345678
```



```
mial@HackWare:~$ sudo create_ap wlo1 wlp0s20f0u1 HackWare 12345678
Config dir: /tmp/create_ap.wlo1.conf.XA9YyWTh
PID: 191249
Network Manager found, set ap0 as unmanaged device... DONE
Creating a virtual WiFi interface... ap0 created.
Sharing Internet using method: nat
hostapd command-line interface: hostapd_cli -p /tmp/create_ap.wlo1.conf.XA9YyWTh/hostapd_ctrl
Configuration file: /tmp/create_ap.wlo1.conf.XA9YyWTh/hostapd.conf
Using interface ap0 with hwaddr c0:b6:f9:da:af:3f and ssid "HackWare"
ap0: interface state UNINITIALIZED->ENABLED
ap0: AP-ENABLED
ap0: STA c8:38:70:ad:bb:48 IEEE 802.11: authenticated
ap0: STA c8:38:70:ad:bb:48 IEEE 802.11: associated (aid 1)
ap0: AP-STA-POSSIBLE-PSK-MISMATCH c8:38:70:ad:bb:48
ap0: AP-STA-POSSIBLE-PSK-MISMATCH c8:38:70:ad:bb:48
ap0: AP-STA-POSSIBLE-PSK-MISMATCH c8:38:70:ad:bb:48
ap0: AP-STA-POSSIBLE-PSK-MISMATCH c8:38:70:ad:bb:48
ap0: STA c8:38:70:ad:bb:48 IEEE 802.11: deauthenticated due to local deauth request
ap0: STA c8:38:70:ad:bb:48 IEEE 802.11: authenticated
ap0: STA c8:38:70:ad:bb:48 IEEE 802.11: associated (aid 1)
ap0: AP-STA-CONNECTED c8:38:70:ad:bb:48
ap0: STA c8:38:70:ad:bb:48 RADIUS: starting accounting session F3A9A50CDF8C4E24
ap0: STA c8:38:70:ad:bb:48 WPA: pairwise key handshake completed (RSN)
```

I can connect to this AP and use the Internet:



On the computer, a new interface of the AP type was created with the name ap0:

```
1 Interface ap0
2   ifindex 21
3   wdev 0x6
4   addr c0:b6:f9:da:af:3f
5   ssid HackWare
6   type AP
7   channel 1 (2412 MHz), width: 20 MHz (no HT), center1: 2412 MHz
8   txpower 22.00 dBm
9   multicast TXQ:
```



```
addr 00:c0:ca:96:c1:cd
ssid Jenechka
type managed
channel 8 (2447 MHz), width: 20 MHz, center1: 2447 MHz
txpower 36.00 dBm

phy#0
Interface ap0
  ifindex 21
  wdev 0x6
  addr c0:b6:f9:da:af:3f
  ssid HackWare
  type AP
  channel 1 (2412 MHz), width: 20 MHz (no HT), center1: 2412 MHz
  txpower 22.00 dBm
  multicast TXQ:
    qsz-byt qsz-pkt flows drops marks overlmt hashcoltx-bytes tx-packets
    0 0 16 0 0 0 0 2202 16
  Unnamed/non-netdev interface
    wdev 0x5
    addr c0:b6:f9:da:af:3e
    type P2P-device
    txpower 0.00 dBm
  Interface wlo1
    ifindex 3
    wdev 0x1
    addr 76:d6:27:66:7d:d6
    type managed
    txpower 22.00 dBm
    multicast TXQ:
      qsz-byt qsz-pkt flows drops marks overlmt hashcoltx-bytes tx-packets
      0 0 0 0 0 0 0 0 0

[mial@HackWare ~]$
```

How to use one Wi-Fi card as a client and an Access Point

The method shown above assumes that you have two network interfaces – and one of them must be a Wi-Fi adapter, and the second can be either a Wi-Fi adapter or a wired connection.

Suppose I do not have a wired connection and I have only one adapter, can one Wi-Fi card be simultaneously used to connect to an Access Point and as an Access Point? Yes, it can. One network interface can simultaneously act as a client and as an AP, that is, it can act as a signal amplifier (wireless repeater).

Let's start by stopping NetworkManager, otherwise in my case I kept getting the error “RTNETLINK answers: Device or resource busy”:

- 1 | `sudo systemctl stop NetworkManager`
- 2 | `sudo airmon-ng check kill`

If you have NetworkManager removed from startup, then you need to start by activating the wireless interface:

- 1 | `sudo ip link set INTERFACE up`

Now we need to create two virtual interfaces in managed mode with different MAC addresses:

- 1 | `sudo iw dev wlp0s20f0u1 interface add wlan0_sta type managed addr 00:c0:ca:00:00:02`
- 2 | `sudo iw dev wlp0s20f0u1 interface add wlan0_ap type managed addr 00:c0:ca:00:00:01`

```

mial@HackWare:~$ sudo iw dev wlp0s20f0u1 interface add wlan0_sta type managed addr 00:c0:ca:00:00:02
mial@HackWare ~]$ sudo iw dev wlp0s20f0u1 interface add wlan0_ap type managed addr 00:c0:ca:00:00:01
mial@HackWare ~]$ iw dev
phy#3
    Interface wlan0_ap
        ifindex 10
        wdev 0x3000000005
        addr 00:c0:ca:00:00:01
        type managed
        txpower 36.00 dBm
    Interface wlan0_sta
        ifindex 9
        wdev 0x3000000004
        addr 00:c0:ca:00:00:02
        type managed
        txpower 36.00 dBm
    Interface wlp0s20f0u1
        ifindex 6
        wdev 0x3000000001
        addr 00:c0:ca:96:cf:cb
        type managed
        txpower 36.00 dBm
phy#0
    Interface wlo1
        ifindex 3
        wdev 0x1
        addr c0:b6:f9:da:af:3e
        type managed
        txpower 0.00 dBm
        multicast TXQ:
            qsz-byt  qsz-pkt  flows  drops  marks  overlmt  hashcol  tx-bytes  tx-packets
            0        0        0      0      0      0        0        0        0
mial@HackWare ~]$

```

One of these interfaces will act as a station, and the other will act as an AP.

Since we disabled NetworkManager, now we need to connect to the remote AP manually.

We create a configuration file:

```
1 | wpa_passphrase Jenechka PASSWORD > wpa_Jenechka.conf
```

We make the connection:

```
1 | sudo wpa_supplicant -i wlan0_sta -c wpa_Jenechka.conf
```

If you want the **wpa_supplicant** process to go to the background, add the **-B** option:

```
1 | sudo wpa_supplicant -B -i wlan0_sta -c wpa_Jenechka.conf
```

Connection takes some time:

```
1 | # sleep 5 # if run in a script, then in this place you need to give time to connect
```

We start the DHCP service to automatically obtain an IP address:

```
1 | sudo dhclient wlan0_sta
```

We look at the state of the interfaces:

```
1 | iw dev
```

```

Файл Правка Вид Поиск Терминал Справка
[mial@HackWare ~]$ iw dev
phy#3
    Interface wlan0_ap
        ifindex 10
        wdev 0x300000005
        addr 00:c0:ca:00:00:01
        type managed
        txpower 36.00 dBm
    Interface wlan0_sta
        ifindex 9
        wdev 0x300000004
        addr 00:c0:ca:00:00:02
        ssid Jenechka
        type managed
        channel 8 (2447 MHz) width: 20 MHz, center1: 2447 MHz
        txpower 36.00 dBm
    Interface wlp0s20f0u1
        ifindex 6
        wdev 0x300000001
        addr 00:c0:ca:96:cf:cb
        type managed
        txpower 36.00 dBm
phy#0
    Interface wlo1
        ifindex 3
        wdev 0x1
        addr c0:b6:f9:da:af:3e
        type managed
        txpower 0.00 dBm
        multicast TXQ:
            qsz-byt qsz-pkt flows drops marks overlmt hashcol tx-bytes tx-packets
            0 0 0 0 0 0 0 0 0
[mial@HackWare ~]$

```

We see that the **wlan0_sta** interface is connected to the AP – this is what we need.

Very important – look at which channel the AP and the client are working on. This channel must be specified in the next command, otherwise nothing will work. In this case, they work on channel 8.

Finally, launch an AP named **HackWare** and password **12345678** on channel **8**:

```
1 | sudo create_ap -c 8 wlan0_ap wlan0_sta HackWare 12345678
```

Everything worked out, and our Wi-Fi signal booster even has a connected client:

```

mial@HackWare:~
Файл Правка Вид Поиск Терминал Справка
[mial@HackWare ~]$ sudo create_ap -c 8 wlan0_ap wlan0_sta HackWare 12345678
Config dir: /tmp/create_ap.wlan0_ap.conf.wIdYpmjf
PID: 1877
Creating a virtual WiFi interface... ap0 created.
Sharing Internet using method: nat
hostapd command-line interface: hostapd_cli -p /tmp/create_ap.wlan0_ap.conf.wIdYpmjf/hostapd_ctrl
Configuration file: /tmp/create_ap.wlan0_ap.conf.wIdYpmjf/hostapd.conf
Using interface ap0 with hwaddr 00:c0:ca:96:cf:cc and ssid "HackWare"
ap0: interface state UNINITIALIZED->ENABLED
ap0: AP-ENABLED
ap0: STA c8:38:70:ad:bb:48 IEEE 802.11: authenticated
ap0: STA c8:38:70:ad:bb:48 IEEE 802.11: associated (aid 1)
ap0: AP-STA-CONNECTED c8:38:70:ad:bb:48
ap0: STA c8:38:70:ad:bb:48 RADIUS: starting accounting session E60EA3FAB27BCDFA
ap0: STA c8:38:70:ad:bb:48 WPA: pairwise key handshake completed (RSN)

```

Commands for displaying the names of network interfaces

A small instruction manual for obtaining information about the network interfaces available in the system.

Show only wireless interfaces:

```
1 | iw dev
```

Output of wireless interfaces along with their PHY, driver and chipset information:

```
1 | sudo airmon-ng
```

Outputting some hardware information:

```
1 | sudo lshw -class network
```

Interface names and descriptions:

```
1 | sudo lshw -class network -short
```

Output of everything that resembles network interfaces (along with virtual ones):

```
1 | ip link show
```

The following way you can see the names of the interfaces and equipment:

```
1 | ls -l /sys/class/net
```



```
mial@HackWare:~  
[mial@HackWare ~]$ ls -l /sys/class/net  
итого 0  
lrwxrwxrwx 1 root root 0 мая 6 18:54 enp3s0 -> ../../devices/pci0000:00/0000:00:1d.6/0000:03:00.0/net/enp3s0  
lrwxrwxrwx 1 root root 0 мая 6 18:54 lo -> ../../devices/virtual/net/lo  
lrwxrwxrwx 1 root root 0 мая 6 19:43 vboxnet0 -> ../../devices/virtual/net/vboxnet0  
lrwxrwxrwx 1 root root 0 мая 7 11:07 wlanmon -> ../../devices/pci0000:00/0000:00:14.0/usb1/1-1/1-1:1.0/net/wlanmon  
lrwxrwxrwx 1 root root 0 мая 6 18:54 wlo1 -> ../../devices/pci0000:00/0000:00:14.3/net/wlo1  
lrwxrwxrwx 1 root root 0 мая 7 08:35 wlp0s20f0u1 -> ../../devices/pci0000:00/0000:00:14.0/usb1/1-1/1-1:1.0/net/wlp0s20f0u1  
[mial@HackWare ~]$
```

This is how you can display the names of interfaces other than virtual ones:

```
1 | find /sys/class/net -type l -not -lname '*virtual*' -printf '%f\n'
```

Please note that the interfaces created in this instruction are not considered virtual in this case, since they are tied to physical hardware.

Interface statistics can be viewed as follows:

```
1 | cat /proc/net/dev
```

END Virtual Wi-Fi interfaces for simultaneous use of a Wi-Fi adapter in different modes